



QUEST BOARD · GUÍA DE ESTUDIO

Tutorial completo de Quests

De cero a entender (y extender) la app: entorno, React, Zustand, event sourcing, persistencia, funcionalidades, Tauri, iOS, sincronización y tests.

Tutorial completo de Quests

- └─ Parte I Fundamentos del entorno
- └─ Parte II React y Zustand
- └─ Parte III Event sourcing y dominio
- └─ Parte IV Persistencia y snapshots
- └─ Parte V Funcionalidades del juego
- └─ Parte VI Interfaz, animaciones e i18n
- └─ Parte VII Tauri, Rust e iOS
- └─ Parte VIII Sincronización
- └─ Parte IX Tests y rendimiento
- └─ Parte X Ejercicios y guía de extensión

Escrito sobre el código del repositorio en el commit **df47a75** (rama `main`), 9 de octubre de 2026.

Pensado para alguien recién graduado en Ingeniería Informática: cada concepto se explica antes de usarlo.

Es una instantánea para estudiar: si algo choca con el código o con `docs/`, mandan ellos.

Índice

Cómo leer este tutorial	3
PARTE I · FUNDAMENTOS DEL ENTORNO	7
Capítulo 1. Instalación y entorno: Node, pnpm, Vite, Tauri, Rust y SQLite	8
Capítulo 2. Arranque de la aplicación	14
PARTE II · REACT Y ZUSTAND	20
Capítulo 3. React básico aplicado al proyecto	21
Capítulo 4. Zustand y el store global	27
PARTE III · EVENT SOURCING Y DOMINIO	32
Capítulo 5. Event sourcing	33
Capítulo 6. EventBody, GameEvent y versionado	38
PARTE IV · PERSISTENCIA Y SNAPSHOTS	45
Capítulo 7. EventStore, SQLite y localStorage	46
Capítulo 8. Proyección completa e incremental	50
Capítulo 9. Snapshots, Immer y rendimiento	55
PARTE V · FUNCIONALIDADES DEL JUEGO	60
Capítulo 10. Modelo de quests y condiciones	61
Capítulo 11. XP, niveles y rangos	66
Capítulo 12. Acciones y flujo completo de usuario	69
Capítulo 13. Todas las funcionalidades de features	73
PARTE VI · INTERFAZ, ANIMACIONES E i18N	83
Capítulo 14. i18n, sonidos, animaciones y CSS	84
PARTE VII · TAURI, RUST E IOS	92
Capítulo 15. Tauri, Rust, permisos y SQLite nativo	93
Capítulo 16. Interfaz móvil e iOS	98
PARTE VIII · SINCRONIZACIÓN	102
Capítulo 17. Sincronización con Google Drive	103
PARTE IX · TESTS Y RENDIMIENTO	111
Capítulo 18. Tests, debugging y arquitectura	112
PARTE X · EJERCICIOS Y GUÍA DE EXTENSIÓN	119
Capítulo 19. Ejercicios prácticos por capítulo	120
Capítulo 20. Glosario y diagramas Mermaid	122

Cómo leer este tutorial

Quests es una aplicación de escritorio (macOS y Windows) y de iPhone que convierte tareas en *quests* de estilo JRPG: aceptas una misión, avanzas sus objetivos, la reportas y ganas experiencia (XP), oro y botín. Por debajo hay bastante ingeniería: un modelo de datos basado en **eventos inmutables**, una **proyección** que calcula el estado, persistencia en **SQLite** a través de **Tauri** (Rust), sincronización entre equipos por **Google Drive** y una interfaz en **React** con animaciones de **GSAP** y **Motion**, en español y japonés.

Este documento está pensado para alguien que acaba de terminar Ingeniería Informática: sabe programar, conoce estructuras de datos y bases de datos, quizá ha hecho algo de web, pero no tiene por qué conocer React, Zustand, Tauri ni el patrón *event sourcing*. Por eso cada capítulo sigue el mismo orden:

1. **El concepto**, explicado en general, sin el proyecto.
2. **Cómo se aplica en Quests**, con el código real (simplificado solo donde se indica).
3. **Lo que suele confundir**, en recuadros de «Cuidado».
4. **Ejercicios** para fijarlo.

Los recuadros

Idea clave. Lo que tienes que llevarte del apartado aunque olvides el resto.

Cuidado. Un error típico, una trampa del entorno o una norma del proyecto que no se puede saltar.

En el código. Dónde está exactamente lo que se explica (archivo y símbolo).

Para recién graduados. Conexión con lo que viste en la carrera (bases de datos, sistemas operativos, redes, ingeniería del software).

Ejercicio. Algo para hacer con el repositorio delante.

Mapa: lo pedido y dónde está

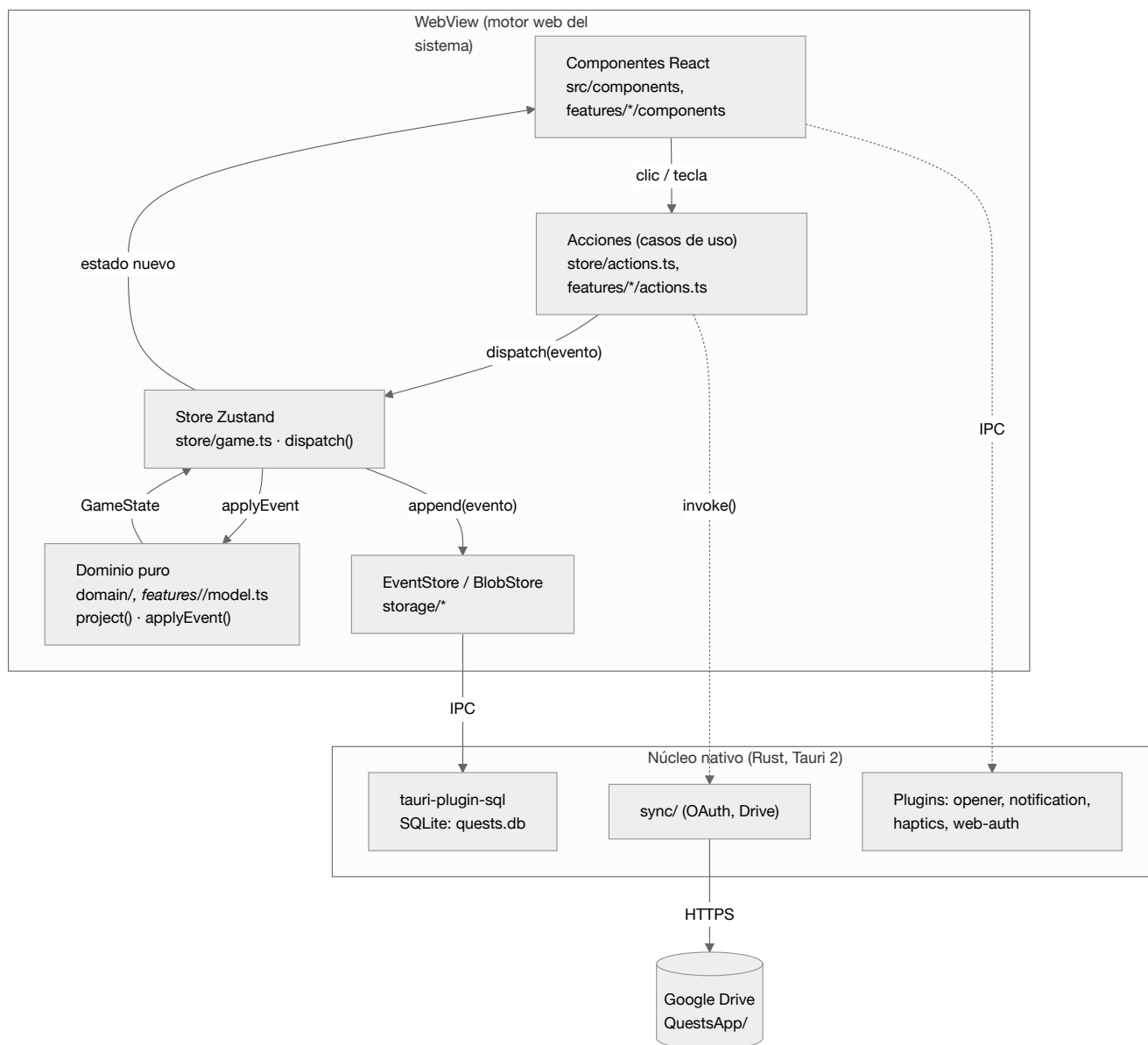
El tutorial cubre los veinte puntos pedidos, agrupados en diez partes. El orden de los capítulos sigue el de las partes; los dos únicos puntos que cambian de sitio son la interfaz móvil (que va con Tauri, porque el iPhone es otro destino de Tauri) y la sincronización (que tiene su propia parte).

Punto pedido	Capítulo	Parte
1. Instalación y entorno: Node, pnpm, Vite, Tauri, Rust y SQLite	1	I
2. Arranque de la aplicación	2	I
3. React básico aplicado al proyecto	3	II

Punto pedido	Capítulo	Parte
4. Zustand y el store global	4	II
5. Event sourcing	5	III
6. <code>EventBody</code> , <code>GameEvent</code> y versionado	6	III
7. <code>EventStore</code> , <code>SQLite</code> y <code>localStorage</code>	7	IV
8. Proyección completa e incremental	8	IV
9. Snapshots, Immer y rendimiento	9	IV
10. Modelo de quests y condiciones	10	V
11. XP, niveles y rangos	11	V
12. Acciones y flujo completo de usuario	12	V
13. Todas las funcionalidades de <code>features</code>	13	V
14. <code>i18n</code> , sonidos, animaciones y CSS	14	VI
15. Tauri, Rust, permisos y <code>SQLite</code> nativo	15	VII
17. Interfaz móvil e iOS	16	VII
16. Sincronización con Google Drive	17	VIII
18. Tests, debugging y arquitectura	18	IX
19. Ejercicios prácticos por capítulo	19 (y al final de cada capítulo)	X
20. Glosario y diagramas Mermaid	20 (y diagramas en todos los capítulos)	X

La app en una página

Antes de entrar en detalles, conviene tener la foto completa. Este diagrama es el «mapa del metro» del proyecto; cada caja tiene su capítulo.



Lee el diagrama de arriba abajo: **el usuario hace algo** → **una acción valida y emite un evento** → **el store lo aplica al estado con funciones puras del dominio** → **React repinta** → **el evento se guarda en SQLite**. Toda la app es ese ciclo repetido. Si al acabar el tutorial puedes explicar este diagrama a otra persona con ejemplos del código, has cumplido el objetivo.

Convenciones del proyecto que verás en todo el texto

- **Idioma:** código, comentarios, documentación y mensajes de commit van en español.
- **Una carpeta por funcionalidad:** `src/features/<nombre>` con su `README.md`. Para tocar una funcionalidad basta con su `README`.
- **La guía de trabajo** está en `docs/AGENTES.md` y es obligatoria. Los mecanismos generales, en `docs/COMO-FUNCIONA.md`; la arquitectura, en `docs/INFORME-TECNICO.md`; las decisiones, en `docs/decisions/` (ADR); los procedimientos, en `docs/runbooks/`.
- **Los datos del propietario son sagrados:** nunca se borra `quests.db` ni su `localStorage`. Para probar desde cero se usa el navegador (`pnpm dev`) con una copia apartada.

Cuidado. Este tutorial es una **instantánea** del código en el commit `df47a75`. Algunos valores cambian con el tiempo (por ejemplo `PROJECTION_VERSION`, que aquí vale 14). La fuente de verdad son el código y `docs/`; si encuentras una diferencia, mandan ellos.

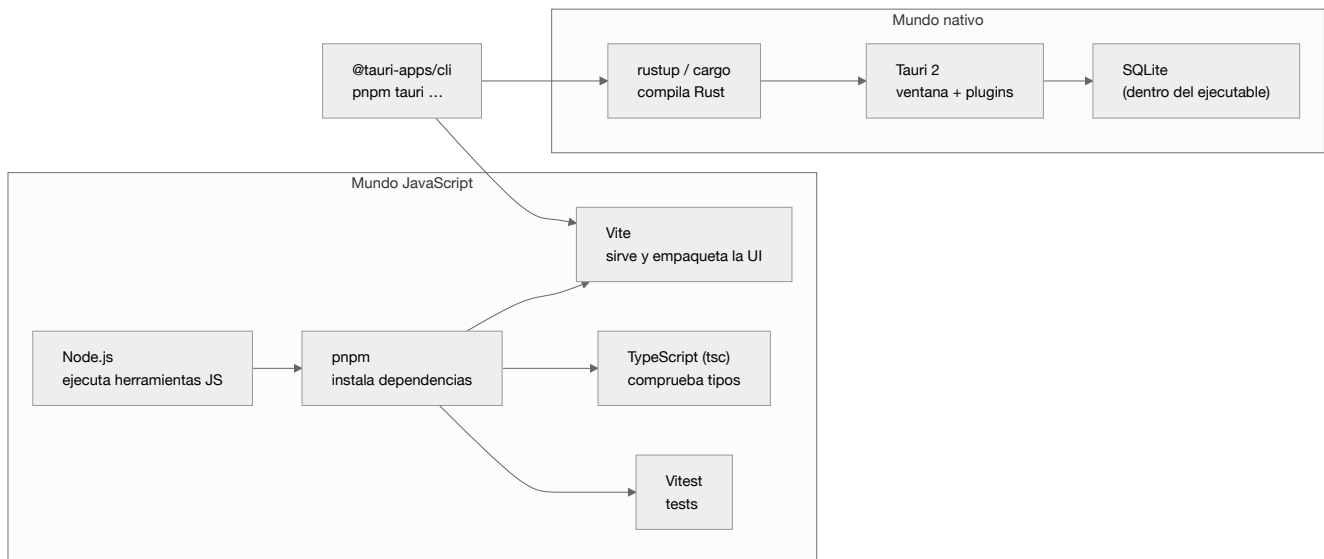


Parte I · Fundamentos del entorno

Qué herramientas necesitas, para qué sirve cada una y qué ocurre exactamente cuando arrancas la app.

Capítulo 1. Instalación y entorno: Node, pnpm, Vite, Tauri, Rust y SQLite

Quests no es «una web» ni «un programa nativo»: es las dos cosas a la vez. Por eso el entorno tiene dos mitades, una del mundo JavaScript (Node, pnpm, Vite, TypeScript) y otra del mundo nativo (Rust, Cargo, las herramientas de compilación de cada sistema). Tauri es el pegamento.



1.1 Node.js: JavaScript fuera del navegador

JavaScript nació para ejecutarse dentro del navegador. **Node.js** es un *runtime* que lo ejecuta en tu sistema operativo, como ejecutarías Python o Java. En Quests, Node **no** forma parte de la app final: la app corre dentro de un WebView. Node se usa para las **herramientas de desarrollo**: el servidor de Vite, el compilador de TypeScript, los tests, el script de documentación (`scripts/docs.mjs`) y la CLI de Tauri.

- **Versión:** la CI usa Node **24** (`.github/workflows/ci.yml`). Usa la misma o una cercana.
- **Instalación en macOS:** `brew install node` o, mejor si trabajas con varios proyectos, un gestor de versiones como `fnm` o `nvm`.
- **Comprobación:** `node --version`.

Para recién graduados. Piensa en Node como la JVM de las herramientas web: el código del proyecto (React, el dominio) no «corre en Node» en producción, pero todo el *tooling* sí.

1.2 pnpm: el gestor de paquetes

Un proyecto JavaScript depende de cientos de paquetes. `package.json` declara las **dependencias directas** con rangos de versión (`"react": "^19.1.0"` acepta cualquier `19.x ≥ 19.1.0`), y el **lockfile** (`pnpm-lock.yaml`) congela la versión exacta de cada paquete, directo o indirecto, para que todos los equipos instalen lo mismo.

Quests usa **pnpm** en lugar de npm. La diferencia práctica: pnpm guarda cada versión de cada paquete una sola vez en un almacén global del disco y crea **enlaces** dentro de `node_modules`. Instala más rápido, ocupa menos y es estricto: un paquete solo puede importar lo que declara (no lo que «se cuelga» por ser dependencia de otra).

- **Versión:** la CI usa pnpm **11.8.0**.
- **Instalación:** `npm install -g pnpm`, `brew install pnpm` o `corepack enable` (Corepack viene con Node).
- **Instalar el proyecto:** `pnpm install`. En la CI se usa `pnpm install --frozen-lockfile`, que **falla** si `package.json` y el lockfile no cuadran en lugar de actualizar el lockfile por su cuenta.

Las dependencias de Quests, agrupadas por función:

Grupo	Paquetes	Para qué
Interfaz	<code>react</code> , <code>react-dom</code>	Componentes y pintado
Estado	<code>zustand</code> , <code>immer</code>	Store global; copia selectiva del estado
Animación	<code>motion</code> , <code>gsap</code>	Transiciones y secuencias encadenadas
Idiomas	<code>i18next</code> , <code>react-i18next</code>	Español y japonés
Tauri (lado JS)	<code>@tauri-apps/api</code> , <code>@tauri-apps/plugin-sql</code> , <code>-notification</code> , <code>-opener</code> , <code>-haptics</code>	Hablar con el núcleo Rust
Fuentes	<code>@fontsource/*</code>	Tipografías locales, sin internet
Desarrollo	<code>vite</code> , <code>@vitejs/plugin-react</code> , <code>typescript</code> , <code>vitest</code> , <code>happy-dom</code> , <code>@tauri-apps/cli</code> , <code>@types/*</code>	Compilar, probar, empaquetar

Los **scripts** de `package.json` son atajos que ejecutas con `pnpm <nombre>`:

```
"scripts": {
  "dev": "vite",
  "build": "tsc && vite build",
  "tauri": "tauri",
  "test": "vitest run",
  "docs:index": "node scripts/docs.mjs",
  "docs:check": "node scripts/docs.mjs --check",
  "iphone": "bash scripts/iphone.sh"
}
```

1.3 TypeScript

TypeScript es JavaScript con **tipos estáticos**. El navegador no lo entiende: Vite le quita los tipos al servir el código (muy rápido, sin comprobar nada) y `tsc` (el compilador oficial) **comprueba** los tipos sin generar archivos (`npx tsc --noEmit`). Por eso son dos pasos distintos y por eso `pnpm build` hace `tsc && vite build`: si los tipos fallan, no hay build.

En Quests los tipos no son decoración: la unión de eventos (`EventBody`), los diccionarios de traducción (si falta una clave en japonés, **no compila**) y los estados de las quests dependen de ellos. Lo verás en los capítulos 6 y 14.

1.4 Vite: servidor de desarrollo y empaquetador

Vite cumple dos papeles:

1. **En desarrollo** (`pnpm dev`), es un servidor web en `http://localhost:1420` que sirve los archivos fuente transformados al vuelo y hace **recarga en caliente** (HMR, *Hot Module Replacement*): cambias un componente y se actualiza en la página sin perder el estado.
2. **En producción** (`pnpm build`), **empaqueta** todo en archivos optimizados dentro de `dist/`, que Tauri incrusta en el ejecutable.

La configuración (`vite.config.ts`) tiene tres cosas importantes:

```
server: {
  port: 1420,
  strictPort: true,           // si 1420 está ocupado, falla en vez de usar otro
  watch: { ignored: ["**/src-tauri/**"] }, // no recargar al compilar Rust
},
plugins: [react(), menuCharacters(), temporalHeroes()],
define: { __APP_VERSION__: JSON.stringify(process.env.npm_package_version ?? "dev") },
```

- **Puerto fijo 1420.** Tauri abre su ventana apuntando a esa URL (`devUrl` en `tauri.conf.json`), así que no puede cambiar. Si está ocupado, lo más probable es que la app ya esté abierta: **úsala** en vez de arrancar otra copia.
- **Plugins propios** (`menuCharacters`, `temporalHeroes`): el código del navegador no puede listar una carpeta de `public/`, así que Vite la lee al compilar y la expone como un **módulo virtual** (`import list from "virtual:menu-characters"`). Así basta con dejar una imagen en `public/menu/` para tener un personaje nuevo en el menú.
- `define`: sustituye `__APP_VERSION__` por la versión de `package.json` (la usa la pantalla de recuperación).

Cuidado. Los archivos estáticos van en `public/`, **nunca** en `dist/`: `dist/` se borra y se regenera en cada build.

1.5 Rust y Cargo

Rust es el lenguaje del núcleo nativo de Tauri: abre la ventana, carga los plugins, habla con SQLite, con el llavero del sistema y con Google. **Cargo** es su gestor de paquetes y de compilación (el equivalente a `pnpm + tsc` en el mundo Rust); las dependencias (llamadas *crates*) se declaran en `src-tauri/Cargo.toml` y se congelan en `src-tauri/Cargo.lock`.

- **Instalación:** con **rustup**, el instalador oficial: `curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh`. La CI usa la versión estable (`dtolnay/rust-toolchain@stable`).
- **Comprobación:** `rustc --version` y `cargo --version`.
- **Dependencias del sistema:**
 - **macOS:** las herramientas de línea de comandos de Xcode (`xcode-select --install`); para iPhone, Xcode completo (capítulo 16).
 - **Windows:** *Microsoft C++ Build Tools* (con «Desarrollo para el escritorio con C++») y *WebView2* (ya viene en Windows 10/11 actualizados).

Crates más relevantes de `Cargo.toml`:

Crate	Para qué
<code>tauri</code> (2)	El framework
<code>tauri-plugin-sql</code> con <code>sqlite</code>	SQLite desde JavaScript (sobre <code>sqlx</code>)
<code>tauri-plugin-opener</code> , <code>-notification</code> , <code>-haptics</code>	Abrir <code>tel:/mailto:/https:</code> , avisos, vibración
<code>tauri-plugin-web-auth</code> (propio, en <code>src-tauri/plugins/</code>)	Inicio de sesión de Google en iOS
<code>reqwest</code> , <code>tokio</code>	HTTP asíncrono hacia Google
<code>keyring</code>	Guardar el <i>refresh token</i> en el llavero del sistema
<code>serde</code> , <code>serde_json</code>	Serializar a/desde JSON (todo lo que cruza a JavaScript)
<code>sha2</code> , <code>base64</code> , <code>rand</code> , <code>url</code>	Utilidades para OAuth y los binarios

Cuidado. En el Mac puede haber **dos Rust**: el de Homebrew (`/opt/homebrew/bin`) y el de `rustup` (`~/ .cargo/bin`). El de Homebrew no sabe compilar para iOS. Pon `~/ .cargo/bin` delante en el `PATH` o desinstala el de Homebrew (lo explica `docs/runbooks/compile-ios.md`).

1.6 Tauri 2

Tauri permite escribir la interfaz con tecnologías web y empaquetarla como app nativa. A diferencia de **Electron**, **no incluye un navegador**: usa el motor web del sistema (WKWebView en macOS e iOS, WebView2 en Windows). Por eso la app ocupa megas y no cientos de megas, y por eso hay que probar en los dos motores.

La CLI de Tauri está como dependencia de desarrollo (`@tauri-apps/cli`), así que se ejecuta con `pnpm tauri <orden>`:

Orden	Qué hace
<code>pnpm tauri dev</code>	Arranca Vite (<code>beforeDevCommand: "pnpm dev"</code>), compila Rust y abre la ventana nativa apuntando a <code>localhost:1420</code> , con recarga en caliente
<code>pnpm tauri build</code>	<code>pnpm build</code> (frontend a <code>dist/</code>), compila Rust en <i>release</i> , incrusta <code>dist/</code> y genera el instalador (<code>.dmg</code> , <code>.msi</code> , <code>.exe</code>)
<code>pnpm tauri build --debug --no-bundle</code>	Ejecutable de prueba sin instalador (más rápido)
<code>pnpm tauri ios dev / ios build</code>	iPhone y simulador (capítulo 16)
<code>pnpm tauri icon <png></code>	Genera los iconos de todos los tamaños

La carpeta `src-tauri/` es el proyecto Rust:

src-tauri/	
Cargo.toml, Cargo.lock	dependencias Rust
build.rs	script de compilación (incrusta la credencial de Google)
tauri.conf.json	ventana, CSP, comandos de build, identificador
tauri.ios.conf.json	lo propio de iOS (bundle ID)
capabilities/default.json	permisos del WebView
src/main.rs, src/lib.rs	punto de entrada y registro de plugins/comandos
src/sync/	OAuth y Google Drive
plugins/web-auth/	plugin propio (Swift) para iOS
gen/apple/	proyecto de Xcode generado
icons/	iconos

1.7 SQLite

SQLite es una base de datos relacional **embebida**: no hay servidor, la base es un archivo y la biblioteca va dentro del programa. En Quests **no tienes que instalar nada**: tauri-plugin-sql la compila dentro del ejecutable. La base vive en la carpeta de datos de la app, según el identificador `com.quests.app`:

- macOS: `~/Library/Application Support/com.quests.app/quests.db`
- Windows: `%APPDATA%\com.quests.app\`

Para **inspeccionarla** sirve el cliente de línea de comandos `sqlite3` (en macOS viene de serie), siempre con consultas de solo lectura:

```
sqlite3 ~/Library/Application\ Support/com.quests.app/quests.db \
"SELECT json_extract(body, '$.type') AS tipo, count(*) FROM events GROUP BY tipo;"
```

Cuidado. `quests.db` es el **progreso real** del propietario. Nunca se borra, se edita ni se «arregla» a mano. Si necesitas datos vacíos, usa el navegador (`pnpm dev`) con tu propio `localStorage` (capítulo 2).

En el navegador (`pnpm dev`) no hay SQLite: el mismo contrato se cumple sobre `localStorage` (eventos) e `IndexedDB` (archivos). Lo verás en el capítulo 7.

1.8 Editor y extensiones

El repositorio recomienda (`.vscode/extensions.json`) las extensiones de **Tauri** y **rust-analyzer** para VS Code. Con TypeScript, VS Code ya trae el servicio de lenguaje. Un consejo práctico: activa «format on save» y abre la carpeta raíz del repositorio, no `src/`, para que el editor encuentre `tsconfig.json`.

1.9 La credencial de Google (opcional)

La sincronización necesita un cliente OAuth de Google en `src-tauri/google-client.json`, que **no** está en el repositorio (lo excluye `.gitignore`). Sin él, la app compila y funciona igual; la sincronización sale como «sin configurar». `build.rs` lee el archivo (o las variables `QUESTS_GOOGLE_CLIENT_ID` y `QUESTS_GOOGLE_CLIENT_SECRET`) y lo incrusta al compilar. Lo verás en el capítulo 17.

1.10 Lista de comprobación

```
node --version      # 24.x
pnpm --version      # 11.x
rustc --version     # estable
cargo --version
pnpm install        # en la raíz del repositorio
npx tsc --noEmit    # sin errores
pnpm test           # en verde
pnpm dev            # http://localhost:1420 en el navegador
pnpm tauri dev      # ventana nativa (la primera compilación de Rust tarda minutos)
```

Ejercicios

Ejercicio 1.1. Abre `package.json` y `pnpm-lock.yaml`. Busca `zustand` en los dos. ¿Qué rango declara `package.json` y qué versión exacta fija el lockfile? ¿Qué pasaría en la CI si cambias el rango sin regenerar el lockfile?

Ejercicio 1.2. Ejecuta `pnpm dev` dos veces en dos terminales. ¿Qué hace la segunda y por qué? Relaciónalo con `strictPort` y con `devUrl` de `tauri.conf.json`.

Ejercicio 1.3. En `src-tauri/Cargo.toml`, localiza `[profile.release]` y `[profile.release.build-override]`. Explica con tus palabras para qué sirve cada opción y por qué las macros no se reducen con `strip` (pista: el comentario lo cuenta).

Ejercicio 1.4. Añade un archivo `.webp` cualquiera a `public/menu/` con `pnpm dev` en marcha (y quítalo después). ¿Qué ocurre en la página? Encuentra en `vite.config.ts` el código responsable.

Capítulo 2. Arranque de la aplicación

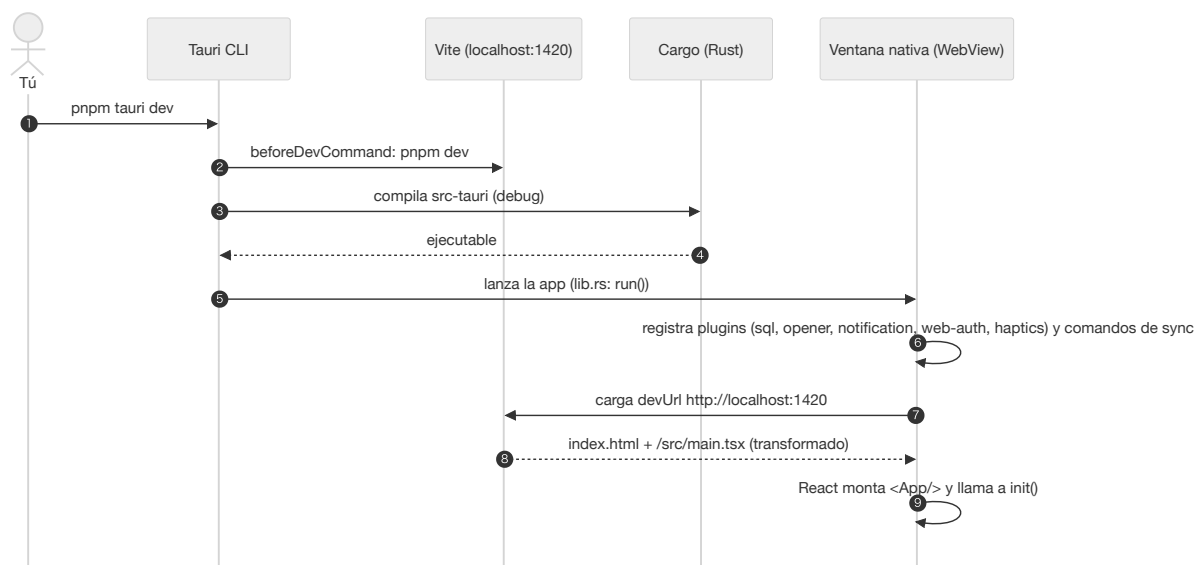
En este capítulo seguimos el camino completo desde que escribes `pnpm tauri dev` hasta que ves el tablón con las quests. Entender el arranque es entender la arquitectura.

2.1 Tres formas de ejecutar Quests

Modo	Orden	Dónde corre la UI	Dónde van los datos	Sincroniza
Navegador	<code>pnpm dev</code>	Tu navegador	<code>localStorage</code> + <code>IndexedDB</code>	No
Nativo de desarrollo	<code>pnpm tauri dev</code>	Ventana Tauri (WebView del sistema)	SQLite (<code>quests.db</code>)	Sí, si hay credencial
Nativo empaquetado	<code>pnpm tauri build</code>	Ventana Tauri, con <code>dist/</code> incrustado	SQLite	Sí

El modo **navegador** es el que más usarás para desarrollar la interfaz: es el mismo código, arranca al instante y no toca los datos del propietario. El nativo hace falta cuando tocas Rust, un plugin, un permiso, la CSP o SQLite.

2.2 Qué pasa con `pnpm tauri dev`



En Rust, el punto de entrada es mínimo: `main.rs` llama a `quests_lib::run()` y `lib.rs` construye la app:

```

tauri::Builder::default()
    .plugin(tauri_plugin_sql::Builder::default().build())
    .plugin(tauri_plugin_opener::init())
    .plugin(tauri_plugin_notification::init())
    .plugin(tauri_plugin_web_auth::init())
    .plugin(tauri_plugin_haptics::init())
    .manage(sync::SyncState::default())
    .invoke_handler(tauri::generate_handler![
        sync::sync_configured, sync::sync_signed_in, sync::sync_sign_in,
        sync::sync_sign_out, sync::sync_account,
        sync::drive_list, sync::drive_download, sync::drive_upload,
    ])
    .run(tauri::generate_context!())
    .expect("error while running tauri application");

```

`generate_context!()` lee `tauri.conf.json` en tiempo de compilación (ventana de 1280 × 780, mínimo 1024 × 680, tema oscuro, CSP...).

2.3 Del HTML a React

`index.html` es casi vacío: un `<div id="root">` y un `<script type="module" src="/src/main.tsx">`. Todo lo demás lo pinta React. `main.tsx` hace, en este orden:

```

import "@fontsource/cormorant-garamond/latin-600.css"; // fuentes locales (solo latín)
// ... más fuentes
import "./i18n"; // inicializa i18next (idioma guardado o el del sistema)
import "./styles/theme.css"; // tokens de color
import "./styles/app.css";
import App from "./App";
import { ErrorBoundary } from "./features/recovery";

ReactDOM.createRoot(document.getElementById("root") as HTMLElement).render(
  <React.StrictMode>
    <ErrorBoundary>
      <App />
    </ErrorBoundary>
  </React.StrictMode>,
);

```

- `React.StrictMode`: en desarrollo, React monta los componentes **dos veces** y ejecuta los efectos dos veces para destapar efectos mal escritos. Lo verás con `init()`.
- `ErrorBoundary`: si un componente lanza un error al pintarse, en vez de dejar la ventana en negro sale una pantalla de recuperación (`features/recovery`).

2.4 `init()`: abrir los datos y calcular el estado

`App` llama a `init()` del store en su primer efecto:

```

useEffect(() => { init(); }, [init]);

```

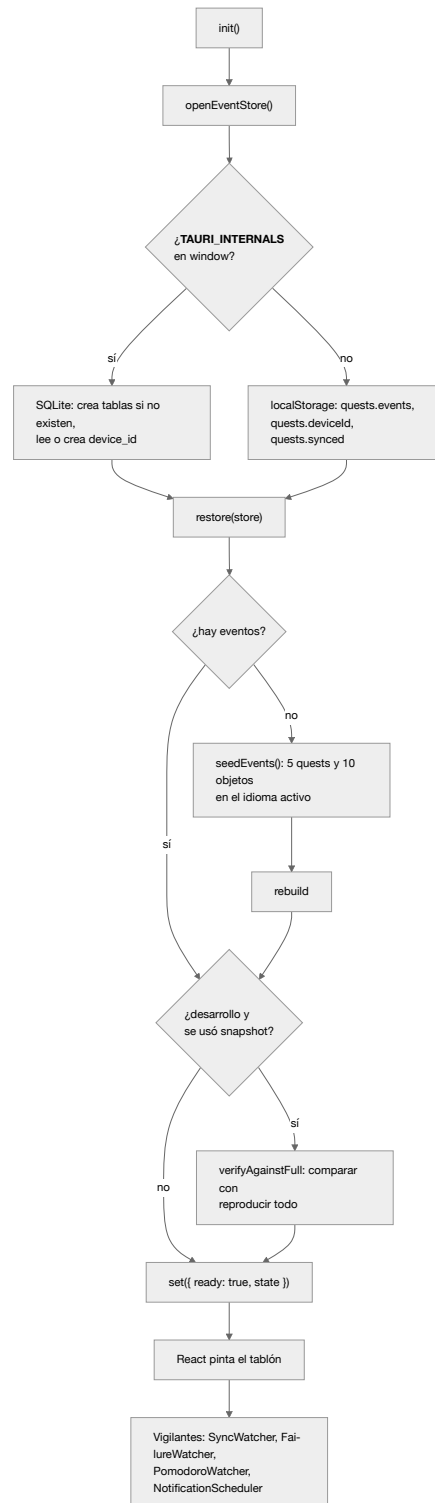
`init()` está en `src/store/game.ts` y se protege contra la doble ejecución del modo estricto guardando la promesa:

```
let initOnce: Promise<void> | undefined;
init() {
  initOnce ??= load(); // la segunda llamada reutiliza la primera
  return initOnce;
}
```

Cuidado. Sin `initOnce`, en desarrollo las dos ejecuciones verían la base vacía a la vez y **sembrarían los datos de ejemplo dos veces**. Es un caso real de condición de carrera.

`load()` es el corazón del arranque:

```
async function load() {
  try {
    const store = await openEventStore(); // SQLite (Tauri) o localStorage (navegador)
    let r = await restore(store); // snapshot + cola, o todo desde cero
    if (r.count === 0) { // base vacía: datos de ejemplo
      let last: EventPos | undefined;
      for (const body of seedEvents()) {
        const e = newEvent(store, body, last);
        await store.append(e);
        last = e;
      }
      r = await rebuildProjection(store);
    }
    if (import.meta.env.DEV && r.snapCount > 0) { // solo en desarrollo
      const full = await verifyAgainstFull(store, r);
      if (full) r = { ...full, snapCount: 0 };
    }
    snapCount = r.snapCount;
    set({ store, projected: r, state: finishProjection(r.acc), ready: true });
    maybeSnapshot(r);
  } catch (err) {
    set({ error: String(err), ready: true }); // el tablón muestra app.dbError
  }
}
```



Tres detalles finos:

1. **Cómo sabe si está en Tauri.** Tauri inyecta `window.__TAURI_INTERNALS__` en su WebView. `isTauri()` (en `src/storage/eventStore.ts`) solo mira eso.
2. **Datos de ejemplo** (`src/domain/seed.ts`). Si la base está vacía se crean **eventos normales**: cinco quests (un dragón, una torre, el gimnasio, la biblioteca, el mercado) y diez objetos. Sus ids son **fijos** (`seed:quest:dragon...`), para que dos equipos que arrancan vacíos y se sincronizan no tengan los ejemplos duplicados (capítulo 17).

3. **Comprobación en desarrollo.** Si arrancó desde un snapshot, compara el resultado con el de reproducir todos los eventos. Si no coinciden, casi seguro alguien cambió la proyección sin subir `PROJECTION_VERSION` (capítulo 9).

2.5 Los vigilantes

Cuando el estado está listo, `App` monta en `Windows` unos componentes que no pintan nada visible y actúan en segundo plano:

Vigilante	Funcionalidad	Qué hace
<code>SyncWatcher</code>	<code>sync</code>	Mira si se puede sincronizar y sincroniza al abrir, cada 5 min, al volver y al cerrar
<code>FailureWatcher</code>	<code>failure</code>	Al arrancar y cada minuto, emite <code>quest_failed</code> / <code>temporal_failed</code> de lo que venció
<code>PomodoroWatcher</code>	<code>pomodoro</code>	Sonidos y avisos al cambiar de fase
<code>NotificationScheduler</code>	<code>notifications</code>	Recalcula y programa los avisos del sistema

2.6 Probar desde cero sin tocar nada del propietario

En el navegador, los datos son las claves `quests.*` del `localStorage` de `http://localhost:1420`. El procedimiento seguro (en `docs/runbooks/verificar-ui.md`) es: **descargar una copia** de esas claves, **apartarlas**, recargar (la app arranca con los ejemplos), probar y **restaurar** desde la copia. Nunca vacíes el `localStorage` del navegador del propietario sin su permiso.

Para recién graduados. Es el mismo principio que en administración de sistemas: antes de un cambio destructivo, copia de seguridad verificable y plan de vuelta atrás.

2.7 Qué hay en `localStorage`

Además de los eventos (solo en el navegador), cada equipo guarda ahí sus **preferencias**, que no se sincronizan:

Clave	Qué guarda
<code>quests.events</code> , <code>quests.deviceId</code> , <code>quests.synced</code>	Eventos, id del equipo y eventos subidos (solo navegador)
<code>quests.snapshot</code>	Snapshot de la proyección (solo navegador; en Tauri va en SQLite)
<code>quests.lang</code>	Idioma
<code>quests.muted</code> , <code>quests.music</code> , <code>quests.haptics</code>	Silencio, música, vibración
<code>quests.calendarView</code> , <code>quests.temporalAccept</code>	Vista del calendario, filtro de encargos
<code>quests.menuCharacter</code>	Personaje elegido para hoy en el menú
<code>quests.notify</code> , <code>quests.notifyIds</code>	Avisos activados e ids programados

Clave	Qué guarda
<code>quests.failSeen</code>	Hasta cuándo se han enseñado los fallos

La lista viva la genera `pnpm docs:index` en `docs/INDEX.md` («Preferencias por equipo»).

Ejercicios

Ejercicio 2.1. Arranca `pnpm dev` y, en la consola del navegador, ejecuta `JSON.parse(localStorage.getItem("quests.events")).length`. ¿Cuántos eventos hay en una instalación nueva? Relaciónalo con `seedEvents()`.

Ejercicio 2.2. Quita temporalmente `initOnce ??=` (deja `return load()`) en una rama de pruebas, arranca con el `localStorage` vacío y observa el tablón. Explica lo que ves y deshaz el cambio.

Ejercicio 2.3. Dibuja (en papel o en Mermaid) qué componentes de `App.tsx` se montan siempre y cuáles dependen de `section`. ¿Por qué `Top`, `Bottom` y `Windows` van envueltos en `memo`?

Ejercicio 2.4. Busca en el código dónde se muestra `app.dbError`. ¿Qué tendría que fallar para que salga?

Parte II · React y Zustand

Cómo se pinta la interfaz, cómo se actualiza cuando cambian los datos y dónde vive el estado global.

Capítulo 3. React básico aplicado al proyecto

3.1 La idea de React en un párrafo

En la web clásica, el programa **modifica el DOM** a mano («busca este `<div>` y cámbiale el texto»). React invierte el planteamiento: escribes **funciones que describen cómo debe verse la interfaz para unos datos dados**, y React se encarga de calcular qué ha cambiado y de tocar solo eso en el DOM. Esas funciones son los **componentes**.

```
interfaz = f(estado)
```

Cuando el estado cambia, React vuelve a llamar a la función (un **render** o pintado), compara el resultado con el anterior (*reconciliación*) y aplica al DOM las diferencias mínimas.

Para recién graduados. Es programación declarativa, como SQL: dices *qué* quieres, no *cómo* conseguirlo paso a paso. El «motor de ejecución» es React.

3.2 Componentes, JSX y props

Un componente es una función que recibe un objeto de **props** (parámetros) y devuelve **JSX**, una sintaxis parecida a HTML dentro de TypeScript que el compilador traduce a llamadas de funciones. El ejemplo más pequeño del proyecto es `Tabs` (`src/components/Tabs.tsx`), las pestañas de categoría:

```

export const TABS: { id: Tab; color: string }[] = [
  { id: "all", color: "var(--gold)" },
  { id: "request", color: "var(--request)" },
  { id: "elite", color: "var(--elite)" },
  { id: "repeat", color: "var(--repeat)" },
];

export function Tabs() {
  const tab = useGame((s) => s.tab); // lee del store global (capítulo 4)
  const setTab = useGame((s) => s.setTab);
  const { t: tr } = useTranslation(); // traducciones (capítulo 14)

  return (
    <div className="tabs" role="tablist">
      {TABS.map((t) => (
        <button
          key={t.id}
          role="tab"
          aria-selected={tab === t.id}
          className={`tab ${tab === t.id ? "on" : ""}`}
          onClick={() => { sfx.move(); setTab(t.id); }}
        >
          {tab === t.id && (
            <motion.span layoutId="tab-hl" className="tab-hl" /> // el resaltado se desliza
          )}
          <span className="tab-lbl">{tr(`tabs.${t.id}`)}</span>
        </button>
      ))}
    </div>
  );
}

```

Fíjate en cinco cosas que se repiten en todo el proyecto:

1. **className** en lugar de **class** (palabra reservada de JavaScript).
2. **Expresiones entre llaves**: `{tab === t.id && (...)}` pinta algo solo si la condición es cierta.
3. **Listas con map** y una **key** estable por elemento (`t.id`). La **key** le dice a React qué elemento es cuál entre dos pintados; nunca uses el índice si la lista puede reordenarse.
4. **Manejadores de eventos** (`onClick`) que llaman a acciones; el componente no decide reglas del juego.
5. **Nada de texto escrito a mano**: todo pasa por `t("clave")`.

Un componente con props es `QuestCard` (`src/components/QuestCard.tsx`). Sus props dicen todo lo que necesita para pintarse:

```

<QuestCard
  key={q.id}
  quest={q}
  status={effectiveStatus(q, now)} // calculado con la hora (capítulo 10)
  now={now}
  due={questDue(q, temporals)} // fecha límite efectiva
  lock={blockers(q, quests)} // requisitos pendientes
  selected={q.id === selected?.id}
  onSelect={selectCard}
/>

```

Idea clave. Un componente de Quests **muestra** datos y **llama a acciones**. La lógica que decide («¿se puede aceptar?», «¿está completa?») vive en el dominio o en un `actions.ts`, nunca en el componente. Es una norma del proyecto (`docs/AGENTES.md §5.2`).

3.3 Estado local y hooks

Los **hooks** son funciones que empiezan por `use` y que solo se pueden llamar en el cuerpo de un componente (no dentro de `if` ni de bucles: React los identifica por el orden de llamada). Los que verás en Quests:

Hook	Para qué	Ejemplo en Quests
<code>useState</code>	Un valor local que, al cambiar, repinta el componente	<code>useNow()</code> guarda la hora actual (<code>src/lib/time.ts</code>)
<code>useEffect</code>	Efectos tras pintar: suscripciones, temporizadores, <code>init()</code>	El teclado global de <code>App</code> (<code>window.addEventListener("keydown", ...)</code>)
<code>useLayoutEffect</code>	Como <code>useEffect</code> , pero antes de que el navegador pinte	Las animaciones de GSAP de <code>QuestCard</code> (evita un parpadeo)
<code>useRef</code>	Una caja mutable que no provoca re-pintado; también referencia a un nodo del DOM	<code>prevStatus</code> y <code>stampRef</code> en <code>QuestCard</code>
<code>useMemo</code>	Recordar un cálculo caro mientras no cambien sus dependencias	Filtrar y ordenar las quests visibles en <code>App</code>
<code>useCallback</code>	Recordar una función para que su identidad no cambie	<code>selectCard</code> en <code>App</code>
<code>useSyncExternalStore</code>	Suscribirse a una fuente externa a React	<code>useIsPhone()</code> (una <i>media query</i>) y <code>useMusic()</code>

Un hook propio sencillo, `useNow`, que hace que «Vuelve en 3 h» avance solo:

```
export function useNow(intervalMs = 20_000) {
  const [now, setNow] = useState(() => Date.now());
  useEffect(() => {
    const t = setInterval(() => setNow(Date.now()), intervalMs);
    return () => clearInterval(t); // limpieza al desmontar o al cambiar intervalMs
  }, [intervalMs]); // lista de dependencias
  return now;
}
```

La **lista de dependencias** de `useEffect` dice cuándo repetir el efecto: `[]` solo al montar; `[intervalMs]` al montar y cada vez que cambie. La función que devuelve el efecto es la **limpieza**: se ejecuta antes de repetirlo y al desmontar el componente. Olvidar la limpieza deja temporizadores y escuchas vivos: una fuga clásica.

3.4 Cuándo se vuelve a pintar un componente

Un componente se repinta cuando:

1. Cambia su **estado local** (`setNow`).
2. Cambia algo del **store** que ha seleccionado (capítulo 4).

3. Se repinta su **padre**... salvo que esté envuelto en `memo` y sus props sean las mismas (comparadas por identidad).

Quests tiene muchas tarjetas y animaciones, así que cuida mucho el punto 3:

```
// QuestCard solo se repinta si cambian SUS props.
export const QuestCard = memo(forwardRef<HTMLButtonElement, Props>(function QuestCard(props,
ref) { ... })));

// En App: un manejador estable. Sin useCallback, cada pintado de App crearía una función
// nueva, las props de todas las tarjetas «cambiarían» y se repintarían todas.
const selectCard = useCallback((id: string) => {
  useGame.getState().select(id);
}, []);
```

Y App separa en piezas con `memo` y **sin props** lo que no depende del tablón (Top, Bottom, Windows): cuando cambia una quest y App se repinta, esas piezas no se repintan; cada una sigue a su trozo del store por su cuenta.

Cuidado. `memo` compara props **por identidad** (`===`). Un objeto o un *array* creado en cada pintado (`style={{...}}`, `[...lista]`) siempre es «distinto». Por eso es tan importante que el estado conserve la identidad de lo que no cambia (capítulo 9).

3.5 Derivar en vez de duplicar

App no guarda la lista de quests visibles: la **calcula** de la proyección con `useMemo`:

```
const inTab = useMemo(
  () =>
    [...quests.values()]
      .filter((q) => q.status !== "done" && !q.reserved && (tab === "all" || q.category ===
tab))
      .sort((a, b) => ORDER[a.category] - ORDER[b.category] || a.createdAt - b.createdAt),
  [quests, tab],
);
const visible = useMemo(
  () => inTab.filter((q) => matchesHorizon(horizon, questDue(q, temporals), now)),
  [inTab, horizon, temporals, now],
);
const selected = visible.find((q) => q.id === selectedId) ?? visible[0];
```

Es la misma filosofía que el event sourcing a pequeña escala: **guarda lo mínimo y calcula el resto**. Guardar la lista filtrada sería guardar una copia que hay que mantener sincronizada a mano.

3.6 Refs y el DOM: animaciones con GSAP

React controla el DOM, pero a veces necesitas tocar un nodo concreto (para animarlo con GSAP, medirlo o darle el foco). Para eso están las **refs**:

```

const stampRef = useRef<HTMLDivElement>(null);
const prevStatus = useRef(status);

useLayoutEffect(() => {
  const was = prevStatus.current;
  prevStatus.current = status;
  const active = status === "active";
  if (!active || was === "active") { // sin transición: estado final, sin animar
    gsap.set(stampRef.current, { opacity: active ? 1 : 0 });
    return;
  }
  const tl = gsap.timeline(); // de no activa a activa: el sello cae
  tl.fromTo(stampRef.current, { scale: 2.8, opacity: 0 }, { scale: 1, opacity: 1, duration: 0.2 });
  return () => { tl.kill(); }; // limpieza: nunca tl.progress(1)
}, [status]);

return <div ref={stampRef} className="stamp">...</div>;

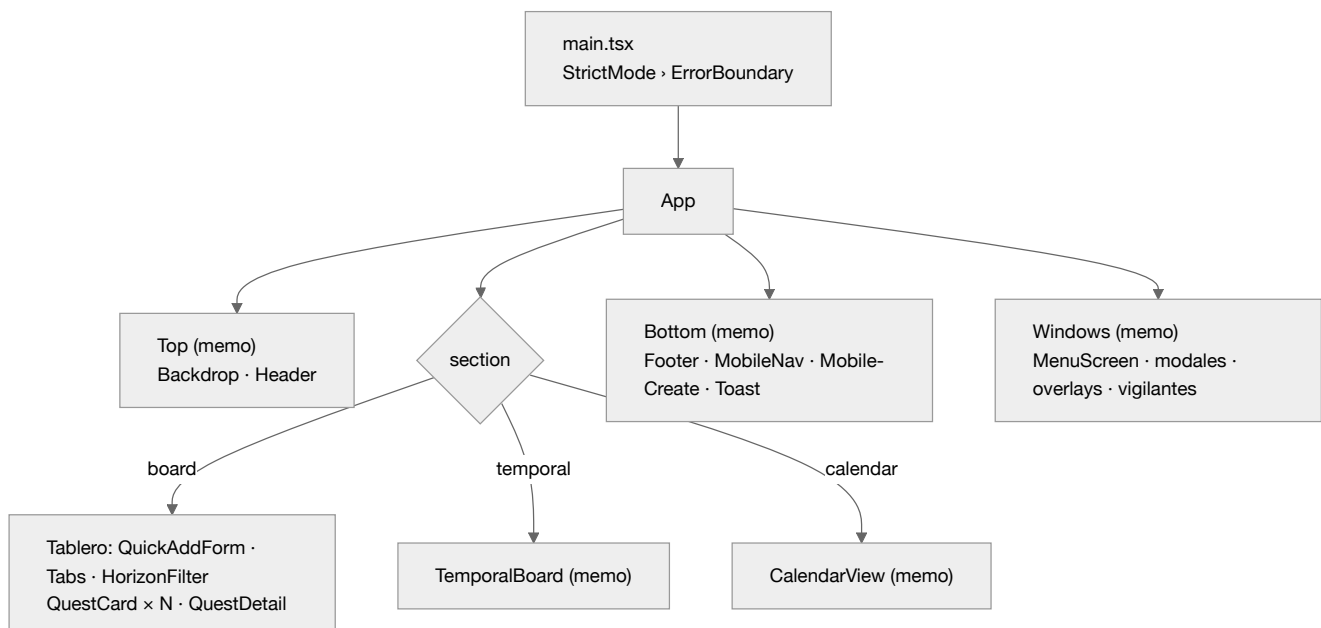
```

`prevStatus` es una ref y no estado porque **no debe** repintar nada: solo recuerda el valor anterior para saber si hubo transición. Este patrón (estado anterior en un `useRef`, animación solo en la transición que interesa, `gsap.set` en los demás casos y limpieza con `tl.kill()`) es norma del proyecto; lo verás en detalle en el capítulo 14.

3.7 Componentes de clase: el error boundary

Casi todo `Quests` son funciones, con una excepción obligada: el **error boundary** (`features/recovery`). React solo permite atrapar errores de pintado con los métodos de clase `getDerivedStateFromError` y `componentDidCatch`. Envuelve a toda la app en `main.tsx`; «Volver a intentarlo» monta `App` otra vez con una `key` nueva (los componentes que fallaron se crean de cero) sin perder el store, porque `init()` solo corre una vez.

3.8 El árbol de componentes



3.9 Teclado: un único `keydown`

Quests se juega con teclado (`Enter` acepta o reporta, `+` avanza, `T` cambia de tablón...). En vez de un escuchador por botón, `App` registra **uno solo** en `window` y traduce teclas a acciones. Antes de nada comprueba si estás escribiendo en un campo o si hay una ventana abierta:

```
const busy = s.creating || s.clear || s.collection || temporalBusy() || merchantBusy()
  || characterBusy() || chronicleBusy() || calendarBusy() || editingBusy() || failureBusy()
  || searchBusy() || quickBusy() || menuBusy() || customizeBusy();
if (busy || e.metaKey || e.ctrlKey || typing) return;
```

Cada funcionalidad con ventana exporta su `...Busy()` desde su `ui.ts`. Así una funcionalidad nueva no necesita que `App` conozca sus detalles: solo que pregunte si está ocupada.

Ejercicios

Ejercicio 3.1. En `Tabs.tsx`, cambia temporalmente `key={t.id}` por `key={Math.random()}`. ¿Qué le pasa a la animación del resaltado al cambiar de pestaña? Explica por qué.

Ejercicio 3.2. Quita `useCallback` de `selectCard` en `App.tsx` y usa las herramientas de React (React DevTools, «Highlight updates») para ver qué tarjetas se repintan al moverte con las flechas. Vuelve a dejarlo como estaba.

Ejercicio 3.3. Escribe un hook `useCountdown(target: number)` que devuelva los segundos que faltan hasta `target` y se actualice cada segundo, con su limpieza. ¿Qué dependencias necesita?

Ejercicio 3.4. En `App.tsx`, ¿por qué la lista no se reordena al aceptar una quest? Encuentra el criterio de orden y razona qué pasaría con la animación del sello si se ordenara por estado.

Capítulo 4. Zustand y el store global

4.1 El problema del estado compartido

En React, el estado local vive en un componente. Pero el nivel del jugador lo muestran la cabecera, el menú, «Quest Clear» y el personaje; las quests, el tablón, el calendario, «Mi día» y la búsqueda. Pasar todo eso de padres a hijos por props («prop drilling») se vuelve inmanejable. La solución es un **store**: un objeto fuera de React con el estado compartido, al que cada componente se **suscribe**.

Zustand es una de las bibliotecas más pequeñas para esto. Se eligió frente a Redux o el Context de React por ser mínima y, sobre todo, porque **se puede usar fuera de los componentes** (ADR-05): las acciones, los atajos de teclado y los tests leen el store con `useGame.getState()`.

4.2 Crear un store

```
import { create } from "zustand";

interface Contador {
  n: number;
  sumar(): void;
}

export const useContador = create<Contador>((set, get) => ({
  n: 0,
  sumar: () => set({ n: get().n + 1 }), // set mezcla el objeto con el estado actual
}));
```

`create` devuelve un **hook** con tres usos:

```
const n = useContador((s) => s.n); // 1. en un componente: se repinta si cambia s.n
useContador.getState().sumar();    // 2. fuera de React: leer o llamar sin suscribirse
useContador.setState({ n: 10 });   // 3. fuera de React: escribir directamente
```

4.3 Selectores: suscribirse solo a lo necesario

`useGame((s) => s.tab)` es un **selector**: el componente se repinta **solo** cuando cambia el valor que devuelve (comparado con `Object.is`). Es la herramienta principal de rendimiento:

```
// Bien: un valor primitivo; repinta solo si cambia el número.
const active = useGame((s) => [...s.state.quests.values()].filter((q) => q.status === "active").length);

// Bien: una referencia que solo cambia cuando cambian las quests.
const quests = useGame((s) => s.state.quests);
```

Cuidado. Un selector que **crea** un objeto o un *array* nuevo en cada llamada (`s`) => `[...s.state.quests.values()]` devuelve algo «distinto» siempre. En Zustand 5 eso provoca repintados constantes (e incluso un bucle infinito). Selecciona la referencia (`s.state.quests`) y deriva con `useMemo`, o devuelve un primitivo, o usa `useShallow`.

4.4 `useGame`: el store del juego

El store principal está en `src/store/game.ts`. Su interfaz resume la arquitectura:

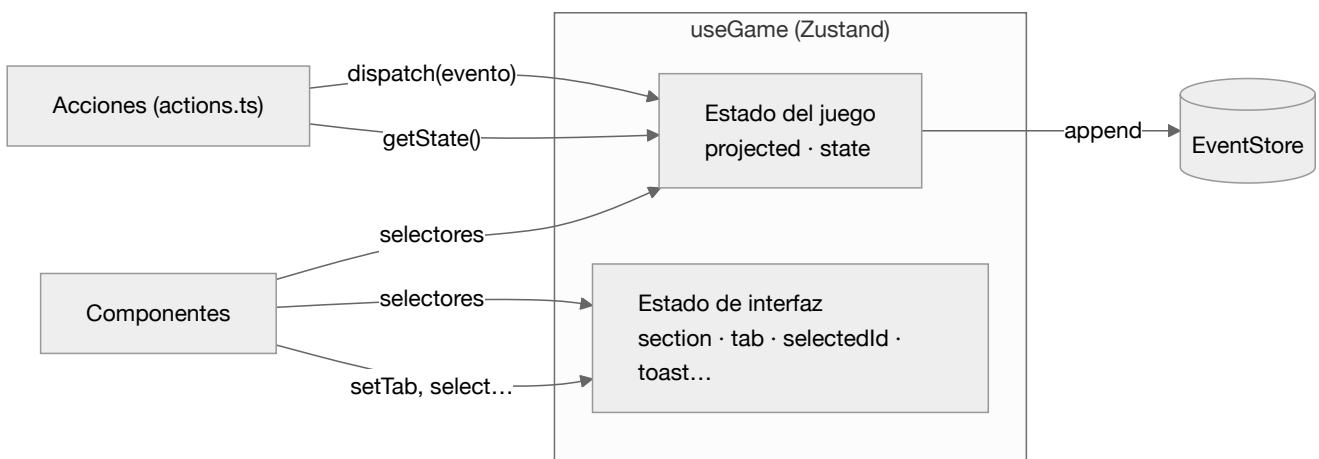
```
interface GameStore {
  ready: boolean;           // ¿terminó init()?
  error?: string;           // fallo al abrir la base
  store?: EventStore;       // el almacén de eventos abierto (la fuente de verdad)
  projected: Projected;     // acumulador de la proyección + último evento aplicado + cuántos
  state: GameState;         // el estado calculado que lee la interfaz

  // Estado de interfaz común
  section: Section;         // "board" | "temporal" | "calendar"
  tab: Tab;                 // "all" | "elite" | "repeat" | "request"
  selectedId?: string;
  creating: boolean;        // formulario de quest abierto
  clear?: ClearResult;      // «Quest Clear» en pantalla (jugador antes y después)
  collection?: CollectionTab; // ventana de objetos abierta
  toast?: { text: () => string; key: number; action?: ToastAction };

  init(): Promise<void>;
  dispatch(body: EventBody): Promise<GameEvent | undefined>; // ÚNICO punto de escritura
  rebuild(): Promise<void>;
  setSection, setTab, select, setCreating, setClear, setCollection, say ...
}
```

Hay dos tipos de cosas mezcladas a propósito:

- **Estado del juego** (`projected`, `state`): **solo** cambia a través de `dispatch`, que recibe un evento. Nunca se hace `setState({ state: ... })` desde fuera.
- **Estado de interfaz común** (`section`, `tab`, `selectedId`...): se cambia con *setters* normales. No genera eventos ni se sincroniza: es de esta ventana.



4.5 `dispatch`: el único punto de escritura

Todo cambio en el juego pasa por aquí. Este es el código real, comentado:

```
async dispatch(body) {
  if (rebuilding) await rebuilding; // si se está recalculando todo, espera
  const { store, projected } = get();
  if (!store) return;
  const e = newEvent(store, body, projected.last); // añade id, deviceId, ts y v
  // Reloj muy atrasado o deshacer: el evento cambia el pasado → recalcul todo.
  if (!goesAfter(projected, e) || e.type === "event_undone") {
    await store.append(e);
    await get().rebuild();
    return e;
  }
  // Caso normal: aplicar SOLO el evento nuevo, copiando solo lo que cambia (Immer).
  const next = applyNext(projected, e);
  const state = next.acc === projected.acc
    ? get().state // nada cambió
    : viewProjection(next.acc, { acc: projected.acc, state: get().state });
  set({ projected: next, state }); // 1. la interfaz se actualiza YA
  await store.append(e); // 2. después se guarda en disco
  maybeSnapshot(next); // 3. cada 100 eventos, snapshot
  return e; // 4. para ofrecer «Deshacer»
}
```

Y `newEvent` construye el evento completo:

```
const newEvent = (store: EventStore, body: EventBody, last?: EventPos) =>
  ({ ...body, id: uid(), deviceId: store.deviceId, ts: nextTs(Date.now(), last), v:
    EVENT_VERSION }) as GameEvent;
```

Idea clave. `dispatch` es **optimista**: primero actualiza la memoria (la animación arranca sin esperar a SQLite) y luego escribe en disco. Si el disco fallara, el evento se perdería, pero la interfaz nunca espera al disco. Los capítulos 5, 8 y 9 explican cada pieza (`nextTs`, `applyNext`, `viewProjection`, el snapshot).

4.6 Los avisos: `say`

```
say: (text, action) => {
  const key = Date.now() + Math.random();
  set({ toast: { text, key, action } });
  // Se borra a los 4,5 s (7 s si lleva botón), salvo que haya llegado otro aviso.
  setTimeout(() => get().toast?.key === key && set({ toast: undefined }), action ? 7000 : 4500);
},
```

Dos detalles de diseño: el texto es una **función** `(( ) => i18n.t(...))` para que se traduzca al pintarse y siga al idioma si lo cambias con el aviso en pantalla; y la `key` evita que el temporizador de un aviso viejo borre uno nuevo.

4.7 Un store por funcionalidad para la interfaz

Si todas las funcionalidades metieran su estado de interfaz en `useGame`, el store crecería sin control. La norma es que **cada funcionalidad tiene su propio store de interfaz en su `ui.ts`**. Ejemplo, el tablón de encargos (`src/features/temporal/ui.ts`):

```
export const useTemporalUi = create<TemporalUi>((set) => ({
  showDone: false,
  accept: loadAccept(), // filtro recordado en localStorage
  select: (selectedId) => set({ selectedId }),
  open: (id, origin) => set({ viewing: { id, origin }, selectedId: id }),
  closeView: () => set({ viewing: undefined }),
  setAccept: (accept) => {
    try { localStorage.setItem("quests.temporalAccept", accept); } catch {}
    set({ accept });
  },
  // ...
}));

/** Hay una ventana o animación abierta: el teclado del tablón espera. */
export function temporalBusy(): boolean {
  const s = useTemporalUi.getState();
  return !(s.viewing || s.form || s.posted || s.cleared || s.viewer);
}
```

Cuidado. Un `ui.ts` guarda **estado de interfaz** (qué está abierto, qué está elegido, un filtro). Nunca datos del juego: esos se calculan de los eventos. Si dudas, pregúntate: «¿lo tiene que ver igual el otro equipo?». Si la respuesta es sí, es un evento.

4.8 Acciones fuera de React

Las acciones (capítulo 12) son funciones normales que leen el store con `getState()`:

```
export async function abandonQuest(id: string) {
  const { state, dispatch } = useGame.getState();
  const q = state.quests.get(id);
  if (q?.status !== "active") return; // valida
  sfx.cancel();
  const e = await dispatch({ type: "quest_abandoned", questId: id });
  offerUndo(e, () => i18n.t("toast.abandoned", { title: q.title }));
}
```

Como no son hooks, se llaman igual desde un botón, una tecla, otra acción o un test.

Ejercicios

Ejercicio 4.1. Escribe un componente `ContadorActivas` que muestre cuántas quests hay en curso. Hazlo primero con un selector que devuelva un `array` y observa la consola; después corrígelo como en la cabecera (`Header.tsx`).

Ejercicio 4.2. ¿Qué campos de `GameStore` son estado del juego y cuáles de interfaz? Para cada uno de interfaz, razona por qué no debe sincronizarse.

Ejercicio 4.3. Lee `say`. ¿Qué pasaría si se quitara la comprobación `get().toast?.key === key`? Diseña una secuencia de dos avisos que lo demuestre.

Ejercicio 4.4. En una rama de pruebas, intenta hacer `useGame.setState({ state: ... })` desde un componente para «subir de nivel». ¿Qué normas rompe? ¿Qué pasaría al reiniciar la app y al sincronizar con otro equipo?



Parte III · Event sourcing y dominio

La decisión que lo sostiene todo: guardar hechos, no estado. Qué es un evento, cómo se ordena y cómo se versiona.

Capítulo 5. Event sourcing

5.1 Guardar estado frente a guardar hechos

En la carrera seguramente diseñaste bases de datos **CRUD**: una tabla `jugador` con `nivel = 4`, `xp = 1150` que se actualiza con `UPDATE`. Funciona, pero tiene tres problemas para una app como Quests:

1. **Se pierde la historia.** Si el nivel es 4, no sabes cómo llegaste. Una crónica, un «deshacer» o unas estadísticas exigen guardar algo más.
2. **Sincronizar es difícil.** Si el Mac dice `xp = 1150` y el iPhone dice `xp = 1200`, ¿quién tiene razón? Hay que inventar reglas de conflicto para cada campo.
3. **Los errores son permanentes.** Si un fallo de lógica guarda un valor malo, el valor malo queda escrito.

Event sourcing (*fuentes de eventos*) guarda en cambio la **lista de hechos** que ocurrieron, en orden, y nunca los modifica:

```
1. quest_created      «Recado del Mercado»
2. quest_accepted    «Recado del Mercado»
3. progress_added     Fruta +1
...
9. quest_completed   «Recado del Mercado», recompensa {xp: 85, gold: 1560}
```

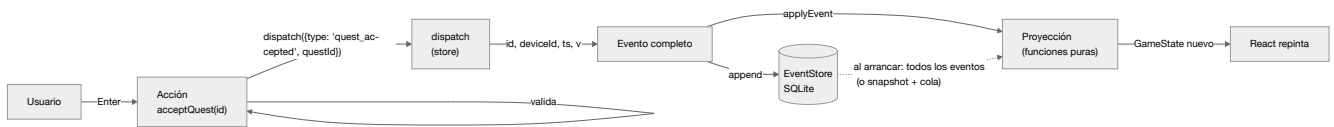
El estado (nivel, XP, oro, qué quests hay y en qué estado están) se **calcula** reproduciendo la lista. Esa función de cálculo se llama **proyección**.

Para recién graduados. Es exactamente cómo funciona un libro de contabilidad: el saldo no se apunta, se calcula sumando los movimientos. También es la idea del *write-ahead log* de las bases de datos que viste en Sistemas: el registro de operaciones es la verdad y las tablas son una consecuencia.

5.2 Las tres consecuencias que aprovecha Quests

Consecuencia	Dónde se ve
Sincronizar es unir listas. Cada evento tiene un id único; dos equipos unen sus listas y calculan lo mismo. No hay «versión buena»	Capítulo 17: cada equipo sube su archivo de eventos a Drive y baja los de los demás
El historial sale gratis. Deshacer es otro evento; la crónica, las rachas y los atributos se calculan del pasado, incluso lo jugado antes de que existieran	<code>features/undo</code> , <code>chronicle</code> , <code>streaks</code> , <code>attributes</code>
El dominio es una función pura. Misma lista, mismo resultado, en cualquier equipo y en los tests	<code>project(events)</code> en <code>src/domain/projection.ts</code>

5.3 El ciclo completo



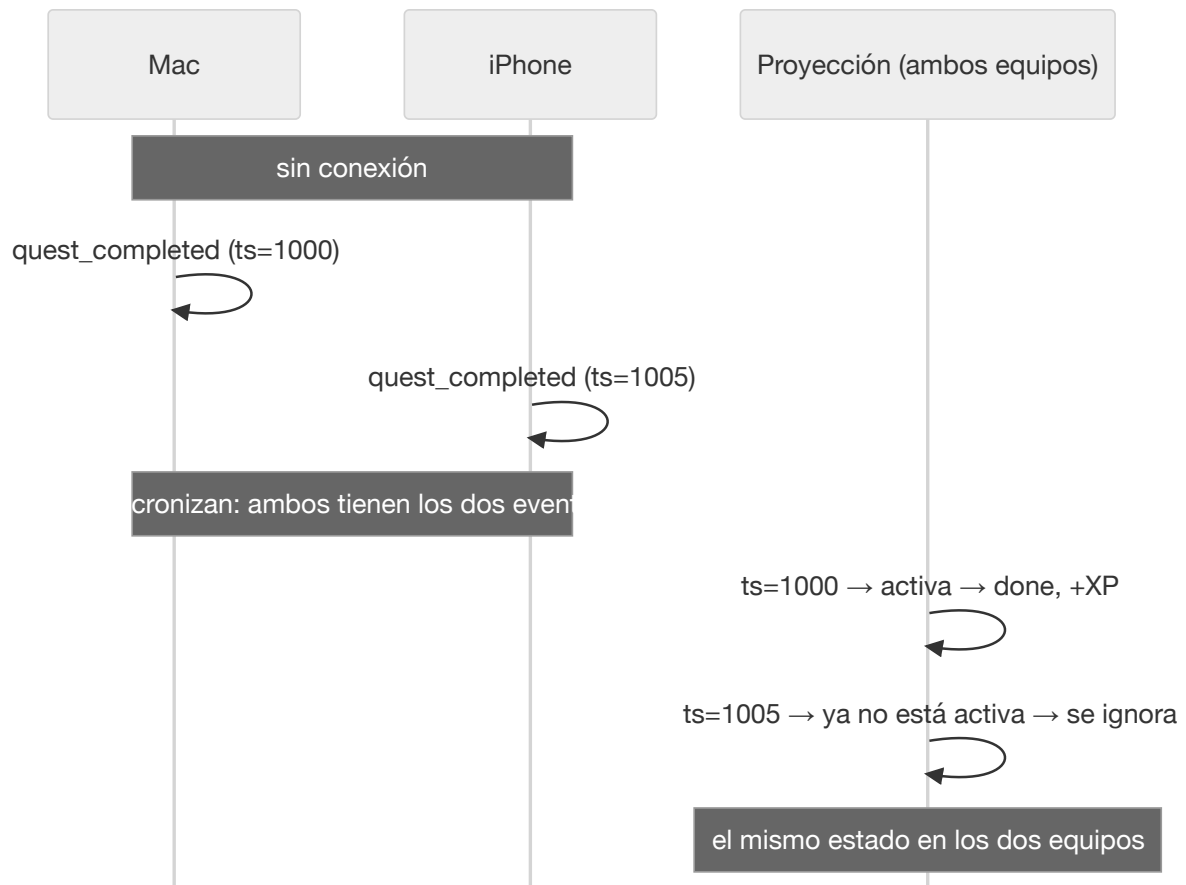
La regla de oro del proyecto: **nadie modifica el estado directamente**. Solo se emiten eventos; el estado se calcula a partir de ellos. Las animaciones *observan* los cambios de estado; nunca los provocan.

5.4 La proyección y sus guardas

La proyección es un gran `switch` sobre el tipo de evento. Cada `case` lleva una **guarda**: una condición que, si no se cumple, hace que el evento **se ignore** en lugar de romper algo. Simplificado:

```
switch (e.type) {
  case "quest_accepted":
    if (q && (q.status === "available" || (q.status === "cooldown" && e.ts >= q.availableAt))
        && prerequisitesMet(q, quests) && !inReserve(temporals, q.id)) {
      q.status = "active";
      q.progress = {};
    }
    break;
  case "quest_completed":
    if (q?.status === "active") { // ← guarda
      acc.xp += e.reward.xp;
      acc.gold += e.reward.gold;
      // ...
    }
    break;
}
```

¿Por qué ignorar y no lanzar un error? Porque **los eventos vienen de varios equipos**. Imagina que completas la misma quest en el Mac y en el iPhone sin conexión. Al sincronizar habrá dos `quest_completed`. Al reproducir, el primero pasa la quest a `done`; el segundo ya no la encuentra activa y se ignora. **La XP no se duplica**, sin código especial de sincronización.



5.5 Tres decisiones finas que conviene memorizar

1. **El progreso es un delta** (`progress_added { amount: +1 }`), no un valor absoluto. Si dos equipos suman +1 cada uno, el total es +2. Con un valor absoluto (`progress = 3`) uno pisaría al otro.
2. **Lo que no debe cambiar se copia en el evento.** `quest_completed` lleva una copia de la recompensa; `gear_purchased`, del precio. Si mañana cambia la fórmula, lo ya ganado o pagado no cambia.
3. **El paso del tiempo no genera eventos.** Una quest repetible en espera guarda `availableAt`; la interfaz decide con la hora:

```

export function effectiveStatus(q: QuestState, now: number) {
  if (q.status === "cooldown" && now >= (q.availableAt ?? 0)) return "available";
  return q.status;
}

```

Así una quest «vuelve» sola aunque la app esté cerrada, sin que nadie emita un evento `quest_returned`. Las fases del pomodoro, los plazos, el escaparate semanal y el personaje del día siguen la misma idea. La excepción justificada son los **fallos** (`quest_failed`): sacan la quest del tablón y quedan en la crónica, así que tienen que ser iguales en todos los equipos (ADR-44).

5.6 El azar también se guarda

El botón de una quest es aleatorio. Pero si la proyección tirara dados, el Mac y el iPhone calcularían botines distintos al reproducir los mismos eventos. La solución: **el azar se resuelve en la acción y el resultado viaja en el evento**.

```
// En la acción (reportQuest): aquí sí se usa Math.random.
const drops = rollDrops(dropTableFor(q.category), state.items.values(), before.pity,
Math.random);
await dispatch({ type: "quest_completed", questId: id, reward: q.reward, drops });

// En la proyección: solo se LEE e.drops. Determinista.
receiveItems(items, reward.itemId, e.drops ?? [], e.ts);
```

Además, al ir dentro de `quest_completed`, el botón **hereda su guarda**: si el completado se ignora por duplicado, su botón también.

Idea clave. El dominio (`src/domain/` y los `model.ts`) **no llama** a `Date.now()` ni a `Math.random()`. El tiempo entra como parámetro (`now`) o sale del evento (`e.ts`); el azar entra como parámetro (`rnd`) y su resultado se guarda en el evento. Es lo que permite reproducir el pasado y testearlo.

5.7 Qué va en eventos y qué no

Va en eventos (se sincroniza)	No va en eventos
Todo lo del juego que debe verse igual en todos los equipos: quests y su progreso, encargos, agenda, objetos, compras, equipo puesto, personajes añadidos, deshacer, fallos	Preferencias de cada equipo (<code>localStorage</code>): idioma, silencio, música, vista del calendario, avisos
	Lo que se calcula : rachas, atributos, crónica, plazos, escaparate, rotación de personajes
	Lo que está en el código o en <code>public/</code> : las 69 piezas de serie, los personajes de serie
	Archivos grandes : van al almacén de binarios (capítulo 7); el evento lleva solo la referencia

5.8 Ventajas e inconvenientes (con honestidad)

Event sourcing no es gratis. Lo que cuesta, y cómo lo resuelve Quests:

Coste	Solución en Quests	Capítulo
Reproducir todo el historial en cada arranque crece con el tiempo	Snapshot cada 100 eventos	9
Recalcular todo el estado en cada acción sería lento	<code>applyNext</code> : aplicar solo el evento nuevo, copiando lo que cambia	8, 9
Los eventos antiguos no se pueden cambiar si cambia su formato	Versión en cada evento y <i>upcasting</i> al leer	6

Coste	Solución en Quests	Capítulo
Varios equipos con relojes distintos	Reloj lógico híbrido y orden total (<code>ts</code> , <code>id</code>)	6
Hay que pensar cada regla como «¿qué pasa si este evento llega dos veces o en otro orden?»	Guardas + tests con historiales aleatorios	18

Ejercicios

Ejercicio 5.1. Escribe a mano la lista de eventos que produce este uso: creas «Leer 20 páginas» (contador $\times 20$), la aceptas, sumas +1 tres veces, la abandonas, la vuelves a aceptar y sumas +1. ¿Cuánto progreso tiene al final? Comprueba tu respuesta leyendo los `case` de `quest_abandoned` y `quest_accepted`.

Ejercicio 5.2. Busca en `src/domain/projection.test.ts` el test «una quest completada suma su recompensa una sola vez». Explica qué evento se ignora y por qué.

Ejercicio 5.3. Propón un diseño CRUD para el inventario de objetos y enumera tres situaciones de sincronización entre dos equipos que tendrías que resolver a mano. ¿Cómo las resuelve el diseño con eventos?

Ejercicio 5.4. ¿Por qué el fin de la espera de una quest repetible no es un evento y el fallo de una quest sí? Responde con las palabras «crónica», «otros equipos» y «tiempo».

Capítulo 6. `EventBody`, `GameEvent` y versionado

6.1 Uniones discriminadas en TypeScript

Antes de ver los eventos hay que entender una herramienta de TypeScript: la **unión discriminada**. Es un tipo que puede ser una de varias formas, cada una con un campo común (el **discriminante**) que dice cuál es:

```
type Forma =
  | { kind: "circulo"; radio: number }
  | { kind: "rect"; ancho: number; alto: number };

function area(f: Forma) {
  switch (f.kind) {
    case "circulo": return Math.PI * f.radio ** 2; // aquí TypeScript SABE que hay radio
    case "rect":    return f.ancho * f.alto;        // y aquí, ancho y alto
  }
}
```

Dentro de cada `case`, TypeScript **estrecha** el tipo: no te deja leer `f.radio` en un rectángulo. Es el equivalente tipado de la herencia con *pattern matching*.

6.2 `EventBody`: qué ha pasado

`src/domain/events.ts` define la unión de todos los eventos del juego. Los seis del núcleo se escriben ahí; el resto los aporta cada funcionalidad desde su `events.ts`:

```
export type EventBody =
  | { type: "quest_created"; quest: QuestDef }
  | { type: "quest_deleted"; questId: string }
  | { type: "quest_accepted"; questId: string }
  | { type: "quest_abandoned"; questId: string }
  | { type: "progress_added"; questId: string; conditionId: string; amount: number }
  | { type: "quest_completed"; questId: string; reward: RewardDef; drops?: Drop[] }
  | PomodoroEventBody // features/pomodoro/events.ts
  | ItemEventBody      // features/items/events.ts
  | TemporalEventBody  // features/temporal/events.ts
  | MerchantEventBody
  | EquipmentEventBody
  | ChecklistEventBody
  | CollectibleEventBody
  | AgendaEventBody
  | EditingEventBody
  | FailureEventBody
  | UndoEventBody
  | MenuEventBody
  | LivingEventBody
  | CompanionEventBody;
```

Un ejemplo de funcionalidad, los encargos temporales:

```
export type TemporalEventBody =
  | { type: "temporal_created"; temporal: TemporalDef }
  | { type: "temporal_updated"; temporalId: string; patch: TemporalPatch }
  | { type: "temporal_attached"; temporalId: string; attachment: AttachmentRef }
  | { type: "temporal_detached"; temporalId: string; attachmentId: string }
  | { type: "temporal_linked"; temporalId: string; questId: string } // delta
  | { type: "temporal_unlinked"; temporalId: string; questId: string } // delta
  | { type: "temporal_accepted"; temporalId: string }
  | { type: "temporal_postponed"; temporalId: string }
  | { type: "temporal_completed"; temporalId: string; reward: TemporalReward } // copia
  | { type: "temporal_deleted"; temporalId: string }
  | { type: "temporal_art_added"; art: TemporalArt }
  | { type: "temporal_art_removed"; artId: string };
```

Fíjate en el estilo: enlazar una quest es un **delta** (`temporal_linked`) y no «la lista entera de quests enlazadas»; así, si dos equipos enlazan quests distintas a la vez, se suman en vez de pisarse. Y `temporal_completed` copia la recompensa.

En total hay del orden de cincuenta tipos de evento; `docs/INDEX.md` («Eventos → funcionalidad») tiene la lista generada desde el código.

6.3 `EventMeta` y `GameEvent`: quién, cuándo y en qué formato

El cuerpo dice **qué** pasó. Los metadatos dicen **quién**, **cuándo** y **con qué formato**:

```
export interface EventMeta {
  id: string; // UUID: identifica el evento para deduplicar al fusionar
  deviceId: string; // qué equipo lo generó
  ts: number; // reloj lógico híbrido, en milisegundos
  v?: number; // versión del formato (EVENT_VERSION). Falta en los anteriores a la 1
}

export type GameEvent = EventMeta & EventBody; // intersección: metadatos + una de las formas
```

Quien escribe una acción solo construye el **cuerpo**; `dispatch` añade los metadatos. **Nunca** se construye un `ts` a mano fuera de los tests.

6.4 El orden: `(ts, id)`

Para que todos los equipos lleguen al mismo estado, todos tienen que reproducir los eventos **en el mismo orden**. El orden total es: primero `ts`, y si empatan, `id` (comparado como texto):

```
export function comparePos(a: EventPos, b: EventPos): number {
  return a.ts - b.ts || (a.id < b.id ? -1 : a.id > b.id ? 1 : 0);
}
```

El `id` como desempate es arbitrario pero **determinista**: todos los equipos desempatan igual.

6.5 El reloj lógico híbrido (HLC)

¿Por qué no usar simplemente `Date.now()` como `ts`? Dos problemas reales:

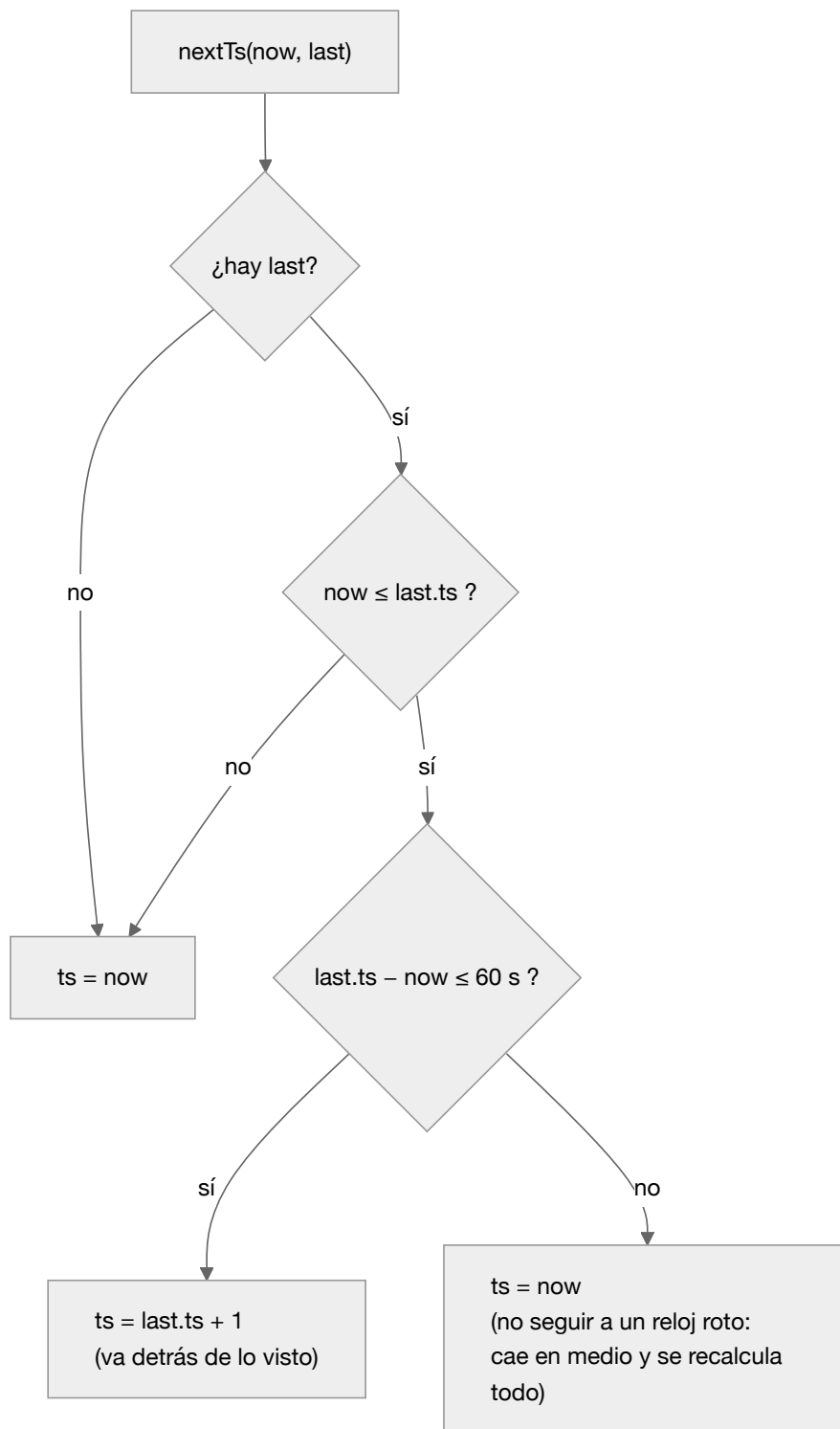
1. **Varios eventos en el mismo milisegundo.** Una acción puede emitir dos eventos seguidos con el mismo `Date.now()`. El desempate por `id` (aleatorio) podría ponerlos al revés: «completar» antes que «aceptar».
2. **Relojes que no coinciden.** Si el iPhone va 30 s atrasado, podría abandonar una quest «antes» de que el Mac la aceptara, aunque en realidad la abandonó después de ver la aceptación sincronizada.

La solución es un **reloj lógico híbrido** (*Hybrid Logical Clock*, ADR-25): un `ts` que es la hora física salvo cuando hace falta ir «por detrás» de lo ya visto:

```
export const MAX_DRIFT_MS = 60_000;

export function nextTs(now: number, last?: EventPos): number {
  return last && now <= last.ts && last.ts - now <= MAX_DRIFT_MS ? last.ts + 1 : now;
}
```

- Normalmente `ts = now`.
- Si el último evento aplicado (de este equipo o **fusionado de otro**) va igual o por delante del reloj, `ts = último + 1`: lo que haces después de ver algo queda **detrás** de ello.
- **Deriva máxima de 1 minuto.** Si el último evento va más de un minuto por delante (un equipo con la hora mal, por ejemplo en el año 2099), no se le sigue: el evento lleva la hora del reloj, cae en medio del historial y se recalcula todo. Así un reloj roto no arrastra a los demás.



Para recién graduados. En Sistemas Distribuidos viste los relojes de Lamport: un contador que garantiza que si A causó B, entonces $ts(A) < ts(B)$. Un HLC combina eso con la hora física para que los `ts` sigan pareciéndose a fechas reales. Aquí el contador va «dentro» de los milisegundos (cada evento seguido gasta 1 ms), así que el formato de los eventos, las consultas de SQLite y el snapshot no cambian.

6.6 Versionado: cuando cambia la forma de un evento

Los eventos guardados son **inmutables**: no se reescriben nunca, ni en tu base ni en el Drive de otros equipos. Pero el código evoluciona. ¿Qué pasa si hay que cambiar la forma de un evento? La respuesta es el **upcasting**: convertir los eventos antiguos al formato actual **al leerlos**, sin tocar los guardados.

Cada evento nuevo lleva `v: EVENT_VERSION`. Los anteriores a la versión 1 no la llevan y cuentan como versión 0:

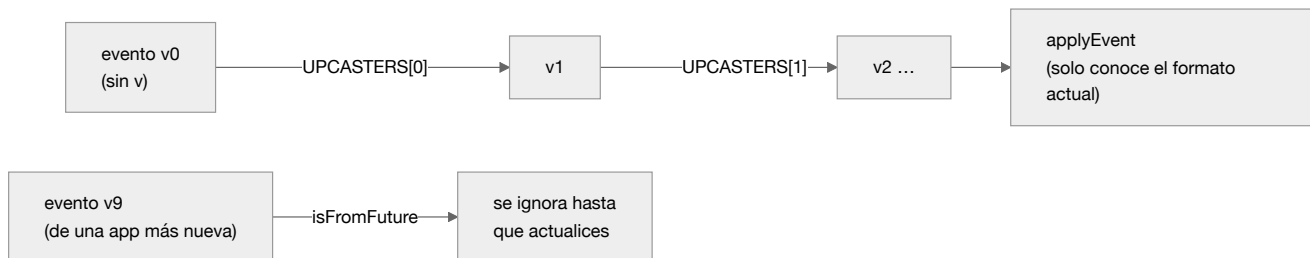
```
// src/domain/upcast.ts
/** UPCASTERS[n] convierte un evento de la versión n a la n + 1. Hay uno por versión. */
export const UPCASTERS: readonly Step[] = [
  (e) => e, // 0 → 1: mismo formato; solo empieza a llevar `v`
];

export const versionOf = (e: Pick<GameEvent, "v">) => e.v ?? 0;
export const isFromFuture = (e: Pick<GameEvent, "v">) => versionOf(e) > EVENT_VERSION;

export function upcastEvent(e: GameEvent): GameEvent {
  let v = versionOf(e);
  if (v >= EVENT_VERSION) return e;
  let out = e;
  for (; v < EVENT_VERSION; v++) out = UPCASTERS[v](out); // paso a paso: 0→1→2...
  return { ...out, v: EVENT_VERSION }; // copia: el original no se toca
}
```

Y `applyEvent` empieza siempre así:

```
if (isFromFuture(raw)) return; // de una versión más nueva de la app: no sé interpretarlo
const e = upcastEvent(raw);
```



Cuidado. Un evento de una versión **futura** (lo escribió otro equipo ya actualizado) se **ignora**, no se rechaza ni se borra. Cuando actualices, `PROJECTION_VERSION` habrá subido, el snapshot se descartará y el evento se aplicará.

6.7 Los formatos de antes de versionar: `legacy.ts`

Antes de que existiera `EVENT_VERSION` hubo dos cambios de formato. Se resuelven reconociendo la **forma** del dato en dos archivos:

- `features/pomodoro/legacy.ts`: el pomodoro era un objeto único de la quest (`pomodoroConfig`); hoy es un **tipo de objetivo**. `upcastQuestDef` lo convierte en una condición con un id estable

(`<questId>:pomodoro`), el mismo en todos los equipos.

- `features/items/legacy.ts`: la recompensa traía un objeto como **texto** (`item: "Poción de Vigor"`); hoy apunta a una entidad (`itemId`). `upcastReward` crea un objeto con id `legacy:<nombre>`.

Cuidado. No se añaden conversiones «por forma» nuevas: lo nuevo va en `UPCASTERS`. Los `legacy.ts` son historia.

6.8 Qué subir en cada cambio

Esta tabla (de `docs/runbooks/migrar-evento.md`) es una de las más importantes del proyecto:

Cambio	EVENT_VERSION	PROJECTION_VERSION	Up-caster
Un tipo de evento nuevo	No	Sí, si añade algo al acumulador o cambia lo que dan eventos guardados	No
Un campo opcional nuevo cuyo valor ausente es el comportamiento de antes	No	Solo si cambia el resultado de eventos guardados	No
Cambiar la forma de un evento existente (renombrar, quitar, reinterpretar)	Sí	Sí	Sí
Cambiar una regla de la proyección (un <code>case</code> , una guarda, un <code>apply*Event</code>)	No	Sí	No
Cambiar solo <code>finishProjection</code> (curva de niveles...)	No	No	No

`PROJECTION_VERSION` (en `src/domain/projection.ts`) no es la versión de los eventos sino la de la **lógica** que los interpreta: si cambia, los snapshots guardados con la lógica vieja se descartan (capítulo 9).

6.9 Deshacer sin borrar: `event_undone`

Deshacer es el mejor ejemplo de «los eventos no se borran». Deshacer es **otro evento**, `event_undone { eventId }`, y la proyección salta el evento deshecho:

```
export function project(events: GameEvent[]): GameState {
  const acc = newProjectionAcc();
  const undone = undoneIn(events); // ids deshechos
  for (const e of events) if (!undone.has(e.id)) applyEvent(acc, e);
  return finishProjection(acc);
}
```

`undoneIn` solo cuenta un deshacer si el evento existe, es de un tipo deshacible (`UNDOABLE`: crear, editar, aceptar, abandonar, retirar...) y no han pasado más de **15 minutos**. Completar una quest **no** se deshace (se volvería a tirar el cofre), ni comprar ni fallar.

Ejercicios

Ejercicio 6.1. Añade (en una rama de pruebas) un `case` con un tipo inventado, `"quest_inventado"`, en el `switch` de `applyEvent`. ¿Qué error da TypeScript? ¿Qué te dice eso de la unión discriminada?

Ejercicio 6.2. Calcula a mano `nextTs` para: (a) `now = 1000`, sin `last`; (b) `now = 1000`, `last.ts = 1000`; (c) `now = 1000`, `last.ts = 50_000`; (d) `now = 1000`, `last.ts = 100_000`. Comprueba con los tests de `src/domain/events.test.ts`.

Ejercicio 6.3. Supón que hay que renombrar `progress_added.amount` a `delta`. Escribe el upcaster de la versión 1 a la 2 y enumera todos los cambios necesarios según la tabla del apartado 6.8.

Ejercicio 6.4. Lee `undoneIn`. ¿Qué pasa si se deshace dos veces el mismo evento? ¿Y si el `event_undone` llega con un `ts` anterior al del evento que deshace?



Parte IV · Persistencia y snapshots

Dónde se guardan los eventos y los archivos, cómo se calcula el estado (entero o evento a evento) y cómo se evita reproducirlo todo en cada arranque.

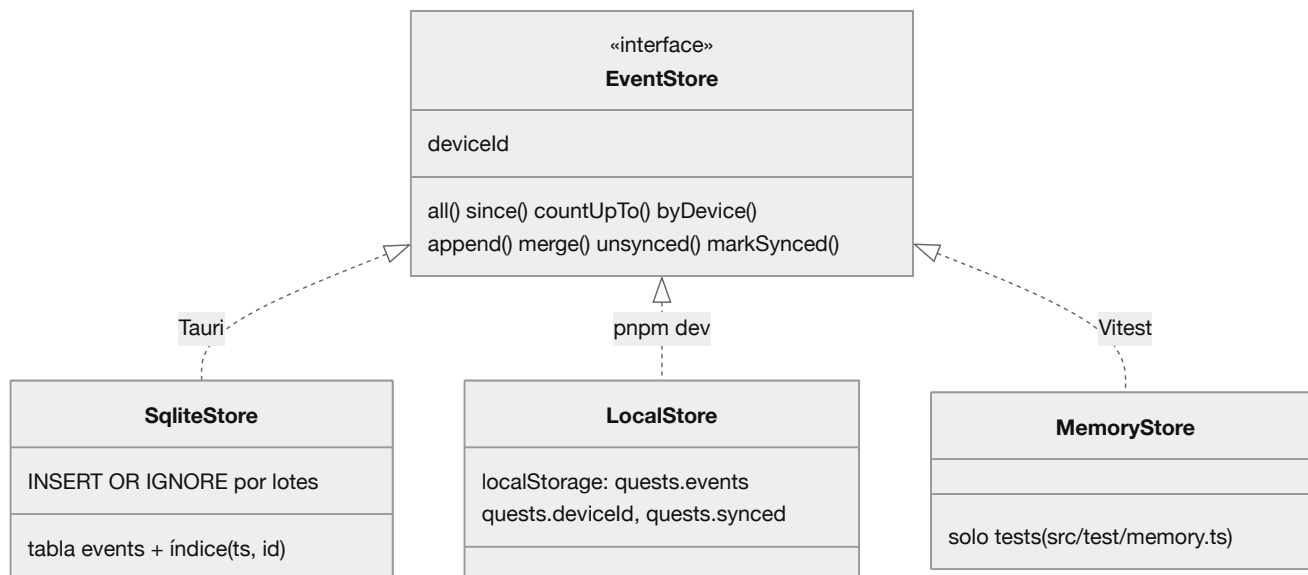
Capítulo 7. EventStore, SQLite y localStorage

7.1 Programar contra una interfaz

El resto de la app **no sabe** si los eventos están en SQLite, en localStorage o en memoria: solo conoce una interfaz, EventStore (src/storage/eventStore.ts):

```
export interface EventStore {
  deviceId: string;
  all(): Promise<GameEvent[]>; // todos, ordenados por (ts, id)
  since(pos: EventPos): Promise<GameEvent[]>; // los posteriores a pos: la cola de un
  snapshot
  countUpTo(pos: EventPos): Promise<number>; // cuántos hay hasta pos (¿vale el snapshot?)
  byDevice(deviceId: string): Promise<GameEvent[]>; // los de un equipo (para subirlos a Drive)
  append(event: GameEvent): Promise<void>; // uno nuevo, local
  merge(events: GameEvent[]): Promise<number>; // remotos: inserta solo los nuevos
  unsynced(): Promise<GameEvent[]>; // los que faltan por subir
  markSynced(ids: string[]): Promise<void>;
}
```

Es un **almacén append-only**: se añade, nunca se modifica ni se borra un evento. Hay tres implementaciones:



`openEventStore()` elige: `isTauri() ? openSQLiteStore() : openLocalStore()`.

Para recién graduados. Es el principio de inversión de dependencias (la «D» de SOLID) y el patrón *Repository*: la lógica depende de una abstracción y la infraestructura se enchufa. Gracias a eso la sincronización se prueba con almacenes en memoria y un Drive falso (capítulo 18), y un futuro backend propio se enchufaría aquí sin tocar la interfaz.

7.2 El esquema de SQLite

Al abrir la base se crean las tablas si no existen:

```
CREATE TABLE IF NOT EXISTS events (
  id          TEXT PRIMARY KEY,      -- UUID del evento
  device_id   TEXT NOT NULL,         -- equipo que lo generó
  ts          INTEGER NOT NULL,      -- reloj lógico híbrido (ms)
  body        TEXT NOT NULL,         -- JSON con type, v y datos
  synced      INTEGER NOT NULL DEFAULT 0 -- 1 = ya subido a Drive
);
CREATE INDEX IF NOT EXISTS events_ts ON events (ts, id);
CREATE TABLE IF NOT EXISTS meta (key TEXT PRIMARY KEY, value TEXT NOT NULL);
```

Decisiones a notar:

- **El cuerpo va como JSON en una columna de texto.** Con decenas de tipos de evento, una tabla por tipo sería inmanejable y cambiaría con cada funcionalidad. Solo lo que se usa para **ordenar y filtrar** va en columnas propias (`id`, `device_id`, `ts`, `synced`).
- **El índice** (`ts`, `id`) coincide con el orden de la proyección: `ORDER BY ts, id` y las consultas de `since/countUpTo` lo usan.
- **La tabla meta** es un almacén clave-valor para lo que no es un evento: el `device_id`, el snapshot, los cursores de la sincronización y la cuenta conectada.

Cómo se leen y se escriben:

```
// Leer: la fila se convierte en evento juntando el JSON con las columnas.
const toEvent = (r: Row) => ({ ...JSON.parse(r.body), id: r.id, deviceId: r.device_id, ts: r.ts }) as GameEvent;

// La cola de un snapshot: lo que va después de (ts, id).
"SELECT id, device_id, ts, body FROM events WHERE ts > $1 OR (ts = $1 AND id > $2) ORDER BY ts, id"
```

7.3 INSERT OR IGNORE e idempotencia

Al escribir, la clave primaria `id` y `INSERT OR IGNORE` hacen que insertar un evento que ya existe **no haga nada**. Por eso merge (fusionar los eventos de otro equipo) es **idempotente**: fusionar dos veces lo mismo da el mismo resultado que una.

```
const res = await db.execute(
  `INSERT OR IGNORE INTO events (id, device_id, ts, body, synced) VALUES ${rows.join(", ")}`,
  params,
);
added += res.rowsAffected; // cuántos eran nuevos de verdad
```

Además se inserta **por lotes** de 150 filas por sentencia (5 parámetros por fila, lejos del límite de 999 parámetros de las versiones antiguas de SQLite). Una sentencia por evento era lenta al fusionar miles.

Idea clave. Idempotencia + ids únicos + orden total + guardas = sincronización sin conflictos. Ninguna de las cuatro basta sola.

7.4 El sustituto del navegador: `localStorage`

Con `pnpm dev` no hay Tauri ni SQLite. `openLocalStore()` cumple el **mismo contrato** sobre `localStorage`:

```
const KEY = "quests.events";
const read = (): GameEvent[] => JSON.parse(localStorage.getItem(KEY) ?? "[]");
const write = (ev: GameEvent[]) => localStorage.setItem(KEY, JSON.stringify(ev));

return {
  deviceId,
  async all() { return read().sort(compareEvents); },
  async append(e) { write([...read(), e]); },
  async merge(events) {
    const ids = new Set(read().map((e) => e.id));
    const fresh = events.filter((e) => !ids.has(e.id)); // el «OR IGNORE» a mano
    write([...read(), ...fresh]);
    return fresh.length;
  },
  // ...
};
```

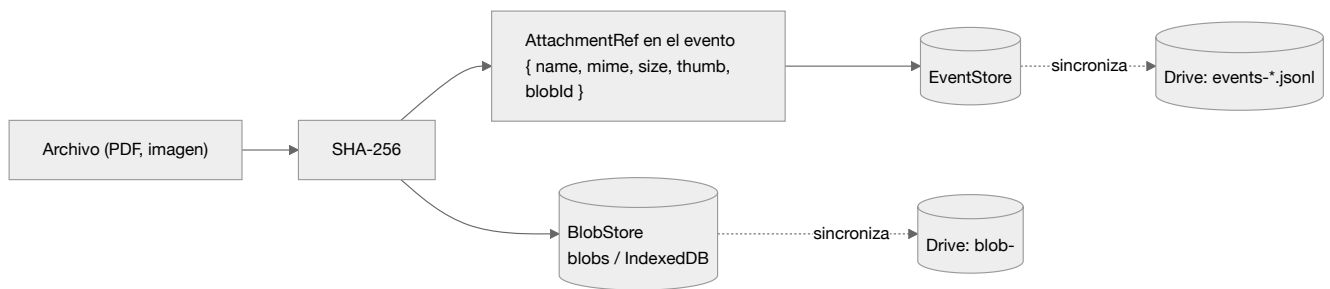
Es deliberadamente simple (lee y escribe la lista entera): solo se usa para desarrollar. Tiene un límite de unos 5 MB por origen, de ahí que los archivos no puedan ir en los eventos.

7.5 Los archivos: el almacén de binarios

Los PDF y las imágenes de los encargos, la imagen grande de los fondos del mercader, las imágenes de los personajes añadidos y la versión nítida de los objetos **no van en los eventos**: un PDF de varios MB se leería en cada arranque y reventaría el `localStorage`. El evento guarda una **referencia** (nombre, tipo, tamaño, una miniatura pequeña y el `blobId`) y el contenido va al **almacén de binarios** (`src/storage/blobStore.ts`):

```
export interface BlobStore {
  put(data: Blob): Promise<string>; // guarda y devuelve su SHA-256
  get(id: string): Promise<Blob | undefined>; // undefined si este equipo aún no lo tiene
  remove(id: string): Promise<void>;
  ids(): Promise<Set<string>>;
  readBase64(id: string): Promise<{ mime: string; data: string } | undefined>;
}
```

- **Direccionado por contenido**: la clave es el **SHA-256** del archivo. El mismo archivo tiene la misma clave en todos los equipos, guardarlo dos veces no ocupa el doble y, al bajarlo de Drive, se puede **verificar** recalculando el hash.
- **En Tauri**: tabla `blobs` en el mismo `quests.db` (`id`, `mime`, `size`, `data` en **base64**, `created`, `synced`), con la misma conexión. Va en base64 porque el puente JavaScript↔Rust viaja en JSON.
- **En el navegador**: IndexedDB (`quests.blobs`), que admite archivos grandes.
- **Limpieza compartida**: antes de borrar un binario hay que comprobar que **nadie** lo usa (`blobsInUse`, en `src/domain/blobs.ts`, junta los de encargos, mercader, personajes y objetos).



Cuidado. La referencia puede llegar a otro equipo **antes** que el archivo (los eventos y los binarios se sincronizan por separado). Por eso la interfaz siempre tiene un plan B: la miniatura incluida en el evento, borrosa, mientras llega la imagen.

7.6 Las imágenes pequeñas sí van en el evento

Hay una excepción razonada: el **icono** de un objeto o de una pieza del mercader se reduce en el navegador a 160 px y va como *data URL* WebP **dentro** del evento (de 3 a 30 KB, ADR-10). Es pequeño, se necesita siempre para pintar y así no depende de que llegue un binario.

Ejercicios

Ejercicio 7.1. Con la app nativa cerrada o en una **copia** de la base, ejecuta (solo lectura) `SELECT json_extract(body, '$.type'), count(*) FROM events GROUP BY 1 ORDER BY 2 DESC;` ¿Qué tipo de evento es el más frecuente? ¿Te sorprende?

Ejercicio 7.2. Escribe un `EventStore` en memoria (un *array*) que cumpla la interfaz. Compáralo después con `memoryEvents` de `src/test/memory.ts`.

Ejercicio 7.3. ¿Por qué `merge` marca como sincronizados los eventos que inserta (`synced = 1`) y `append` no?

Ejercicio 7.4. Explica por qué guardar un PDF en un evento rompería, a la vez, el arranque, el `localStorage` y la sincronización.

Capítulo 8. Proyección completa e incremental

8.1 Partir la proyección en piezas

Conceptualmente, la proyección es `estado = project(eventos)`. En el código está partida en piezas para poder usarla de dos maneras: **completa** (todos los eventos desde cero) e **incremental** (solo el evento nuevo sobre el estado anterior).

Función (src/domain/projection.ts)	Qué hace
<code>newProjectionAcc()</code>	Un acumulador vacío
<code>applyEvent(acc, e)</code>	El <code>switch</code> con sus guardas, para un evento. Modifica <code>acc</code>
<code>pendingSettle(acc) / applySettle(acc, s)</code>	Lo que se deriva entre entidades (encargo de cada quest, reserva, valor de los encargos): calcularlo leyendo y aplicarlo
<code>viewProjection(acc, prev?)</code>	Construye el <code>GameState</code> que lee la interfaz, reutilizando lo que no cambió
<code>finishProjection(acc)</code>	<code>settleProjection</code> + <code>viewProjection</code>
<code>project(events)</code>	Todo junto: acumulador vacío, <code>applyEvent</code> de cada uno (salvo deshechos), <code>finishProjection</code>

8.2 El acumulador: `ProjectionAcc`

El **acumulador** es el estado a medio calcular. Cada funcionalidad aporta su trozo:

```
export interface ProjectionAcc {
  quests: Map<string, QuestState>;
  deletedQuests: Set<string>; // «lápida»: un id retirado no vuelve aunque llegue otro
  quest_created
  items: ItemsAcc; // catálogo, inventario, pity...
  temporals: TemporalAcc; // encargos e ilustraciones
  merchant: MerchantAcc; // catálogo del jugador, comprado
  equipment: EquipmentAcc; // lo que lleva puesto
  attributes: AttributesAcc; // XP por área
  chronicle: ChronicleAcc; // el diario
  agenda: AgendaAcc;
  characters: CharactersAcc; // personajes del menú y sus frases
  styles: StylesAcc; // cómo se mueve cada personaje
  companion: CompanionAcc; // compañero de «Mi día»
  xp: number;
  gold: number;
  completedCount: number;
}
```

Cuidado. El acumulador **solo** guarda datos serializables (`Map`, `Set`, objetos planos y números): se guarda como snapshot (capítulo 9). Nada de clases, funciones ni fechas `Date`.

8.3 `applyEvent`: un evento, con sus guardas

Ya viste fragmentos. Aquí el `case` más rico, `quest_completed`, entero en espíritu:

```

case "quest_completed":
  if (q?.status === "active") { // guarda: solo una vez
    const { reward, item } = upcastReward(e.reward, e.ts);
    acc.xp += reward.xp; // la recompensa COPIADA en el evento
    acc.gold += reward.gold;
    receiveItems(items, reward.itemId, e.drops ?? [], e.ts); // garantizado + botín ya tirado
    gainAttribute(acc.attributes, q.area, reward.xp, e.ts); // atributo del área
    acc.completedCount++;
    q.completions++;
    q.lastCompletedAt = e.ts;
    q.progress = {}; q.checked = {}; q.acceptedAt = undefined;
    q.pomodoros = freshPomodoros(q.conditions);
    if (recurs(q)) { // se repite: espera
      q.status = "cooldown";
      q.availableAt = returnsAt(q, e.ts);
      q.streak = nextStreak(q.streak, e.ts, q.cooldownMinutes, streakUntil(q, e.ts));
    } else {
      q.status = "done";
    }
    record(acc.chronicle, { k: "quest", ts: e.ts, xpAfter: acc.xp, title: q.title, ... });
  }
  break;

```

Un solo evento alimenta **siete** funcionalidades (XP y oro, objetos, atributos, racha, repetición, crónica, contador). Ninguna tiene evento propio: todas se calculan de `quest_completed`. Por eso son **retroactivas**: lo completado antes de que existieran también cuenta.

Otras guardas interesantes miran **otras entidades en ese punto de la historia**:

Evento	Se ignora si...
<code>quest_accepted</code>	faltan requisitos (<code>prerequisitesMet</code>) o está en reserva (<code>inReserve</code>)
<code>temporal_completed</code>	el encargo no está aceptado o una quest enlazada no está terminada (<code>linkDone</code>)
<code>gear_purchased</code> , <code>collectible_purchased</code>	el oro no llega en ese punto , el artículo no existe o ya es tuyo
<code>gear_equipped</code>	la pieza no es tuya
<code>quest_failed</code> , <code>temporal_failed</code>	no ha acabado el día de su fecha, o ya está terminado

Idea clave. Lo que **puede cambiar al fusionar** (el escaparate de esa semana, el rango) **no** se comprueba en la guarda sino en la acción. Si se comprobara en la proyección, una compra legítima hecha sin conexión podría desaparecer al fusionar eventos de otro equipo que cambian el contexto.

8.4 `finishProjection`: lo que depende del conjunto

Hay cosas que no se pueden calcular evento a evento porque dependen de **varias entidades a la vez**:

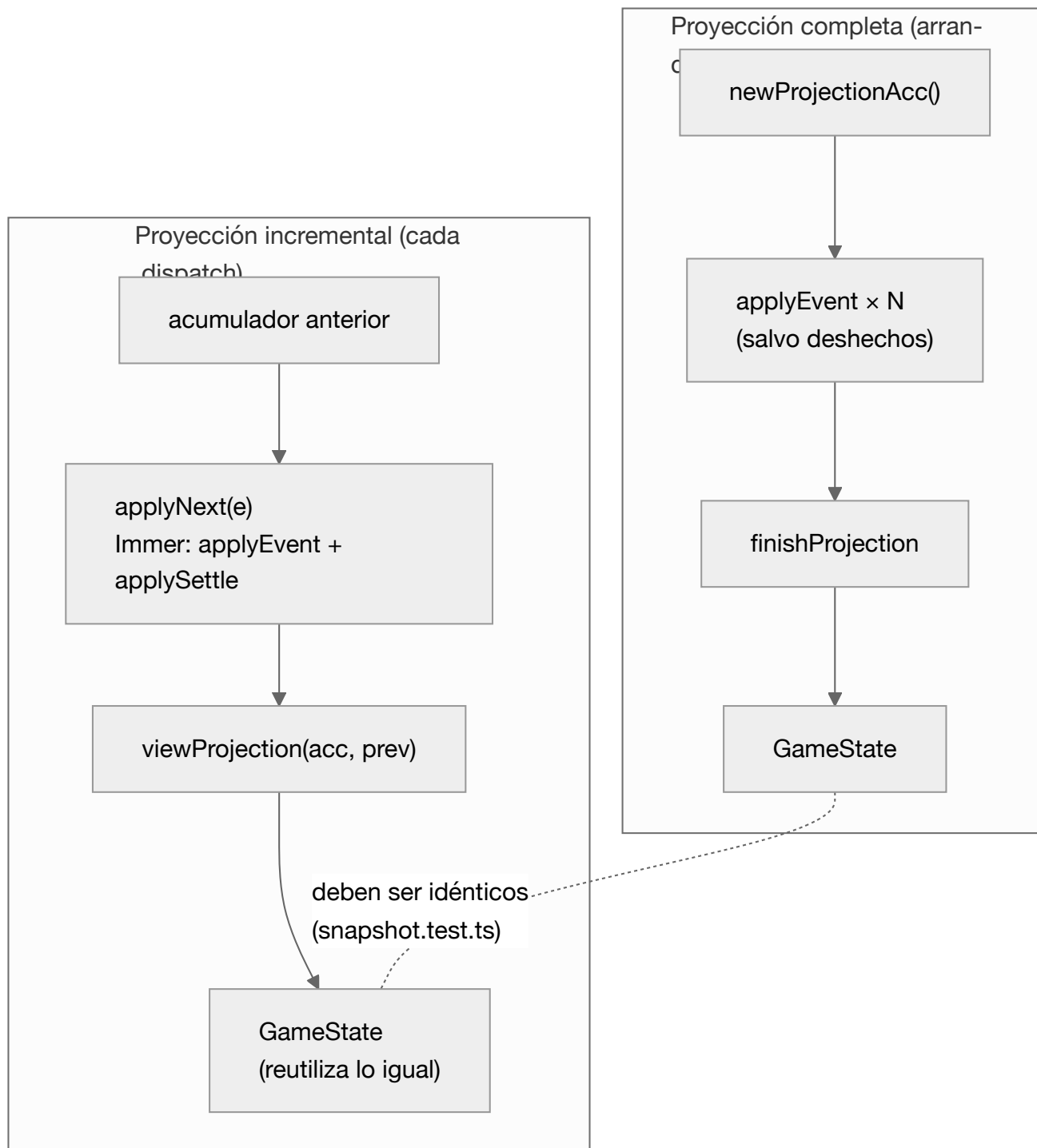
- **El encargo de cada quest** (`temporalId`) y si está **en reserva** (`reserved`): dependen de qué encargos pendientes la enlazan y si están aceptados.
- **El valor de un encargo pendiente**: depende de lo que valen sus quests enlazadas.

- **Nivel y rango:** dependen de la XP total.

`pendingSettle(acc)` calcula **qué** hay que cambiar sin modificar nada (devuelve `undefined` si ya está al día) y `applySettle` lo escribe. Separarlo así permite al store copiar **solo** las quests y encargos que cambian (capítulo 9). `viewProjection` construye el `GameState`:

```
export function viewProjection(acc: ProjectionAcc, prev?: { acc: ProjectionAcc; state: GameState }): GameState {
  const old = prev?.state;
  const lv = levelFromXp(acc.xp);
  const player: PlayerState = { xp: acc.xp, gold: acc.gold, ...lv, rank: rankFor(lv.level), /* ...
*/ };
  const next: GameState = {
    quests: acc.quests,
    items: acc.items.catalog,
    temporals: acc.temporals.board,
    // reutiliza lo que no cambió:
    gear: old && prev.acc.merchant.catalog === acc.merchant.catalog ? old.gear :
fullCatalog(acc.merchant),
    player: old && sameShallow(old.player, player) ? old.player : player,
    // ...
  };
  return old && sameShallow(old, next) ? old : next; // nada cambió: el MISMO objeto
}
```

8.5 Dos caminos, el mismo resultado



La garantía fundamental, comprobada por los tests: **aplicar los eventos uno a uno da exactamente lo mismo que reproducirlos todos de golpe**. Un detalle sutil que lo hace posible: al cerrar un encargo (cumplirlo o quemarlo) su valor se **fija** primero (`fixValue`), porque si no dependería de si se había «asentado» antes (evento a evento) o no (reproduciendo todo).

8.6 Cuándo no se puede ir incremental

El camino incremental solo vale si el evento nuevo va **después** de todo lo aplicado:

```
export const goesAfter = (p: Pick<Projected, "last">, e: EventPos) => !p.last || comparePos(e, p.last) > 0;
```

Hay tres casos en que no, y entonces se recalcula todo (`rebuild()`):

1. **Reloj muy atrasado:** el `ts` cae en medio del historial (más allá de la deriva del HLC).
2. **Deshacer** (`event_undone`): cambia el pasado.
3. **Eventos fusionados de otro equipo:** tras `merge`, la sincronización llama a `rebuild()`.

Mientras se recalcula, los `dispatch` que lleguen **esperan** (`if (rebuilding) await rebuilding`).

Ejercicios

Ejercicio 8.1. Escribe a mano el `GameState` (solo `player.xp`, `player.level` y el estado de la quest) tras cada evento de esta lista: crear quest de 150 XP, aceptarla, completarla, completarla otra vez.

Ejercicio 8.2. ¿Por qué `deletedQuests` es un `Set` en el acumulador y no se calcula? Piensa en dos equipos que arrancan vacíos (ids fijos de los ejemplos) y uno retira una quest.

Ejercicio 8.3. Busca `fixValue` y construye un ejemplo de dos historias (incremental y completa) que darían valores distintos a un encargo cumplido si no existiera.

Capítulo 9. Snapshots, Immer y rendimiento

9.1 El problema de crecer

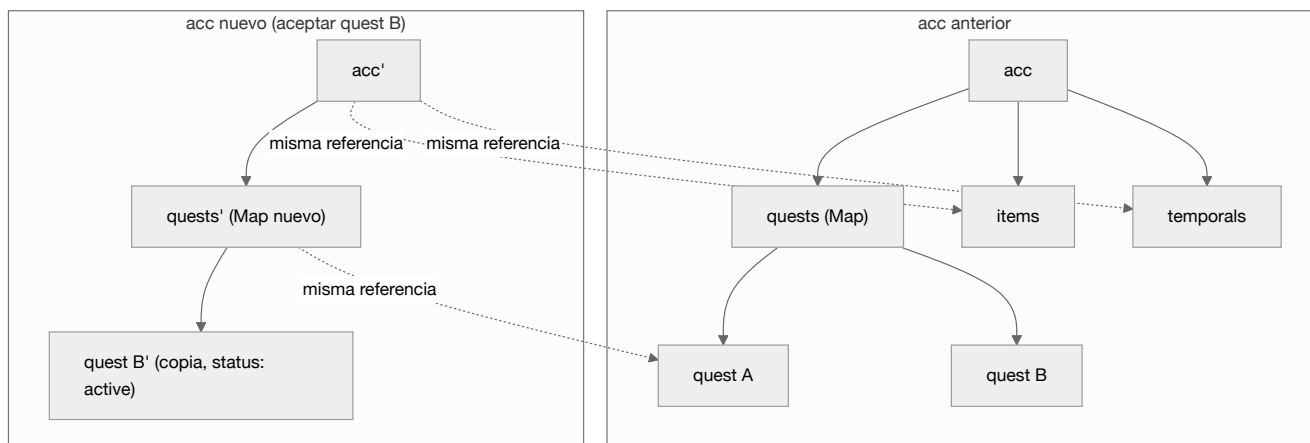
Con event sourcing ingenuo, el coste de **arrancar** crece con el historial (hay que reproducirlo entero) y el coste de **cada acción** también (si recalculas todo el estado). Quests resuelve los dos con dos técnicas independientes (`features/snapshot` , ADR-16 y ADR-54):

1. **Snapshot al arrancar**: se guarda el acumulador cada 100 eventos y al abrir se aplican solo los posteriores.
2. **Copia selectiva en cada dispatch**: se aplica solo el evento nuevo y se copia solo lo que cambia, con **Immer**.

9.2 Inmutabilidad y por qué importa en React

React (y `memo`, y los selectores de Zustand) deciden si algo cambió comparando **referencias**: `antes === después`. Si modificas un objeto **en su sitio** (`q.status = "active"`), la referencia no cambia y React cree que no ha pasado nada. Si **copias todo** el estado en cada acción, todo parece haber cambiado y React repinta todo.

Lo ideal es la **compartición estructural**: copiar solo el camino de lo que cambia y reutilizar el resto.



Solo la quest B, el Map de quests y la raíz son objetos nuevos. La tarjeta de la quest A recibe **la misma** referencia y, con `memo`, no se repinta.

9.3 Immer: escribir como si mutaras

Escribir compartición estructural a mano es tedioso y propenso a errores. **Immer** permite escribir código que *parece* mutar un **borrador** (*draft*) y produce un objeto nuevo copiando solo lo tocado:

```
import { produce } from "immer";
const nuevo = produce(viejo, (draft) => {
  draft.quests.get("B").status = "active"; // parece una mutación...
});
// ...pero viejo no ha cambiado, y nuevo comparte con viejo todo lo demás.
```

Lo elegante en Quests es que `applyEvent` **ya está escrito mutando** el acumulador (es lo natural para la proyección completa). Con Immer, la misma función sirve para el camino incremental sin reescribirla:

```
// src/features/snapshot/model.ts
enableMapSet(); // Immer con Map y Set
const immer = new Immer({ autoFreeze: false }); // sin congelar (recorrería todo el árbol)

export function applyNext(p: Projected, e: GameEvent): Projected {
  let acc = immer.produce(p.acc, (draft) => applyEvent(draft as ProjectionAcc, e));
  // Lo derivado entre entidades se calcula LEYENDO y solo se escribe lo que difiere.
  const settle = pendingSettle(acc);
  if (settle) acc = immer.produce(acc, (draft) => applySettle(draft as ProjectionAcc, settle));
  return { acc, last: { ts: e.ts, id: e.id }, count: p.count + 1 };
}
```

Y la guinda: si un evento no cambia nada (por ejemplo, uno ignorado por su guarda), Immer devuelve **el mismo objeto**, `dispatch` lo detecta (`next.acc === projected.acc`) y la interfaz ni se entera.

Cuidado. La compartición estructural tiene una regla dura: **nadie escribe en el estado fuera de** `applyEvent` (dentro de un borrador). Si un componente hiciera `q.status = "x"` sobre el estado, estaría escribiendo en un objeto **compartido** con el estado anterior y con el snapshot. Los tests congelan el acumulador a propósito para cazar estas escrituras.

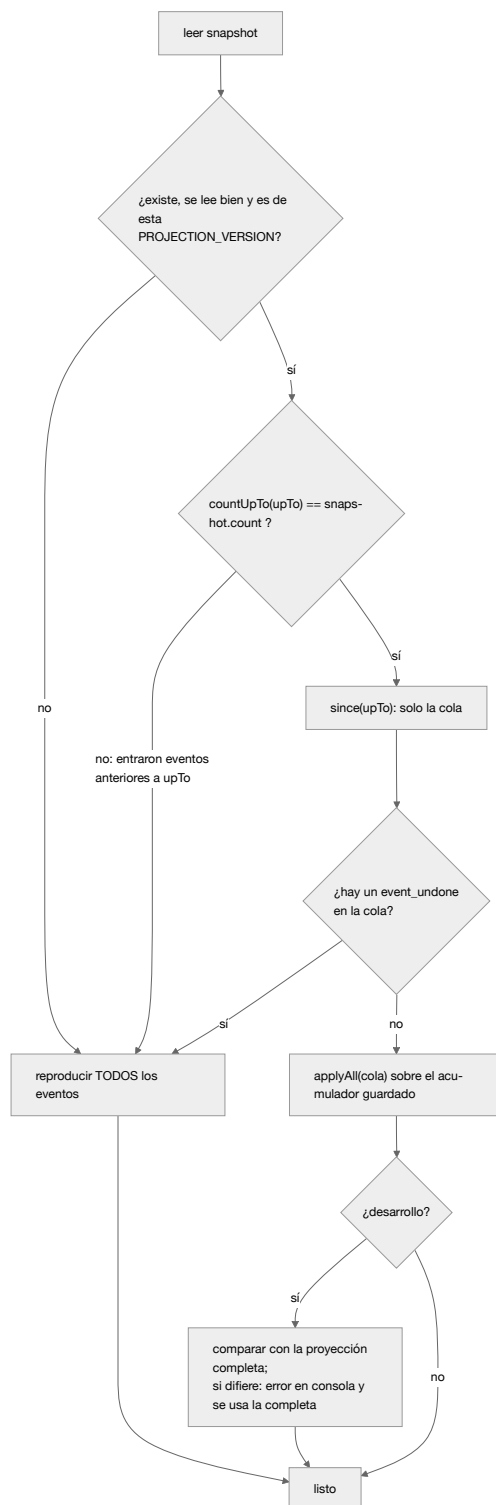
9.4 El snapshot

Un **snapshot** es una copia guardada del acumulador hasta un evento. Es una **caché**: si falta o no vale, se recalcula todo sin perder nada (los eventos siguen siendo la verdad).

```
export interface Snapshot {
  format: number; // SNAPSHOT_FORMAT: la forma de este objeto
  projection: number; // PROJECTION_VERSION con la que se calculó
  upTo: EventPos; // último evento incluido
  count: number; // cuántos eventos incluye
  acc: ProjectionAcc;
  savedAt: number;
}
```

- **Cuándo se guarda:** cada 100 eventos (`SNAPSHOT_EVERY`) desde el anterior, tras un `rebuild()` y al arrancar si se recalculó todo con 100 o más. En cola, de uno en uno.
- **Dónde:** en Tauri, la fila `snapshot` de la tabla `meta`; en el navegador, `localStorage["quests.snapshot"]`.
- **Serialización:** JSON no conserva `Map`, `Set` ni `±Infinity`. Se marcan con una etiqueta (`{ "__snap": "map", "v": [...] }`) en el `replacer` y se reconstruyen en el `reviver`.

9.5 El arranque con snapshot



¿Por qué el **recuento**? Si desde el snapshot entró algún evento con `ts` anterior a `upTo` (fusionado de otro equipo, reloj atrasado), el snapshot ya no representa la historia. Como los eventos **nunca se borran**, basta contar: si hay más eventos hasta `upTo` de los que tenía el snapshot, no vale. Es un `count(*)` sobre el índice `(ts, id)`: barato.

9.6 PROJECTION_VERSION: invalidar la caché

Si cambias cómo `project()` interpreta eventos ya guardados (una guarda, un `case`, una fórmula de `applyEvent`), los snapshots viejos contienen un resultado calculado con la lógica antigua.

`PROJECTION_VERSION` lo resuelve: un snapshot de otra versión **se descarta**.

Cuidado. Si olvidas subirla, en desarrollo te avisará la comparación del arranque (y fallarán los tests de `snapshot + cola`), pero en la app nativa del propietario se arrancaría **con el estado viejo** hasta el siguiente `rebuild()`. Es de los errores más silenciosos posibles.

9.7 Rendimiento en la interfaz

La copia selectiva solo sirve si la interfaz la aprovecha:

- **Selectores finos** en Zustand (capítulo 4): cada componente sigue solo lo suyo.
- `memo` en listas grandes (`QuestCard`) y en las piezas fijas de `App`.
- **Manejadores estables** (`useCallback`) para no romper el `memo`.
- `viewProjection` **reutiliza** `player`, `gear`, `companion` y el propio `GameState` cuando no cambian.
- **Partículas fuera de React** (`src/lib/fx.ts`): cientos de `<span>` animados con GSAP sin que React los reconcilie (capítulo 14).

9.8 Lo que queda por medir

El informe técnico (`docs/INFORME-TECNICO.md`, «Escalabilidad») lista con honestidad dónde se notará el crecimiento: un evento antiguo fusionado obliga a reproducirlo todo; cada deshacer recalcula todo; el archivo de sincronización de cada equipo se reescribe entero. Las medidas propuestas (recalcular desde el snapshot anterior en vez de desde cero, partir el archivo por meses) están en la hoja de ruta. La norma del propietario para optimizar: **medir primero** y proponer el antes y el después con cifras.

Ejercicios

Ejercicio 9.1. En `src/features/snapshot/snapshot.test.ts`, localiza el test que congela el acumulador. ¿Qué error saltaría si alguien escribiera fuera de un borrador?

Ejercicio 9.2. Con `pnpm dev`, crea más de 100 eventos (por ejemplo, sumando y restando progreso) y busca `quests.snapshot` en el `localStorage`. ¿Qué `count` y qué `upTo` tiene? Recarga y comprueba en la consola que no aparece el aviso de discrepancia.

Ejercicio 9.3. Escribe un `replacer/reviver` mínimo que serialice un `Map<string, Set<number>>` y vuelva a construirlo. Compáralo con el del proyecto.

Ejercicio 9.4. En una rama de pruebas y con un snapshot ya guardado en el navegador que incluya alguna quest completada, cambia el case "quest_completed" para sumar el doble de XP (`acc.xp += reward.xp * 2`) **sin** subir `PROJECTION_VERSION`, y recarga. ¿Qué ves en la consola? Explica el mecanismo (`verifyAgainstFull`) y qué habría pasado en la app nativa. Deshaz el cambio.

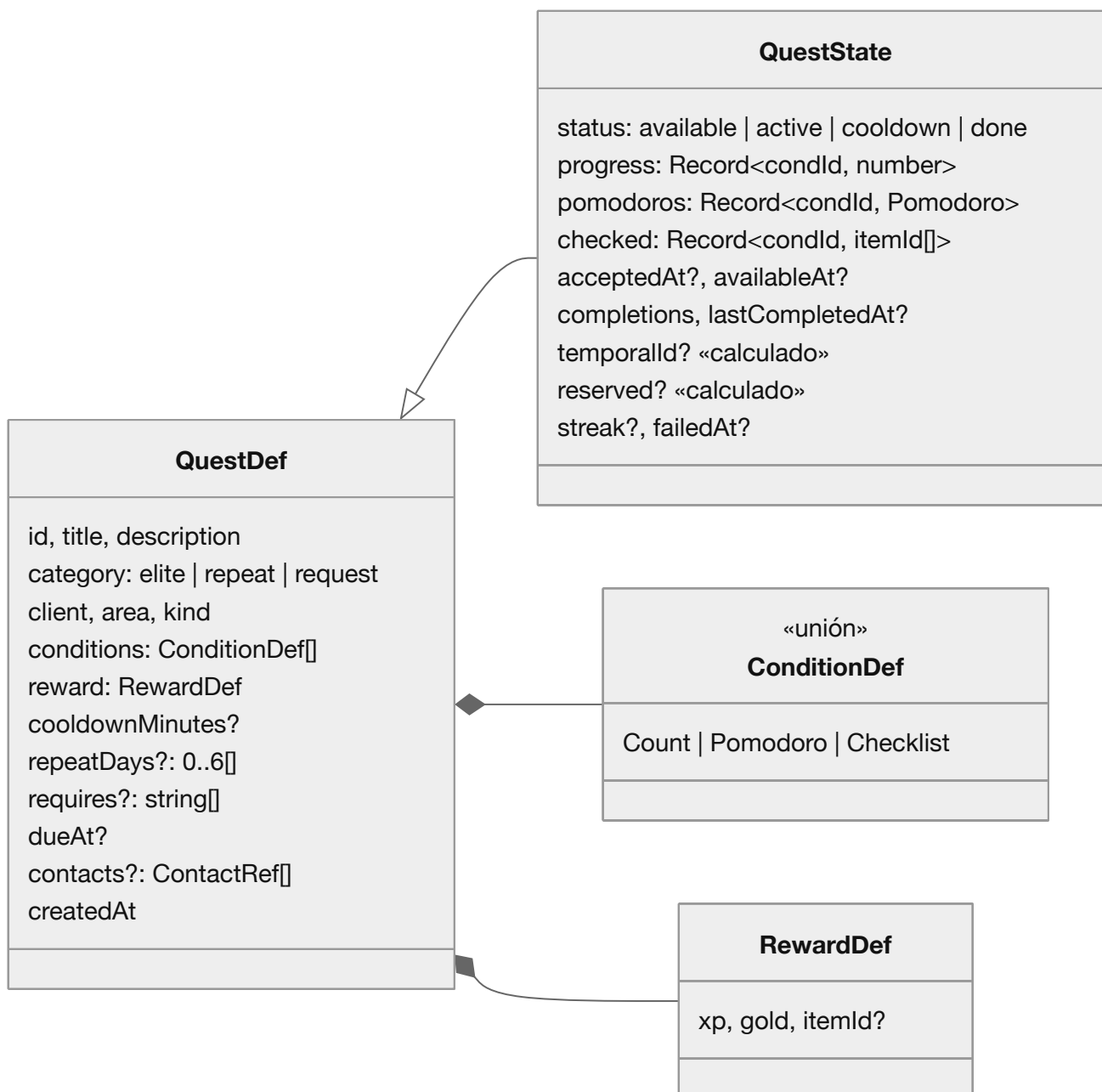
Parte V · Funcionalidades del juego

El modelo de una quest, la economía de XP y oro, el recorrido completo de una acción y las treinta y tres funcionalidades de `src/features/`.

Capítulo 10. Modelo de quests y condiciones

10.1 Definición y estado: dos tipos distintos

El modelo separa **lo que se define** de una quest (lo que el usuario escribe y viaja en `quest_created`) de **su estado de juego** (lo que calcula la proyección):



`QuestState` extends `QuestDef`: el estado lleva la definición más lo que cambia al jugar. Son `interface` de TypeScript, no clases: la lógica vive en funciones puras (`effectiveStatus`, `conditionsMet...`), porque el estado tiene que poder serializarse como JSON y reconstruirse desde eventos.

10.2 Las tres categorías

Categoría	Etiqueta	Color	Peso en la recompensa	Botín
elite	ELITE	Rojo (—elite)	×2	Tabla mejorada, 2 tiradas
request	REQUEST	Verde (—request)	×1	Tabla estándar, 1 tirada
repeat	REPEAT	Violeta (—repeat)	×0,5	Tabla estándar, 1 tirada; vuelve tras su espera

La categoría es una **etiqueta** (color, botón, peso): cualquier quest puede repetirse si tiene repetición, y una élite que se repite cada semana es un «jefe semanal».

10.3 Los objetivos: una unión de tres tipos

Cada quest tiene una lista de objetivos (conditions). Hay tres tipos, distinguidos por `kind`:

```
interface CountConditionDef {           // «Leer páginas ×20»
  id: string; kind?: "count"; label: string; target: number;
}
interface PomodoroConditionDef {        // «Estudiar: 4 rondas de 25 + 5»
  id: string; kind: "pomodoro"; label: string;
  target: number;                       // rondas
  focusMinutes: number; breakMinutes: number;
}
interface ChecklistConditionDef {        // «Maleta: pasaporte, cargador, JR Pass...»
  id: string; kind: "checklist"; label: string;
  items: { id: string; text: string }[];
  target: number;                       // = número de casillas
}
type ConditionDef = CountConditionDef | PomodoroConditionDef | ChecklistConditionDef;
```

Cuidado. En el contador, `kind` es **opcional**: los datos antiguos no lo traen. Por eso existe `isCountCondition = (c) => c.kind === undefined || c.kind === "count"`. Nunca compares `c.kind === "count"` a pelo.

Cada tipo avanza de forma distinta, pero todos exponen un **progreso** y un **objetivo** (`target`), así que una sola función decide si la quest está lista:

```
export function conditionProgress(q: QuestState, c: ConditionDef, now: number): number {
  if (isChecklistCondition(c)) return checklistProgress(q.checked ?? {}, c); // casillas
  marcadas
  if (!isPomodoroCondition(c)) return q.progress[c.id] ?? 0;                // contador
  const p = q.pomodoros[c.id];
  return p ? viewPomodoro(p, planOf(c), now).completedRounds : 0;           // rondas (con la
  hora)
}

export function conditionsMet(q: QuestState, now: number): boolean {
  return q.conditions.every((c) => conditionProgress(q, c, now) >= c.target);
}
```

Tipo	Cómo avanza	Evento	Dónde vive el progreso
Contador	± 1	progress_added (delta, limitado a [0, objetivo])	progress[condId]
Lista	Casilla a casilla	checklist_checked { itemId, done } (un valor por casilla)	checked[condId]
Pomodoro	Con el tiempo	pomodoro_started/paused/resumed/stopped/break_skipped	pomodoros[condId]

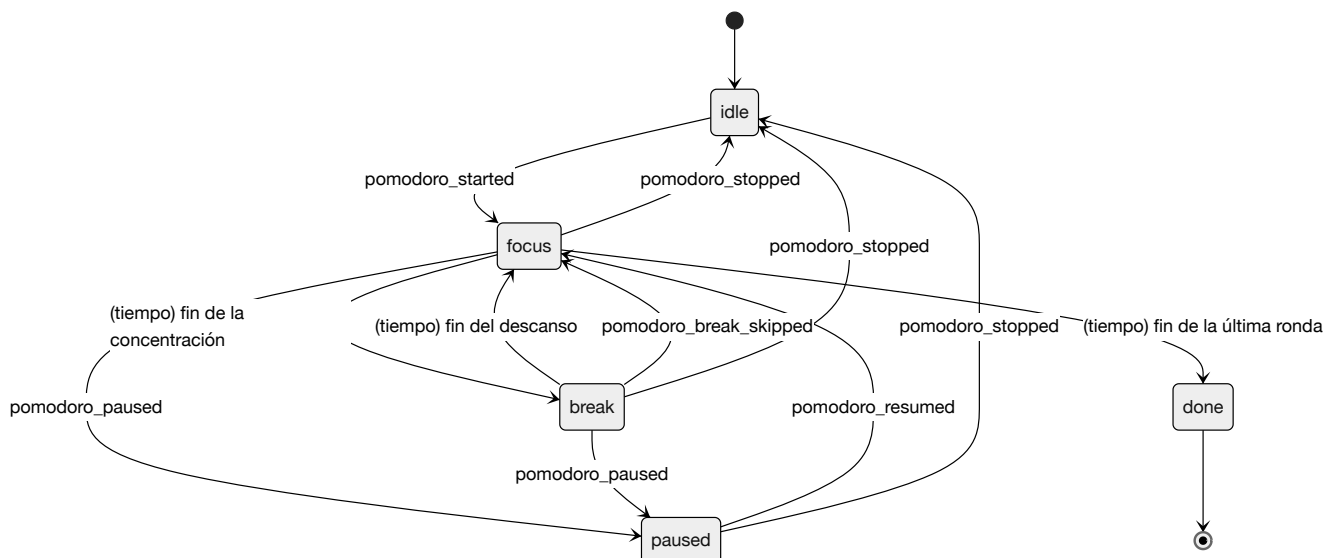
10.4 El pomodoro: una línea de tiempo calculada

El pomodoro es el objetivo más interesante porque **avanza solo con el tiempo** y aun así no genera eventos en cada fase. El truco: toda la secuencia (concentración, descanso, concentración..., sin descanso tras la última) es **una línea de tiempo continua**, y el estado guarda solo cuánto se ha avanzado:

```
interface Pomodoro {
  status: "idle" | "running" | "paused";
  offsetMs: number; // avance acumulado antes del tramo actual
  runningSince?: number; // cuándo empezó a correr el tramo actual
  lastPartialMs?: number; // concentración de la última ronda interrumpida
}
```

Y la vista se calcula con la hora:

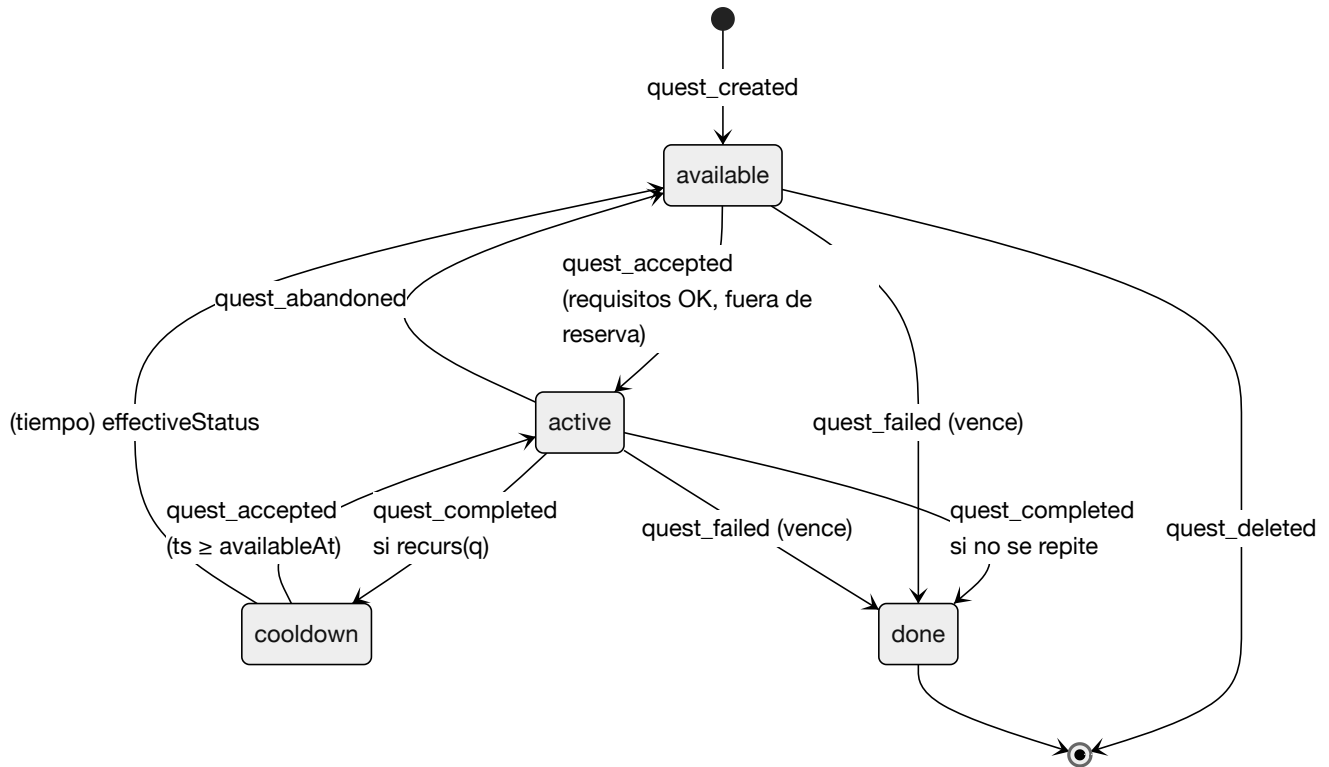
```
F = concentración, B = descanso, N = rondas, ciclo = F + B
total = N·F + (N - 1)·B
t = offsetMs + (corriendo ? now - runningSince : 0)
ronda actual = ⌊t / ciclo⌋ + 1
tramo = (t mod ciclo) < F ? concentración : descanso
rondas completas = min(N, ⌊(t + B) / ciclo⌋)
terminado = t ≥ total
```



Las transiciones con nombre de evento se guardan; las marcadas *(tiempo)* se calculan. Por eso el pomodoro sigue corriendo aunque cierres la app y dos equipos con los mismos eventos ven lo mismo.

Ejercicio. Con $F = 25$, $B = 5$ y $N = 4$, calcula la ronda, el tramo y las rondas completas cuando $t = 0$, 25, 29, 30, 60 y 115 minutos.

10.5 El ciclo de vida de una quest



- **cooldown** no se convierte en **available** con un evento: lo decide `effectiveStatus(q, now)` al pintar.
- **Bloqueada** (le faltan requisitos) y **en reserva** (su encargo no está aceptado) **no son estados**: la quest sigue **available**, pero no se puede aceptar. Se calculan (`blockers`, `reserved`).
- **Fallida**: `status: "done"` con `failedAt`, sin recompensa.

10.6 Repetición, requisitos, fecha límite y contactos

Son campos **opcionales** de `QuestDef` (no hay eventos nuevos: viajan en `quest_created` y cambian con `quest_updated`). Los datos antiguos no los traen y se proyectan igual, sin `upcaster`.

Repetición (`features/complex`):

```
recurs(q) = q.category === "repeat" || (q.cooldownMinutes ?? 0) > 0 || !!q.repeatDays?.length
```

- `cooldownMinutes`: vuelve pasado ese tiempo desde que se completó.
- `repeatDays` (0 = domingo ... 6 = sábado): vuelve a medianoche del siguiente día de la lista. Manda sobre `cooldownMinutes`.
- Una quest que se repite **no tiene fecha límite** (tras la primera vuelta quedaría vencida para siempre).

Requisitos (`requires: string[]`): ids de quests que hay que completar antes. Un requisito se cumple con **una** completación, o si se retira o se fractura (si no, bloquearía para siempre). La proyección quita los que cerrarían un **ciclo** (`wouldCycle`): A requiere B y B requiere A.

Fecha límite (`dueAt`, `features/horizon`): medianoche local del día elegido; vence al acabar ese día. Si la quest pertenece a un encargo, cuenta la fecha que llegue antes.

Contactos (`contacts`, `features/contacts`): teléfono, correo, WhatsApp, enlace o dirección, con su botón para abrirlos con la app del sistema (solo los esquemas `tel:`, `mailto:` y `https:`).

10.7 La recompensa la calcula el juego

Una decisión del propietario: **nadie escribe la recompensa a mano**. Sale de los objetivos y de la categoría (`features/rewards`):

Objetivo	XP	Oro
Pomodoro	1 por minuto de concentración	18 por minuto
Contador $\times N$	$25 \cdot \sqrt{N}$	$450 \cdot \sqrt{N}$
Lista	15 por casilla	270 por casilla

Quest = suma de sus objetivos $\times$ peso de la categoría, redondeado a 5. La raíz del contador evita que un « $\times 999$ » rompa la economía ($\times 1 = 25$ XP, $\times 9 = 75$, $\times 100 = 250$).

La proyección **sustituye** la recompensa de la definición por la calculada al aplicar `quest_created` (`questReward`), y `quest_completed` **copia** la del momento. Así, si un día se cambia `REWARD_RATES`, todas las pantallas enseñan la nueva cifra para lo pendiente, pero lo ya ganado no cambia.

Cuidado. `REWARD_RATES`, `CATEGORY_FACTOR`, `TEMPORAL_GOLD_FACTOR`, los precios del mercader y las probabilidades del botín **no se tocan sin preguntar al propietario**. Y si cambias la fórmula, sube `PROJECTION_VERSION`.

Ejercicios

Ejercicio 10.1. Calcula la recompensa de una élite con un pomodoro de 3 rondas de 90 min y un contador $\times 9$. Comprueba con la tabla de ejemplos del README de `rewards` (690 XP $\cdot$ 12.420 G).

Ejercicio 10.2. ¿Por qué el contador usa `progress_added` con un delta y la lista usa `checklist_checked` con un valor (`done: true/false`)? Piensa en dos equipos que marcan la misma casilla.

Ejercicio 10.3. Dibuja el estado de una quest «lunes y jueves» completada el lunes a las 10:00: ¿qué status y qué `availableAt` tiene? ¿Qué devuelve `effectiveStatus` el martes y el jueves a las 00:01?

Capítulo 11. XP, niveles y rangos

11.1 La curva de experiencia

La XP necesaria para pasar de un nivel al siguiente crece algo más rápido que lineal

(src/domain/leveling.ts):

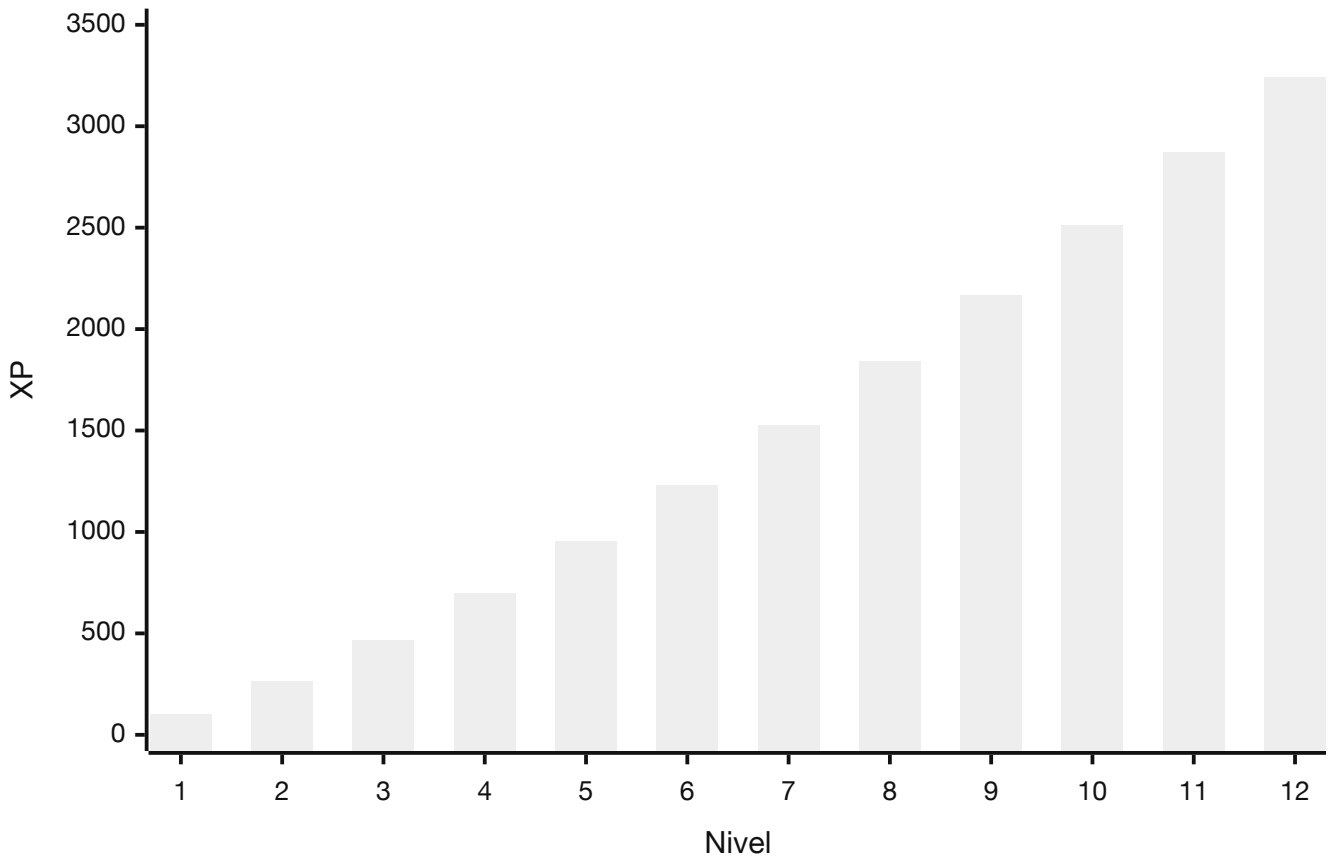
```
export function xpToNext(level: number): number {  
  return Math.round(100 * Math.pow(level, 1.4));  
}
```

levelFromXp(total) va restando nivel a nivel y devuelve el nivel, la XP dentro del nivel y la que falta:

```
export function levelFromXp(totalXp: number) {  
  let level = 1;  
  let remaining = totalXp;  
  while (remaining >= xpToNext(level)) {  
    remaining -= xpToNext(level);  
    level++;  
  }  
  return { level, levelXp: remaining, levelXpNeeded: xpToNext(level) };  
}
```

Nivel	XP para subir	XP total al llegar	Rango	
1		100	0	F
2		264	100	F
3		466	364	E
4		696	830	E
5		952	1.526	D
6		1.229	2.478	D
8		1.838	5.232	C
10		2.512	9.237	C
12		3.242	14.619	B

XP para subir de nivel: $100 \cdot n^{1,4}$



11.2 Rangos

Los rangos van por umbrales de nivel:

```
const RANKS: [number, string][] = [[24, "S"], [17, "A"], [12, "B"], [8, "C"], [5, "D"], [3, "E"], [1, "F"]];  
export const rankFor = (level: number) => RANKS.find(([, min]) => level >= min)![1];
```

El rango no es solo decorativo: es el **requisito** de las piezas del mercader (una legendaria pide rango A, nivel 17).

11.3 Por qué el nivel no se guarda

El nivel y el rango se calculan en `finishProjection` a partir de la XP total. Consecuencia práctica: **cambiar la curva** (la base 100 o el exponente 1,4) se aplica a todo lo ya jugado y **no** obliga a subir `PROJECTION_VERSION` (porque `finishProjection` se ejecuta siempre, también sobre un snapshot). Pero cambia el ritmo del juego: es una decisión del propietario.

11.4 Atributos: un nivel por área

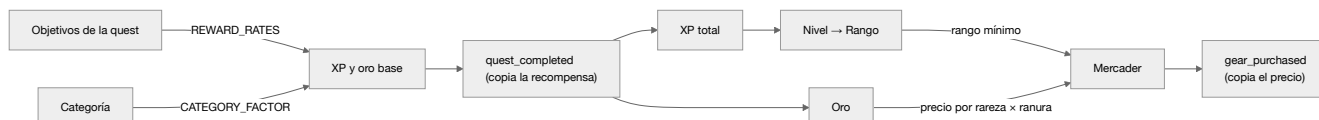
Cada **área** de las quests («Salud», «Estudio»...) es un **atributo** con su propio nivel (features/attributes). La XP de cada `quest_completed` se suma también a su área, con una curva más suave porque cada área recibe solo una parte:

$$\text{attributeXpToNext}(n) = 60 \cdot n^{1,3}$$

Las áreas las escribe el usuario, así que se **normalizan** (sin espacios de más y en minúsculas: «Salud», « salud » y «SALUD» son la misma) y hay 17 **áreas conocidas** con sinónimos en español, japonés e inglés: «Salud» y «健康» suben el mismo atributo. Se dibujan en un radar con las seis áreas con más XP

11.5 La economía: XP, oro y precios

El oro existe para **comprar**. El propietario fijó un objetivo de diseño: «comprar tiene que costar, pero ser habitual». La cadena completa:



Cifras de calibración (de los README de `rewards` y `merchant`): un día bueno (4–5 h de concentración) da unos **6.500 G**; una pieza común cuesta 1.200 G y una legendaria 100.000 G (más el recargo de la ranura); el catálogo de serie entero se compra en un año de días buenos.

Para recién graduados. Es un sistema con dos «monedas» (XP para el rango, oro para el precio) que frenan cosas distintas: al principio frena el rango; más adelante, el oro. Diseñar una economía es ajustar esos frenos con números medidos, no a ojo.

Ejercicios

Ejercicio 11.1. Implementa (en papel o en un test) `xpTotalPara(nivel)` y comprueba la columna «XP total al llegar» de la tabla.

Ejercicio 11.2. ¿Cuántas quests de tipo «encargo con 3 pomodoros de 90 min» (270 XP) hacen falta para llegar al rango C?

Ejercicio 11.3. Si mañana se cambiara el exponente a 1,5, ¿qué pasaría con el nivel de un jugador con 10.000 XP? ¿Y con el oro que ya ganó? ¿Habría que subir `PROJECTION_VERSION`?

Capítulo 12. Acciones y flujo completo de usuario

12.1 Qué es una acción

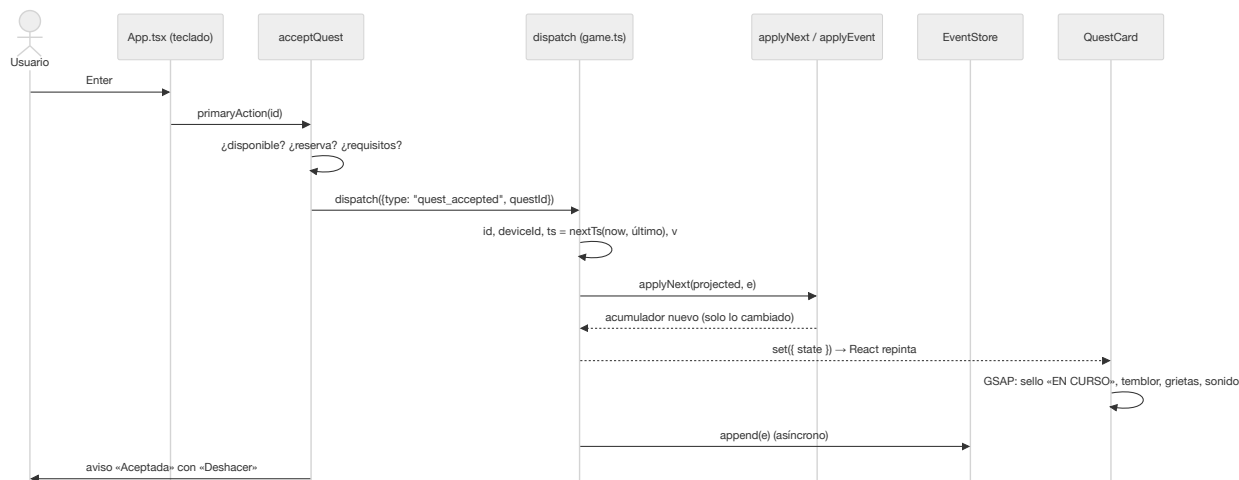
Una **acción** es un caso de uso: la función que se ejecuta cuando el usuario hace algo. Vive en `src/store/actions.ts` (las del núcleo) o en el `actions.ts` de cada funcionalidad. Todas siguen el mismo esquema:

1. **Leer** el estado (`useGame.getState()`).
2. **Validar** y, si no se puede, **avisar** (sonido de cancelar y un aviso traducido).
3. **Resolver lo no determinista** (azar, hora de la acción) y guardarlo en el evento.
4. **Emitir** el evento con `dispatch`.
5. **Reaccionar**: ofrecer «Deshacer», abrir una animación, avisar de lo que se desbloquee.

Idea clave. La validación existe **dos veces**: en la acción (para avisar al usuario con un buen mensaje) y en la guarda de la proyección (para que un evento imposible, venga de donde venga, no rompa nada). No es redundancia: protegen cosas distintas.

12.2 Aceptar una quest

```
export async function acceptQuest(id: string) {
  const { state, dispatch, say } = useGame.getState();
  const q = state.quests.get(id);
  if (!q || effectiveStatus(q, Date.now()) !== "available") return;
  if (q.reserved) { // su encargo no está aceptado
    sfx.cancel();
    say(() => i18n.t("temporal.toast.reserved", { title: q.title, temporal: ... }));
    return;
  }
  const lock = blockers(q, state.quests); // requisitos pendientes
  if (lock.length) {
    sfx.cancel();
    say(() => i18n.t("complex.toast.locked", { title: lock[0].title }));
    return;
  }
  const e = await dispatch({ type: "quest_accepted", questId: id });
  offerUndo(e, () => i18n.t("toast.accepted", { title: q.title })); // aviso con «Deshacer»
}
```



12.3 Avanzar un objetivo

```

export async function addProgress(id: string, conditionId: string, amount: number) {
  const { state, dispatch } = useGame.getState();
  const q = state.quests.get(id);
  const c = q?.conditions.find((c) => c.id === conditionId);
  if (!q || !c || !isCountCondition(c) || q.status !== "active") return;
  const cur = q.progress[c.id] ?? 0;
  if (cur + amount < 0 || cur + amount > c.target) return; // la acción no emite eventos
  inútiles
  sfx.tick();
  await dispatch({ type: "progress_added", questId: id, conditionId, amount });
}

```

El atajo **+** llama a `bumpNext`, que avanza el **primer objetivo incompleto**: +1 a un contador o la siguiente celda de una lista.

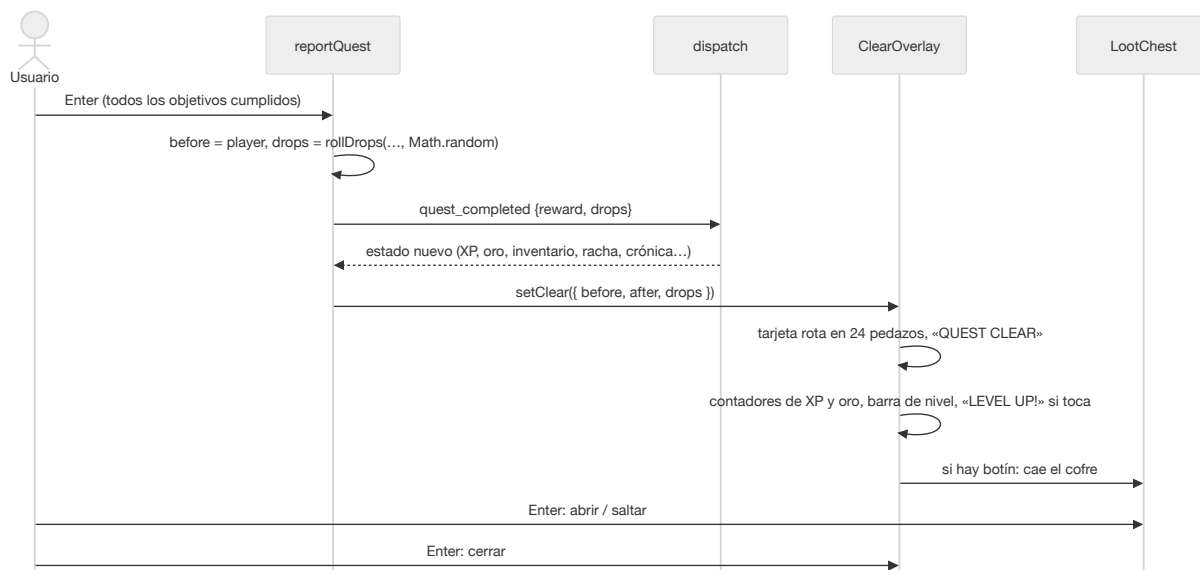
12.4 Reportar: la acción más completa

```

export async function reportQuest(id: string) {
  const { state, dispatch, setClear, say } = useGame.getState();
  const q = state.quests.get(id);
  if (q?.status !== "active" || !conditionsMet(q, Date.now())) return;
  const before = state.player; // 1. jugador ANTES
  const drops = rollDrops(dropTableFor(q.category), state.items.values(), before.pity,
    Math.random()); // 2. azar
  await dispatch({ type: "quest_completed", questId: id, reward: q.reward, drops });
  // 3. evento
  const after = useGame.getState().state; // 4. jugador DESPUÉS
  setClear({ questId: id, before, after: after.player, guaranteed, drops }); // 5. «Quest
  Clear»
  say(() => i18n.t("toast.completed", { title: q.title }));
  // 6. ¿Racha redonda? ¿Desbloquea otra quest? ¿Su encargo ya se puede cumplir?
  const next = unlockedBetween(state.quests, after.quests)[0];
  if (next) say(() => i18n.t("complex.toast.unlocked", { title: q.title, next: next.title }));
  // ...
}

```

El truco de guardar el jugador **antes** y **después** es lo que permite a `ClearOverlay` animar la barra de XP desde donde estaba hasta donde llega, detectar una **subida de nivel** y mostrar el oro ganado con un contador. La interfaz no recalcula nada: compara dos fotos.



12.5 La acción principal

`Enter` o `A` sobre una quest no sabe si aceptar o reportar; lo decide el estado:

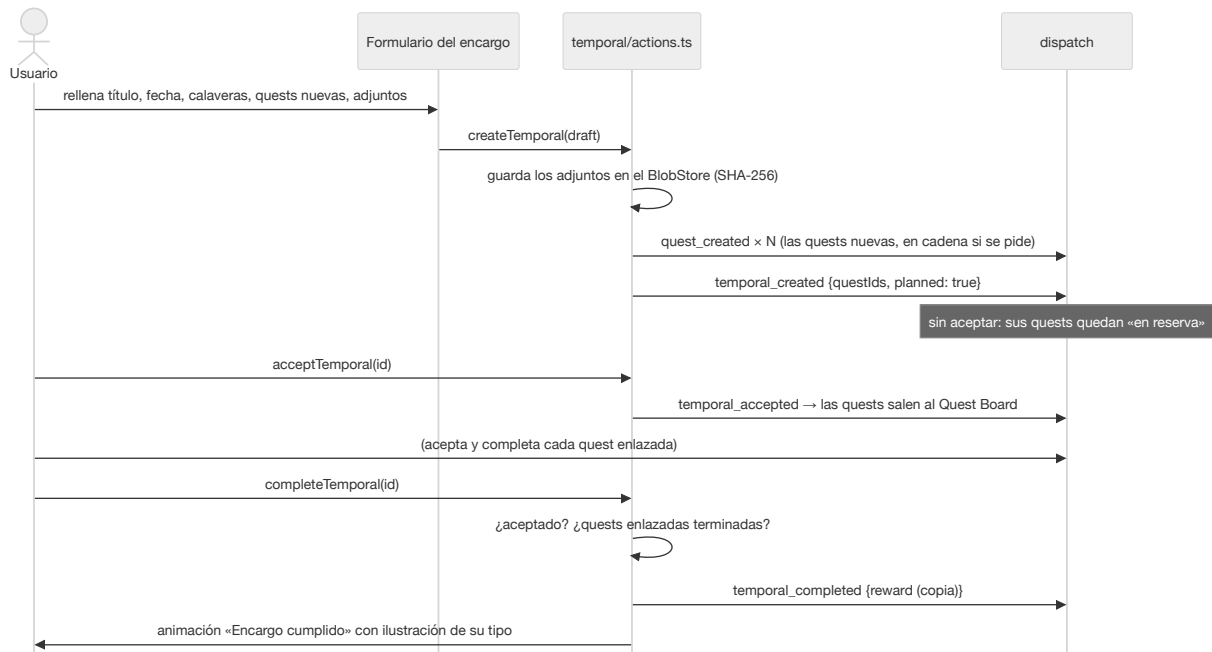
```

export async function primaryAction(id: string) {
  const q = useGame.getState().state.quests.get(id);
  if (!q) return;
  if (q.status === "active") await reportQuest(id);
  else await acceptQuest(id);
}
  
```

En el teléfono hay una variante, `detailPrimaryAction`: si el detalle está abierto a pantalla completa y vas a **aceptar**, primero cierra la hoja y espera a que salga, porque el sello «EN CURSO» cae sobre la **tarjeta** del tablón, que la hoja taparía.

12.6 Un flujo de encargo de principio a fin

Los encargos temporales (`features/temporal`) tienen su propio flujo, más largo:



12.7 Ofrecer «Deshacer»

```
const e = await dispatch({ type: "quest_abandoned", questId: id });
offerUndo(e, () => i18n.t("toast.abandoned", { title: q.title }));
```

`dispatch` devuelve el evento guardado precisamente para esto: `offerUndo` pinta el aviso con un botón que, al pulsarlo, emite `event_undone { eventId: e.id }`. `%Z` hace lo mismo con lo último de la sesión, durante 15 minutos.

Ejercicios

Ejercicio 12.1. Sigue en el código (con el depurador o con `console.log`) un `Enter` sobre una quest disponible, desde el `keydown` de `App.tsx` hasta el `append` del `EventStore`. Anota cada función por la que pasa.

Ejercicio 12.2. ¿Qué pasaría si `reportQuest` leyera `after` antes de `await dispatch(...)`? ¿Y si tirara los datos después?

Ejercicio 12.3. Escribe una acción `resetProgress(questId)` que deje a cero todos los contadores de una quest en curso usando solo eventos existentes. ¿Cuántos eventos emite? ¿Se podría deshacer de una vez?

Ejercicio 12.4. Lee `completeTemporal` y enumera qué comprueba la acción y qué comprueba la guarda de la proyección para el mismo evento.

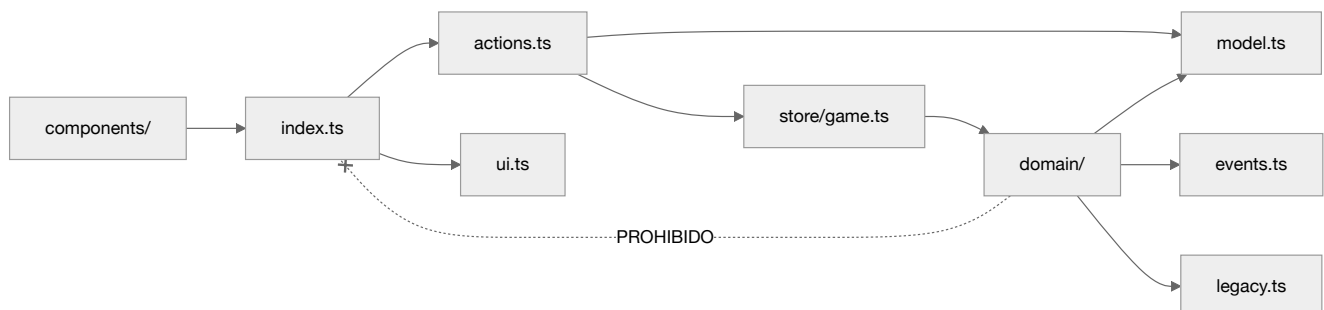
Capítulo 13. Todas las funcionalidades de `features`

13.1 Anatomía de una carpeta

Toda funcionalidad vive en `src/features/<nombre>/` con su `README.md`, que es la fuente única de sus reglas. Para tocar una funcionalidad basta con leer su `README`. La estructura completa (no todas tienen todos los archivos):

```
src/features/temporal/
├─ README.md      OBLIGATORIO: qué hace, reglas, modelo, eventos, interfaz, estado actual
├─ index.ts       API pública para la interfaz (lo que importan App y los componentes)
├─ model.ts       Tipos y funciones puras (sin React, Zustand, Tauri, Date.now ni
Math.random)
├─ events.ts      Sus eventos (se suman a EventBody)
├─ legacy.ts      Conversión de datos antiguos, si la hay
├─ actions.ts     Casos de uso: validan y llaman a dispatch
├─ ui.ts          Estado de interfaz propio (Zustand) y su ...Busy()
├─ i18n.ts        Textos es + ja (ja tipado como typeof es)
├─ temporal.css   Estilos, con el bloque @media (max-width: 760px) al final
└─ components/    Componentes React
```

Cuidado. El dominio (`src/domain/`) puede importar el `model.ts`, `events.ts` o `legacy.ts` de una funcionalidad, **nunca** su `index.ts`: el `index.ts` reexporta `actions.ts`, que importa el store, que importa el dominio... y se crearía un ciclo de importaciones.



13.2 Las 33 funcionalidades de un vistazo

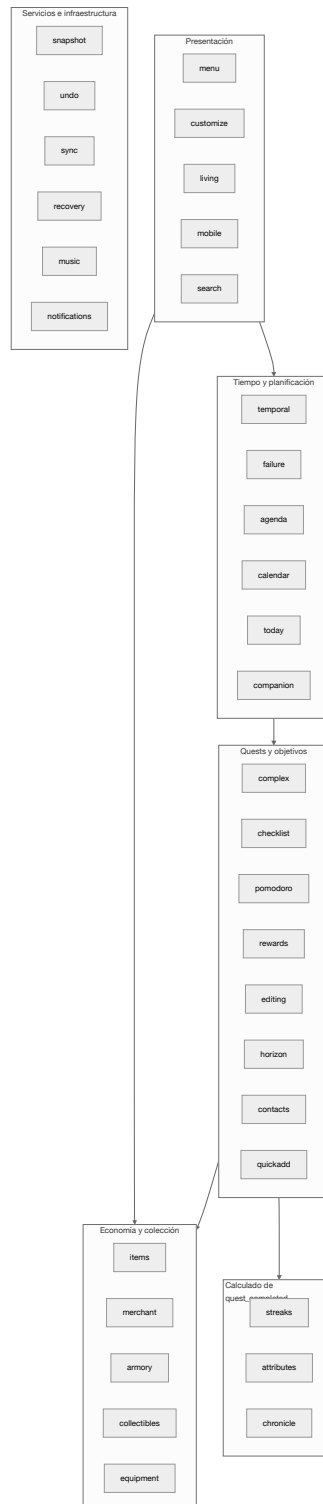
Cada `README` declara su **tipo** en el *frontmatter*:

- **dominio:** reglas del juego (modelo puro, a menudo eventos propios).
- **presentación:** pantallas y vistas que solo leen el estado y llaman a acciones.
- **servicio:** algo local de cada equipo, sin eventos (música, avisos).
- **infraestructura:** mecanismos de la app (snapshot, sincronización, recuperación).

Funcionali- dad	Tipo	En una frase	Eventos propios
agenda	dominio	Bloques por horas, sueltos o que se repiten; sin XP	<code>agenda_*</code> (4)

Funcionali- dad	Tipo	En una frase	Eventos propios
armory	dominio	69 piezas de serie para el mercader, con arte SVG generado	—
attributes	dominio	Un nivel por área, en un radar	—
calendar	presentación	Mi día · Semana · Día	—
checklist	dominio	Objetivo de tipo lista de casillas	checklist_checked
chronicle	dominio	Diario de lo hecho, apuntado por la proyección	—
collectibles	dominio	Un coleccionable mítico+ a la venta cada semana	collectible_purchased
companion	dominio	Un personaje comenta tu día en «Mi día»	companion_* (4)
complex	dominio	Repetición y requisitos entre quests	—
contacts	dominio	Teléfono, correo, enlace... con su botón	—
customize	presentación	Ventana para personalizar personajes e ilustraciones	—
editing	dominio	Editar una quest con un parche	quest_updated
equipment	dominio	El muñeco con el equipo puesto y la decoración	gear_equipped, gear_unequipped
failure	dominio	Quest fracturada o cartel quemado al vencer	quest_failed, temporal_failed
horizon	dominio	Plazos (1 día, 7 días...) y fecha límite	—
items	dominio	Objetos, rarezas, inventario, cofre con <i>pity</i> , almanaque	item_* (3)
living	presentación	Personajes del menú animados con WebGL	character_style_* (2)
menu	presentación	Menú de opciones al estilo gacha, personaje del día	character_*, voice_line_* (5)
merchant	dominio	Hu Tao vende equipo con precio calculado y escape-rate semanal	gear_* (4)
mobile	presentación	Interfaz de teléfono, gestos de iOS, vibración	—
music	servicio	Música de fondo con fundidos	—
notifications	servicio	Avisos del sistema	—
pomodoro	dominio	Objetivo de rondas de concentración y descanso	pomodoro_* (5)
quickadd	presentación	Alta rápida en una línea con marcas	—
recovery	infraestruc- tura	Error boundary con pantalla de recuperación	—
rewards	dominio	XP y oro calculados	—
search	presentación	Búsqueda sin tildes ni distinción de kana	—
snapshot	infraestruc- tura	Proyección incremental y snapshot	—
streaks	dominio	Rachas de las quests que se repiten	—

Funcionali- dad	Tipo	En una frase	Eventos propios
sync	infraestruc- tura	Sincronización con Google Drive	—
temporal	dominio	Encargos con fecha en su propio tablón	temporal_* (12)
today	presentación	«¿Qué hago ahora?»	—
undo	dominio	Deshacer unos minutos con otro evento	event_undone



A continuación, cada funcionalidad con lo esencial. Para el detalle completo, su README.

13.3 Quests y objetivos

complex · Quests complejas

- **Qué hace:** una quest puede **repetirse** (cada N horas o días, o ciertos días de la semana, en cualquier categoría) y tener **requisitos** (otras quests que hay que completar antes; sale con candado).
- **Datos:** campos opcionales de `QuestDef`: `cooldownMinutes`, `repeatDays`, `requires`. Sin eventos propios.
- **Reglas clave:** `recurs(q)`; `returnsAt(q, ts)` (cuándo vuelve); un requisito se cumple con una compleción o si se retira o falla; sin ciclos (`wouldCycle`); máximo 6 requisitos; «bloqueada» es calculado (`blockers`), no un estado.
- **Archivos:** `complex/model.ts` (`recurs`, `returnsAt`, `nextRepeatDay`, `prerequisitesMet`, `blockers`, `unlockedBetween`, `RECURRENCE_PRESETS`).

checklist · Objetivo de tipo lista

- **Qué hace:** un objetivo con casillas que se marcan una a una; se cumple con todas marcadas.
- **Evento:** `checklist_checked { questId, conditionId, itemId, done }`: un **valor** por casilla, no un +1 (si dos equipos marcan la misma, queda marcada una vez).
- **Reglas clave:** `target` = número de casillas (lo fija `cleanChecklist`: sin vacías ni repetidas, hasta 20 de 80 caracteres); las marcas se vacían al aceptar, abandonar o completar.

pomodoro · Rondas de concentración

- **Qué hace:** un tipo de objetivo con N rondas de concentración y descanso (la última sin descanso). Por defecto 90/15 min; de 1 a 12 rondas.
- **Eventos:** `pomodoro_started`, `_paused`, `_resumed`, `_stopped`, `_break_skipped`. Las fases se **calculan** sobre una línea de tiempo (capítulo 10).
- **Reglas clave:** solo corre un pomodoro a la vez en toda la app; parar pierde solo la ronda en curso; aceptar, abandonar o completar la quest los reinicia.
- **Archivos:** `pomodoro/model.ts` (`viewPomodoro`, `applyPomodoroEvent`, `clampPlan`), `legacy.ts` (el formato antiguo de un único pomodoro por quest).

rewards · Recompensa calculada

- **Qué hace:** la XP y el oro de una quest salen de sus objetivos y su categoría; los de un encargo, de sus calaveras y de lo que valen sus quests enlazadas.
- **Reglas clave:** la proyección sustituye la recompensa al crear (`questReward`) y recalcula la de los encargos pendientes (`temporalValue`); lo ganado va copiado en el evento. Tablas en el capítulo 10.
- **Encargo** = base por calaveras (60 XP · 1.350 G con 1 calavera ... 500 XP · 11.250 G con 5) + un bono del 20 % al 60 % sobre lo que valen sus quests.

editing · Editar quests

- **Qué hace:** editar una quest publicada con el mismo formulario (tecla R).
- **Evento:** `quest_updated { questId, patch }`, con **solo lo que cambia** (`diffQuest`): dos equipos que editan campos distintos no se pisan. En los opcionales, `null` quita el valor (JSON no transporta

undefined).

- **Guardas** (`patchQuest`): una terminada no se edita; **en curso** no cambian categoría, objetivos ni repetición (`STRUCTURAL_KEYS`), porque tienen progreso; un campo inválido no cambia, el resto sí; la recompensa se recalcula.

horizon · Plazos

- **Qué hace:** clasifica quests y encargos por cuánto falta para su fecha (1 día, 7 días, 2 semanas, 1 mes, +1 mes, sin fecha) con un filtro en cada tablón (tecla `H`); da la fecha límite opcional de las quests.
- **Reglas clave:** plazos excluyentes y sin huecos (los contadores suman el total); **días naturales** (`daysUntil`, robusto ante el cambio de hora); se calcula con `now`, nunca se guarda; lo vencido cuenta como «1 día».

contacts · Contactos

- **Qué hace:** teléfono, correo, WhatsApp, enlace o dirección en quests y encargos, con botón para llamar, escribir, abrir o ver cómo llegar, y otro para copiar.
- **Reglas clave:** escritos a mano (no se eligen de la agenda del sistema); validación suave (avisa pero deja guardar); `contactHref` solo produce `tel:`, `mailto:` y `https:`, lo mismo que permite el permiso de Tauri (`opener:allow-open-url`). Un test lo comprueba.

quickadd · Alta rápida

- **Qué hace:** una línea para apuntar una quest al vuelo: «Llamar al banco mañana #Hogar !».
- **Marcas** (`parseQuick`): hoy/mañana/pasado mañana, días de la semana, `12/10`, sus equivalentes japoneses (明日...), #área, @quién, ! (élite), x3 (objetivo x3).
- **Reglas clave:** publica un `quest_created` normal (categoría Encargo o Élite, un contador «Hacerlo xN», recompensa calculada); `Mayús+Enter` abre el formulario completo con lo escrito.

13.4 Lo que se calcula de `quest_completed`

streaks · Rachas

- **Qué hace:** las quests que se repiten cuentan las veces seguidas completadas **a tiempo**, con una llama en la tarjeta (brasa, fuego, fuego vivo, llama azul a partir de 30).
- **Reglas clave:** «a tiempo» = antes de que acabe el margen (lo que dura la espera, mínimo un día; o el final del siguiente día que toca); `nextStreak` en la proyección; **romperse no es un evento** (`liveStreak(s, now)` devuelve 0 si pasó el plazo); no da XP ni oro extra; avisos al llegar a 3, 7, 14, 30...

attributes · Atributos

- **Qué hace:** cada área es un atributo con su nivel, en un radar junto al muñeco.
- **Reglas clave:** la XP de cada `quest_completed` válido se suma a su área (`gainAttribute`), retroactivo; áreas normalizadas y 17 conocidas con sinónimos en tres idiomas; curva $60 \cdot n^{1,3}$; un Map y no un objeto (un área llamada «**proto**» rompería un objeto plano).

chronicle · Crónica del aventurero

- **Qué hace:** un diario (tecla `J`) con lo hecho cada día: quests, encargos, compras, fallos, subidas de nivel y de atributo, con estilo de libro gastado.
- **Reglas clave:** la proyección apunta una entrada (`record`) **solo cuando un evento pasa su guarda**, así que nunca cuenta algo que no pasó; cada entrada **copia** lo que puede cambiar (título, nombre del objeto...) y guarda solo `xpAfter`: las subidas de nivel se deducen leyendo en orden; la paginación **mide** cuántos renglones caben.

13.5 Economía y colección

items · Objetos

- **Qué hace:** objetos del almanaque (`ItemDef`) con seis rarezas, tipo fijo, inventario, **cofre** con botón al completar quests y un almanaque por tipo (tecla `I`).
- **Eventos:** `item_created`, `item_updated` (parche), `item_deleted` (id retirado).
- **Probabilidades (estilo Genshin):** estándar 50 % común ... 0,6 % legendario, 1 tirada; élite mejorada y 2 tiradas. **Pity:** desde la tirada 74 sin legendario sube un 6 % por tirada y la 90 lo garantiza; épico o superior garantizado cada 10.
- **Reglas clave:** el azar en la acción y el resultado en `quest_completed.drops`; cada drop copia su rareza; los **coleccionables** (salen en cofres) se tienen o no: un repetido «se quema» (pero cuenta para el pity); el icono va en el evento (160 px) y la versión nítida en el almacén de binarios.

merchant · Mercader (Hu Tao)

- **Qué hace:** vende equipo para el muñeco (8 ranuras) y decoración (fondo y emblema), con su vídeo en bucle y diálogo de JRPG (tecla `C`).
- **Eventos:** `gear_created`, `gear_updated`, `gear_deleted`, `gear_purchased { gearId, price }`.
- **Precios:** por rareza (1.200 G común ... 100.000 G legendaria) × recargo por ranura (×1 manos/pies ... ×1,6 fondo), con **rango mínimo** por rareza. El precio y el rango los calcula el juego; el jugador añade mercancía **sin precio**.
- **Escaparate semanal:** 5 piezas al azar (con la semana como semilla: `seededRandom("<semana>:<id>")`) más las recién llegadas; **se calcula, no se guarda:** todos los equipos ven el mismo sin sincronizar nada.
- **Reglas clave:** la proyección resta el oro **solo si llega en ese punto**; escaparate y rango se comprueban en la acción (`buyBlocker`), no en la guarda.

armory · Equipo de serie

- **Qué hace:** 69 piezas inspiradas en Mushoku Tensei, Re:Zero, Konosuba y JRPG clásicos, para todos, con **arte SVG generado** por código (nada de imágenes de las series).
- **Reglas clave:** viven en el **código** (`BUILTIN_GEAR`), no en eventos ni en el acumulador: las tiene cualquiera, también quien ya tenía datos; ids `armory-<clave>`; no se crean, editan ni retiran con eventos; **no** se borra una pieza ni se cambia su `key` (las compras guardadas la nombran).

collectibles · Coleccionable de la semana

- **Qué hace:** Hu Tao vende cada semana **un** coleccionable mítico o legendario que no tienes, al precio de una pieza de esa rareza +50 %, sin requisito de rango.

- **Evento:** `collectible_purchased { itemId, price }`.
- **Reglas clave:** la oferta se calcula (hash de la semana y el id, FNV-1a); si te sale en un cofre, aparece otra esa semana; la proyección vigila solo lo que no cambia al fusionar (que exista, rareza, que no lo tengas, oro).

equipment · Personaje

- **Qué hace:** un muñeco SVG con el equipo comprado puesto (tecla **P**), sus atributos al lado y la decoración del menú (fondo de toda la app y emblema de la cabecera).
- **Eventos:** `gear_equipped { gearId }`, `gear_unequipped { slot }`.
- **Reglas clave:** solo se pone lo tuyo; ponerte otra pieza quita la anterior; lo retirado se quita solo (`pruneEquipment`); el **prestigio** (suma de estrellas de lo que llevas) se calcula.

13.6 Tiempo y planificación

temporal · Encargos temporales

- **Qué hace:** cosas con fecha (citas, entregas, exámenes) en **su propio tablón** de roble con carteles de pergamino, de 1 a 5 **calaveras** según la dificultad, adjuntos (PDF, imágenes), quests enlazadas y animaciones de clavar y cumplir (tecla **T**).
- **Eventos (12):** crear, editar (parche), adjuntar/quitar adjunto, enlazar/desenlazar quest (deltas), **aceptar/aplazar**, cumplir (con recompensa copiada), retirar, añadir/quitar ilustración.
- **Reglas clave:** es su propia entidad, no una cuarta categoría de quest (ADR-12); se clava **sin aceptar** y sus quests esperan **en reserva** (fuera del Quest Board) hasta aceptarlo; solo se cumple con todas sus quests enlazadas terminadas; la urgencia (`overdue`, `today`, `soon`, `later`) se calcula con `now`; al cumplir sale una ilustración de su tipo elegida al azar (de `public/temporal/<tipo>/` o añadida por el jugador).

failure · Fallos

- **Qué hace:** al acabar el día de su fecha sin terminarla, la quest **se fractura** (24 pedazos, «QUEST FAILED») y el cartel del encargo **se quema** («BURNED»). Sin coste: queda en la crónica y se puede **volver a clavar** una copia.
- **Eventos:** `quest_failed`, `temporal_failed`, emitidos por `FailureWatcher` al arrancar y cada minuto.
- **Reglas clave:** es evento y no cálculo porque saca la quest del tablón, se apunta en la crónica y bloquea su encargo: tiene que ser igual en todos los equipos (ADR-44); si dos equipos lo emiten, la guarda deja contar el primero; lo vencido antes del 6 de octubre de 2026 se perdona (en la acción, no en la guarda); las quests de un encargo quemado fallan con él, salvo las que se repiten.

agenda · Agenda

- **Qué hace:** bloques con hora de inicio y fin («Gimnasio, 19:00–20:30»), de un día o repetidos ciertos días de la semana. **No da XP ni oro:** es para organizarse.
- **Eventos:** `agenda_created`, `agenda_updated` (parche), `agenda_skipped` (quitar solo un día), `agenda_deleted`.
- **Reglas clave:** días como texto `AAAA-MM-DD` en hora local y horas como **minutos desde medianoche** (un bloque de las 9:00 sigue a las 9:00 aunque cambie la hora); datos tolerantes (`normalizeAgenda`); cinco colores con nombre de token.

calendar · Calendario

- **Qué hace:** la tercera sección (tecla `S`), donde se planifica: **Mi día**, **Semana** (siete columnas) y **Día** (rejilla por horas).
- **Reglas clave:** sin eventos propios: se calcula de quests (por su fecha límite), encargos (por su fecha) y agenda (`calendarDay`); los bloques que se solapan se reparten en columnas (`layoutDay`); desde el «+» de cada día se añade un bloque, una quest con esa fecha o un encargo.

today · Mi día

- **Qué hace:** responde a «¿qué hago ahora?»: lo que se pierde esta noche, lo que está en curso, rachas en peligro, lo que toca hoy, lo vencido de antes, la agenda de hoy, los próximos días y lo hecho hoy.
- **Reglas clave:** `todayPlan` lo calcula todo con `now`; una quest sale en un solo bloque; sin nada pendiente, «Día libre».

companion · Compañero de Mi día

- **Qué hace:** un personaje del menú comenta tu día en un cuadro de diálogo de JRPG que se escribe letra a letra.
- **Eventos:** `companion_chosen`, `companion_line_added`, `_updated`, `_removed`.
- **Reglas clave:** siete **situaciones** de más a menos urgente (se pierde esta noche, racha en peligro, en curso, toca hoy, perdido, cumplido, día libre) sacadas del mismo plan que «Mi día»; las frases del jugador sustituyen a las de serie; huecos `{{title}}`, `{{n}}`, `{{time}}`.

13.7 Presentación

menu · Menú de opciones

- **Qué hace:** pantalla al estilo del menú principal de un gacha (tecla `0`): el personaje del día, tarjetas en 3D de cada sección (Quests, Character, Bounties, Merchant, Collection, Chronicle, Calendar, Search, Customize) y los ajustes (idioma, música, silencio, avisos, Google Drive).
- **Eventos:** `character_added`, `character_removed`, `voice_line_added/_updated/_removed`.
- **Reglas clave:** el personaje **rota cada día** sin repetir hasta que salgan todos (`characterOfDay`: vueltas barajadas con su número como semilla, el mismo orden en todos los equipos); elegir otro «para hoy» es una preferencia local; los de serie son los `.webp` de `public/menu/` (módulo virtual de Vite); las ventanas se abren **encima** del menú (capa 50 sobre 47).

customize · Personalización

- **Qué hace:** una sola ventana para personalizar: personajes (imagen, GIF o vídeo), lo que dicen según la hora, cómo se mueven, lo que dicen en «Mi día» e ilustraciones de los encargos.
- **Reglas clave:** **solo presenta:** cada dato lo guarda su funcionalidad dueña (menu, living, companion, temporal) con sus eventos y guardas; lo de serie no se quita; lo añadido se quita con dos toques y no se deshace (la imagen se borra).

living · Personajes vivos

- **Qué hace:** los personajes del menú respiran, se mecen y les da el viento con una **mallá de WebGL** sobre su propia imagen, con aura, brillo, partículas y una entrada gacha opcional.

- **Eventos:** `character_style_set { characterId, style }` (parche sobre `DEFAULT_STYLE`), `character_style_reset`.
- **Reglas clave:** la malla no inventa píxeles, solo desplaza zonas (por altura y distancia al eje); un vídeo o GIF ya se mueve solo y va sin malla; sin WebGL o con «reducir movimiento», imagen quieta. La malla es un **recurso imperativo** (`createStage / destroy`).

mobile · Interfaz de teléfono

- **Qué hace:** adapta todas las pantallas a ~400 pt táctiles: barra de abajo de cristal con selector arrastrable, detalle como hoja a pantalla completa, gestos de iOS y vibración. Capítulo 16.

search · Búsqueda

- **Qué hace:** ventana (`/` o `⌘K`) que busca en quests (también terminadas), encargos, agenda y objetos, y lleva a lo encontrado.
- **Reglas clave:** `fold` normaliza (minúsculas, sin tildes, ancho normal, katakana = hiragana); cada palabra tiene que estar en algún campo; puntúa más el título; máximo 40 resultados.

13.8 Servicios e infraestructura

music · Música de fondo

- **Qué hace:** música en bucle con fundidos y volumen (tecla `M`), preferencia de cada equipo.
- **Reglas clave:** fuera del dominio y de los eventos; **aquí sí una clase**, `MediaPlayer`, con instancia única en `globalThis` (sobrevive a la recarga en caliente) y suscripción con `useSyncExternalStore`; volumen por `GainNode` de Web Audio; la pista se descarga entera y se sirve como `blob`: (el protocolo de Tauri no siempre atiende peticiones por rangos).

notifications · Avisos del sistema

- **Qué hace:** avisos fuera de la app para pomodoros, encargos, agenda, fechas límite y rachas.
- **Reglas clave:** plan calculado de los **próximos dos días** (`planReminders`); en **iOS** se programan (hasta 60; el sistema avisa con la app cerrada); en el **escritorio** un temporizador espera al siguiente (con la app cerrada no avisa); id numérico estable derivado de una clave de texto.

undo · Deshacer

- **Qué hace:** deshacer unos minutos (`⌘Z` o el botón del aviso).
- **Evento:** `event_undone { eventId }`. Capítulo 6.
- **Reglas clave:** ventana de 15 min; solo tipos de `UNDOABLE`; deshacer recalcula todo (cambia el pasado).

snapshot · Snapshot de la proyección

- **Qué hace:** `dispatch` incremental con copia selectiva y arranque desde un snapshot cada 100 eventos. Capítulo 9.

sync · Sincronización

- **Qué hace:** cada equipo sube su archivo de eventos a Google Drive y baja los de los demás. Capítulo 17.

recovery · Recuperación ante fallos

- **Qué hace:** error boundary en la raíz con pantalla de recuperación («CONTINUE?») en vez de la ventana en negro.
- **Reglas clave:** atrapa errores al **pintar**, no los de manejadores ni código asíncrono; «volver a intentarlo» monta la app con una **key** nueva; la pantalla no usa el store (el fallo podría venir de él).

13.9 Cómo leer un README de funcionalidad

Todos siguen la plantilla `docs/templates/feature-readme.md`. El orden de las secciones es siempre el mismo y conviene leerlas así:

Sección	Te dice	Úsala cuando...
<i>Frontmatter</i>	tipo, eventos, preferencias, ADR	quieres saber de un vistazo qué toca
Qué hace	requisitos del propietario y cómo se cumplen	vas a cambiar comportamiento
Reglas y decisiones	el porqué y lo descartado	dudas si algo es un error o una decisión
Modelo / Eventos	tipos, eventos y guardas	tocas la proyección
Interfaz	teclas, animaciones, teléfono	tocas componentes
Archivos / Dónde está cada cosa	qué símbolo tocar	buscas código
Integración	qué toca fuera de su carpeta	revisas el impacto
Dependencias	qué otras funcionalidades usa y para qué	cambias algo compartido
Estado actual	última verificación, tests y lo no probado	vas a dar algo por terminado

Ejercicios

Ejercicio 13.1. Elige tres funcionalidades de tipo «dominio» y, para cada una, localiza en `src/domain/projection.ts` dónde se integra (un `case`, una llamada a su `apply*Event` o un cálculo dentro de otro `case`).

Ejercicio 13.2. ¿Por qué `music` puede ser una clase con estado y `merchant` no? Responde con la norma de `docs/AGENTES.md §5.4`.

Ejercicio 13.3. Las rachas, los atributos y la crónica no tienen eventos. Explica qué ganarían y qué perderían si los tuvieran (`streak_increased`, `attribute_gained`...). Piensa en la retroactividad y en la sincronización.

Ejercicio 13.4. Abre `docs/INDEX.md` («Dependencias entre funcionalidades»). ¿Qué funcionalidad importa más funcionalidades? ¿Tiene sentido por su tipo?

Parte VI · Interfaz, animaciones e i18n

Cómo habla la app en dos idiomas, cómo suena sin archivos de audio, cómo se anima sin pelearse con React y cómo se organizan los estilos.

Capítulo 14. i18n, sonidos, animaciones y CSS

14.1 Internacionalización (i18n)

i18n (*internationalization*: una «i», 18 letras y una «n») es preparar la app para varios idiomas. Quests habla **español** y **japonés** con **i18next** y **react-i18next** ([src/i18n/](#)).

Diccionarios con tipos

El diccionario de referencia es `src/i18n/locales/es.ts`; el japonés se declara **con su tipo**:

```
// src/features/checklist/i18n.ts
export const checklistEs = {
  kind: "Lista",
  add: "+ Añadir lista",
  itemPh: "Casilla {{n}}",          // {{n}}: interpolación
  count: "{{done}} / {{total}}",
};
export const checklistJa: typeof checklistEs = { // ← si falta o sobra una clave: error de compilación
  kind: "チェックリスト",
  add: "+ チェックリストを追加",
  itemPh: "項目 {{n}}",
  count: "{{done}} / {{total}}",
};

// src/i18n/locales/es.ts monta cada funcionalidad bajo su nombre
export const es = {
  // ...
  checklist: checklistEs,
  sync: syncEs,
};
```

Además, `src/i18n/i18next.d.ts` registra las claves en los tipos de `i18next`: `t("detail.reward")` tiene auto-completado y `t("detail.rewrad")` **no compila**.

Idea clave. Gracias a los tipos, olvidar una traducción no es un fallo que descubra el usuario: es un error de `tsc` que para la CI.

Cómo se usa

Dónde	Cómo
En un componente	<code>const { t } = useTranslation();</code> y <code>t("actions.accept")</code> . El hook repinta al cambiar de idioma
Fuera de React	<code>i18n.t("toast.accepted", { title })</code>
En un aviso	<code>say(() => i18n.t(...))</code> : una función , para que el aviso cambie si cambias de idioma con él en pantalla
Frases con etiquetas	<code><Trans i18nKey="detail.emptyHint" components={{ kbd: <kbd /> }} /></code>

Dónde	Cómo
Plurales	<code>hours_one</code> / <code>hours_other</code> , según <code>count</code> (en japonés, iguales)
Números	<code>num(n)</code> : 1.150 en español, 1,150 en japonés

Idioma inicial

```
function initialLang(): Lang {
  const saved = localStorage.getItem("quests.lang");
  if (saved === "es" || saved === "ja") return saved; // preferencia de este equipo
  return navigator.language.toLowerCase().startsWith("ja") ? "ja" : "es"; // si no, el del
  sistema
}
```

El idioma es una **preferencia de cada equipo** (no se sincroniza). Al cambiarlo, `<html lang>` se actualiza y los ajustes de espaciado para japonés se aplican con selectores `:lang(ja)`.

Qué se traduce y qué no

Se traduce	No se traduce
Toda la interfaz, avisos, tiempos («3 h 20 min» / 「3時間20分」), números	Lo que escribe el usuario (títulos, descripciones, objetivos)
Las quests y objetos de ejemplo (en el idioma activo la primera vez)	Las etiquetas decorativas en inglés: ELITE, REQUEST, QUEST CLEAR, LEVEL UP!, los rótulos del menú
Las áreas conocidas («Salud» ↔ 「健康」) y las piezas de serie	Las áreas y piezas que añade el usuario

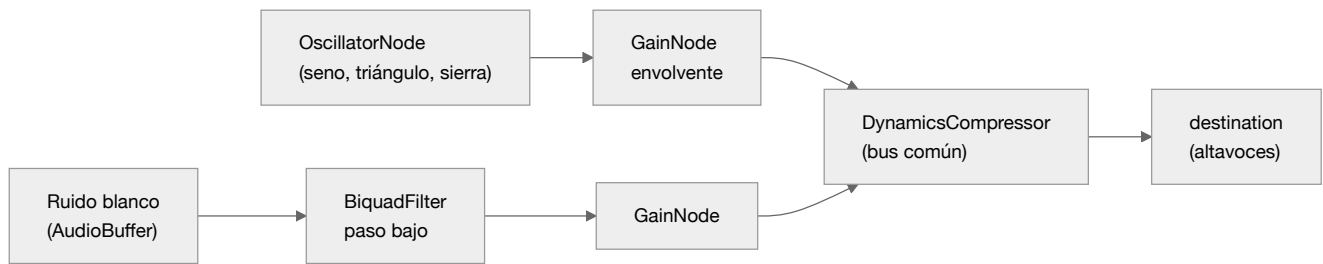
El japonés usa el vocabulario de los JRPG ya fijado en el proyecto (受注する para aceptar, 取りやめる para abandonar, 報酬 para recompensa...).

Añadir un idioma

- `src/i18n/locales/en.ts` con `export const en: Translation = { ... }` (y la parte de cada funcionalidad en su `i18n.ts`).
- Registrarlo en `LANGS`, `locales` y `resources` de `src/i18n/index.ts`.
- El selector lo muestra solo, porque recorre `LANGS`.

14.2 Sonido sintetizado: Web Audio

Quests **no tiene archivos de efectos de sonido**: cada sonido se **fabrica** con la **Web Audio API** (`src/lib/sfx.ts`, ADR-07). La Web Audio API es un grafo de nodos: fuentes (osciladores, ruido) → procesadores (ganancia, filtros, compresor) → salida.



Las dos piezas básicas:

```

/** Un tono con envolvente: sube en 10 ms y cae exponencialmente (suena a «golpe», no a pitido).
 */
function tone(freq: number, start: number, dur: number, type: OscillatorType, gain: number,
  attack = 0.01) {
  const a = ac();
  const o = a.createOscillator();
  const g = a.createGain();
  o.type = type;
  o.frequency.value = freq;
  g.gain.setValueAtTime(0, a.currentTime + start);
  g.gain.linearRampToValueAtTime(gain, a.currentTime + start + attack);
  g.gain.exponentialRampToValueAtTime(0.0001, a.currentTime + start + dur);
  o.connect(g).connect(out()); // out(): el compresor común
  o.start(a.currentTime + start);
  o.stop(a.currentTime + start + dur + 0.05);
}
// noise(duración, volumen, corte): ruido blanco con filtro paso bajo: la base de impactos y
cristal.

```

Efecto	Receta
Sello «EN CURSO»	Ruido grave (corte 1.800 Hz) + seno a 85 Hz (el «bombo») + triángulo a 170 Hz
Progreso	Dos notas agudas cortas (880 y 1.320 Hz)
Cristal roto	Ruido brillante (6.000 Hz) + cuatro tintineos escalonados
Quest Clear	Arpegio de do mayor (do, mi, sol, do)
Level Up	Arpegio de 6 notas
Cartel quemado	Soplo que crece, 18 chasquidos al azar y un acorde grave

Tres reglas:

- **Silencio general** (`isMuted()`, `quests.muted`): lo respeta todo, también la música.
- **Sin interacción previa no suena**: el WebView bloquea el audio hasta el primer clic o tecla; lo que suene al arrancar comprueba `navigator.userActivation.hasBeenActive`.
- **Cada efecto vibra** en el teléfono (`src/lib/haptics.ts`, con su propia preferencia `quests.haptics`): selección al moverse, golpe ligero/medio/fuerte, resultado (éxito, aviso, error). Un efecto nuevo lleva su vibración.

14.3 Animaciones: Motion y GSAP

Quests usa **dos** bibliotecas, cada una para lo suyo (ADR-06):

Biblioteca	Para qué	Ejemplos
Motion (motion/react)	Animaciones ligadas a React: aparecer, desaparecer, cambiar de posición (<i>layout</i>)	Entrada de tarjetas, la pestaña que se desliza (<code>layoutId</code>), ventanas, barras con muelle
GSAP	Secuencias largas y encadenadas con tiempos precisos	El sello «EN CURSO», «Quest Clear», «Level Up!», el cofre, los carteles

Regla práctica: si es «este elemento aparece o se mueve», Motion; si es «primero esto, luego aquello, y a la vez lo otro», GSAP

Motion en dos ejemplos

```
// La pestaña resaltada se desliza de una a otra: mismo layoutId en distintos sitios.
{tab === t.id && <motion.span layoutId="tab-hl" className="tab-hl" transition={{ type: "spring",
stiffness: 500, damping: 38 }} />}

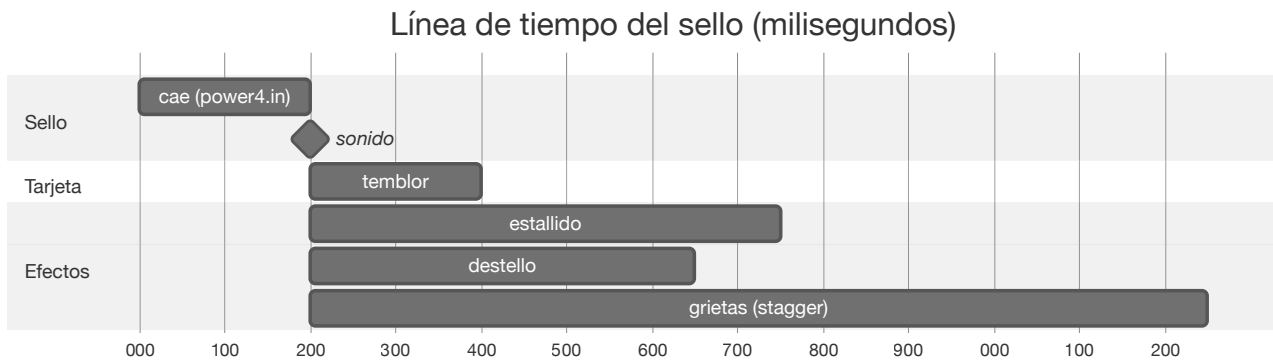
// Un aviso que entra y sale: AnimatePresence deja que el elemento anime su salida antes de
desmontarse.
<AnimatePresence mode="wait">
  {toast && (
    <motion.span key={toast.key} initial={{ opacity: 0, x: -8 }} animate={{ opacity: 1, x: 0 }}
    exit={{ opacity: 0 }}>
      {toast.text()}
    </motion.span>
  )}
</AnimatePresence>
```

Cuidado. Todas las ventanas salen con `MODAL_EXIT` y su fondo con `BACKDROP_EXIT` (`src/lib/motion.ts`), que pone `pointerEvents: "none"` en cuanto empieza a irse. Sin eso, el fondo ya invisible se quedaba encima hasta 1,4 s mientras acababan las animaciones de dentro y **se tragaba el clic siguiente**.

GSAP: líneas de tiempo

Una **línea de tiempo** (*timeline*) encadena animaciones. Cada paso empieza cuando acaba el anterior, salvo con la posición `"<"` («a la vez que el anterior»). El sello al aceptar (`QuestCard.tsx`):

```
const tl = gsap.timeline();
tl.set(cracks, { opacity: 1 })
  .fromTo(stamp, { scale: 2.8, opacity: 0, rotate: -30 },
    { scale: 1, opacity: 1, rotate: -9, duration: 0.2, ease: "power4.in" }) //
cae acelerando
  .add(() => sfx.stamp()) // sonido en el impacto
  .to(card, { keyframes: [{ x: -5, y: 3 }, { x: 4, y: -2 }, { x: -2, y: 1 }, { x: 0, y: 0 }] })
// temblor
  .fromTo(burst, { scale: 0.5, opacity: 1 }, { scale: 1.6, opacity: 0 }, "<") // a la vez
  .fromTo(flash, { opacity: 0.55 }, { opacity: 0 }, "<")
  .fromTo(crackPaths, { strokeDashoffset: 1 }, { strokeDashoffset: 0, stagger: 0.05 }, "<");
```

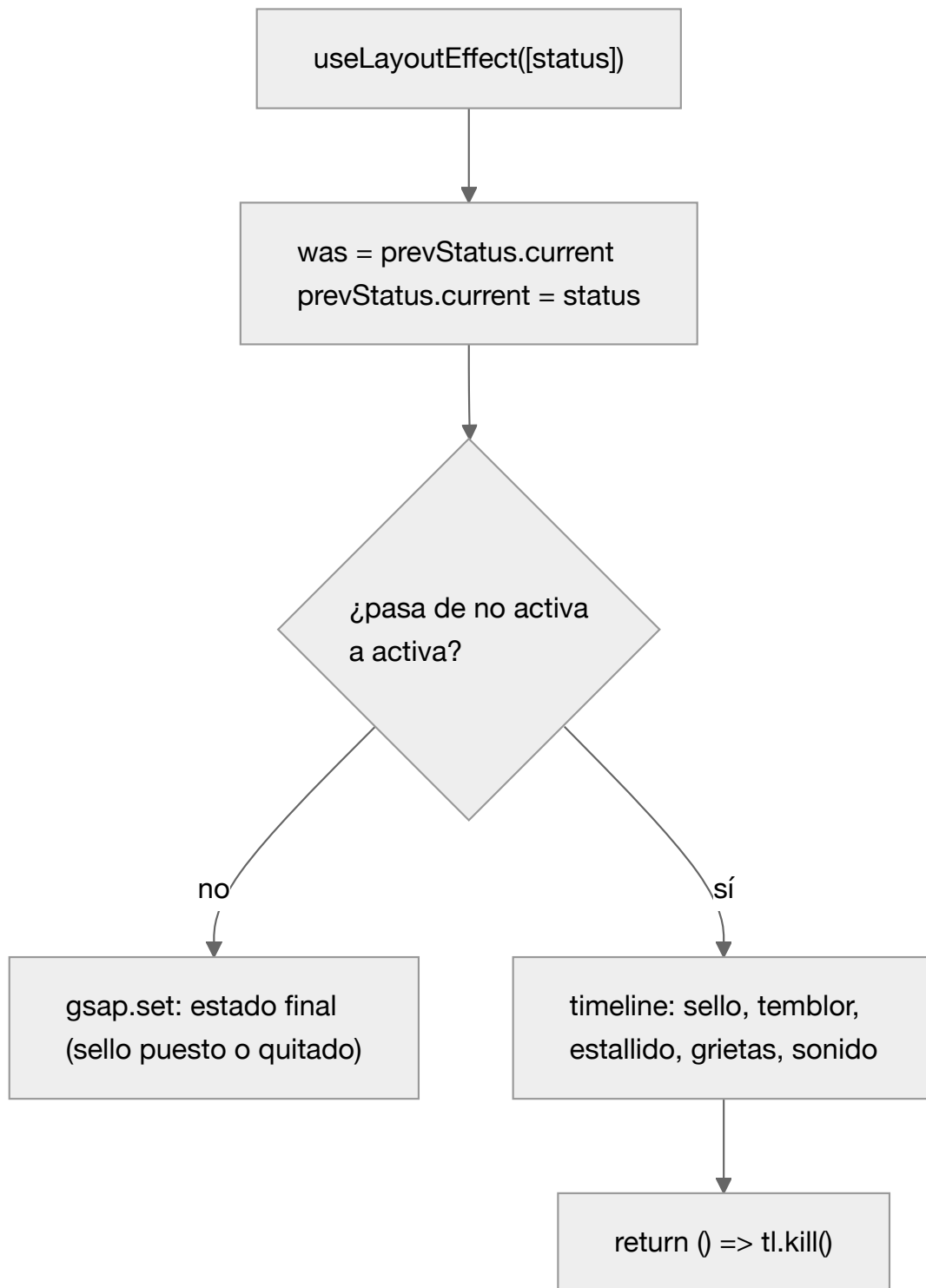


El **easing** es la curva de aceleración: `power4.in` hace que el sello acelere al caer, como un golpe real; `back.out` se pasa un poco y vuelve (rebote). `fromTo` fija inicio y final, así la animación es igual aunque se repita.

El patrón obligatorio para animar con GSAP dentro de React

React pinta y GSAP anima los mismos nodos: si no se ponen de acuerdo, hay parpadeos y estados fantasma. El patrón del proyecto (de `QuestCard`):

1. **Estado anterior en un `useRef`**, para saber si hay transición.
2. **Línea de tiempo solo en la transición que interesa** (de no activa a activa).
3. `gsap.set` **al estado final en los demás casos** (al abrir la app con quests ya aceptadas, el sello sale **ya puesto**, sin animar).
4. **Limpieza con `tl.kill()`**, nunca con `tl.progress(1)` (que volvería a aplicar estilos después de React y dejaba el sello puesto en una quest en espera).
5. **Si GSAP controla la visibilidad de un elemento, React no la controla también.**



Técnicas de las celebraciones

- **Grietas aleatorias pero siempre iguales:** un PRNG con semilla (`seededRandom`, `mulberry32`, en `src/lib/id.ts`) con el id de la quest como semilla. Para que las líneas «se dibujen»: `pathLength={1}`, `strokeDasharray="1 2"` y animar `strokeDashoffset` de 1 a 0.
- **La tarjeta que se rompe** (`ClearOverlay`): una rejilla de 4×3 puntos con los interiores desplazados, cada celda partida en 2 triángulos = 24 pedazos. Cada pedazo es una **copia completa** de la tarjeta recorrida con `clip-path: polygon(...)`; GSAP los aleja del centro con gravedad y giro.
- **Contadores:** GSAP anima un objeto `{ xp: 0, gold: 0 }` y `onUpdate` escribe el número redondeado.

- **Partículas fuera de React** (`src/lib/fx.ts`): `<span>` efímeros que GSAP crea, anima con el plugin `Physics2D` (velocidad, ángulo, gravedad) y borra. React no reconcilia cientos de nodos.
- **Sacudidas en el contenido** (`.cl-stage`), nunca en una capa `position: fixed` (un `transform` en un antecesor rompe el `fixed`).
- **Reducir movimiento**: `calm()` consulta `prefers-reduced-motion`; sin sacudidas y con un tercio de partículas. (El sello, «Quest Clear» y «Level Up!» aún no tienen versión reducida: está en la deuda técnica.)

14.4 CSS: tokens, fuentes y teléfono

Tokens de color

Todos los colores son **variables CSS** en `src/styles/theme.css`; **nunca** se escribe un color suelto:

```
:root {
  --bg: #0a0908;   --panel: rgba(30, 27, 23, 0.72);
  --text: #ece5d6; --muted: #7d7667;
  --gold: #d6b25e; --gold-hi: #f2d68f; --stamp: #79d8d2;
  --elite: #e2604c; --repeat: #a495ee; --request: #63c98e;
  --r-common: #9d988d; --r-uncommon: #5dbb72; --r-rare: #4f94e8;
  --r-epic: #a970f0; --r-mythic: #e5483f; --r-legendary: #f2b53c;
}
```

Un componente recibe un token por variable y sus estilos lo usan sin saber cuál es: cada tarjeta recibe `--cat` con el color de su categoría; cada objeto, `--rc` con el de su rareza.

Cuidado. `gsap.set(el, { "--rc": "var(--r-epic)" })` no aplica un valor `var(...)`; para variables CSS desde JavaScript se usa `el.style.setProperty("--rc", ...)`.

Fuentes locales

Cuatro familias con **Fontsource** (archivos locales, funciona sin conexión, ADR-08): Cormorant Garamond (títulos), Cinzel (versalitas), Shippori Mincho (texto) e IM Fell English (la crónica). **Solo el subconjunto latino**: con los subconjuntos japoneses las fuentes sumaban 26 MB de un build de 27. El japonés lo dibuja el mincho del sistema.

Un detalle tipográfico real: Cormorant trae cifras «elzevirianas» (el 0 parece una «o»); por eso `body` lleva `font-variant-numeric: lining-nums` y las versalitas se piden con `font-variant-caps` (el atajo `font-variant` reiniciaría las cifras).

Que el texto quepa

- El «...» (`text-overflow: ellipsis`) va en el **elemento de texto**, nunca en una caja `flex` o `grid`.
- Columnas que reparten espacio: `minmax(0, 1fr)`, y `min-width: 0` en lo que encoge.
- Se comprueba a 1.024 px (ventana mínima), a 402 × 874 (iPhone) y **en japonés**.
- **Nada de saltos de layout**: lo que se despliega va en capas absolutas.

CSS del teléfono

Cada funcionalidad pone sus ajustes de teléfono en un bloque `@media (max-width: 760px)` al final de su CSS. El JavaScript solo distingue el teléfono cuando el CSS no basta (`isPhone()`, `useIsPhone()`). Detalle en el capítulo 16.

Ejercicios

Ejercicio 14.1. Añade una clave nueva a `checklistEs` sin añadirla a `checklistJa` y ejecuta `npx tsc --noEmit`. Lee el error. Deshaz el cambio.

Ejercicio 14.2. Diseña con `tone` y `noise` un sonido «moneda» (dos notas agudas muy cortas y un brillo de ruido). Escríbelo como una función más de `sfx` que respete `isMuted()` y vibre con un golpe ligero.

Ejercicio 14.3. En `QuestCard`, cambia `tl.kill()` por `tl.progress(1)` en la limpieza (en una rama de pruebas), acepta una `quest` y repórtala enseguida si es repetible. ¿Qué ves en la tarjeta en espera? Explica por qué.

Ejercicio 14.4. Busca un color escrito a mano (`#...` o `rgb(...)`) en un CSS de `src/features/`. Si lo encuentras, ¿qué token debería usar? Si no, explica qué comprobarías antes de añadir un token nuevo.

Parte VII · Tauri, Rust e iOS

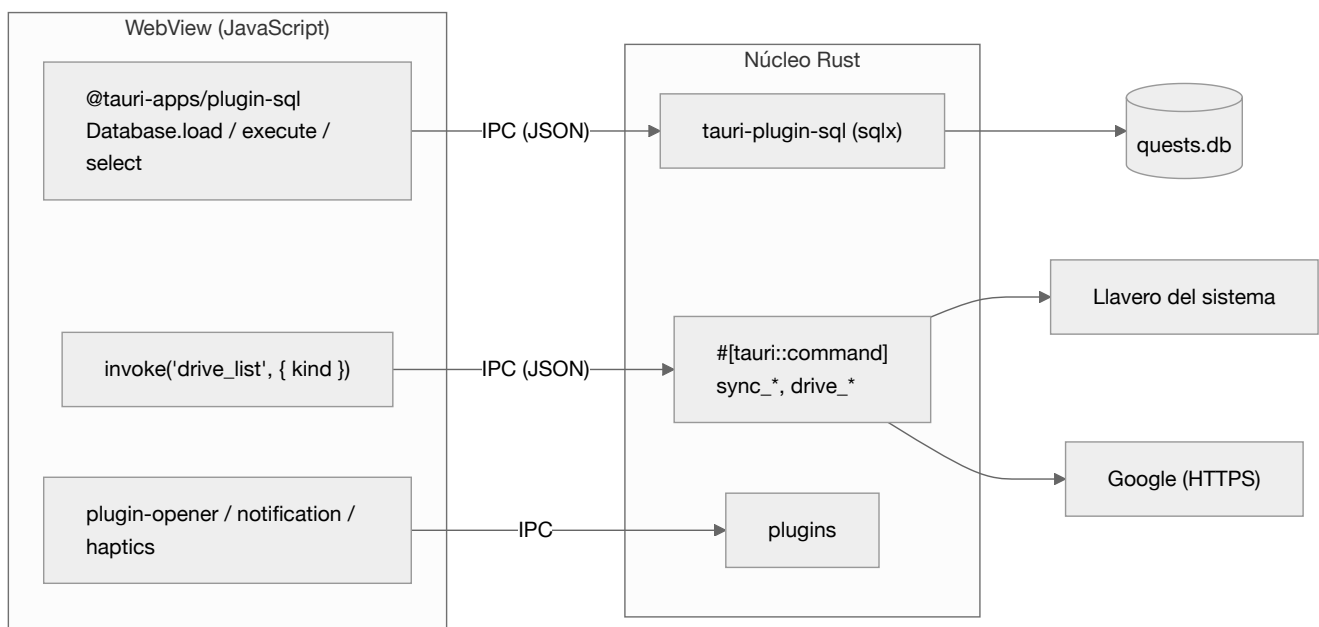
Los dos procesos de una app Tauri, el puente entre ellos, los permisos y la CSP, SQLite nativo y la misma app compilada para el iPhone.

Capítulo 15. Tauri, Rust, permisos y SQLite nativo

15.1 Dos procesos y un puente

Una app Tauri son **dos programas que se hablan**:

Proceso	Qué es	Qué hace en Quests
Núcleo nativo (Rust)	Un ejecutable compilado	Abre la ventana, carga plugins, accede a disco y al llavero, hace las llamadas a Google
WebView	El motor web del sistema (WKWebView en macOS/iOS, WebView2 en Windows)	Ejecuta React, las animaciones y toda la lógica del juego



IPC (*Inter-Process Communication*) es el mecanismo por el que el WebView pide cosas al núcleo. En la práctica, una llamada de JavaScript se **serializa a JSON**, viaja a Rust, se ejecuta y la respuesta vuelve serializada:

```
const db = await Database.load("sqlite:quests.db"); // abre (o crea) la base
await db.execute("INSERT OR IGNORE INTO events ...", [id, ...]); // esto NO toca SQLite desde JS:
                                                                    // viaja por IPC a tauri-plugin-
sql
```

Para recién graduados. Es una RPC local: el WebView es el cliente y el núcleo el servidor. Igual que en cualquier RPC, los argumentos y resultados tienen que ser serializables, y cada llamada cuesta (por eso los eventos se insertan por lotes y los binarios cruzan en base64).

15.2 Rust lo justo para leer el proyecto

No necesitas dominar Rust para trabajar en Quests (casi todo está en TypeScript), pero sí leer `src-tauri/src/`. Lo mínimo:

Concepto	Qué significa	Ejemplo en Quests
<code>struct</code> , <code>enum</code>	Datos; un <code>enum</code> puede llevar datos en cada variante (como una unión discriminada)	<code>enum SyncError { NotConfigured, Network(String), Drive(u16, String), ... }</code>
<code>Result<T, E></code>	Éxito o error, sin excepciones	<code>type R<T> = Result<T, SyncError></code>
<code>?</code>	«Si es error, devuélvelo ya»	<code>let c = client().ok_or(SyncError::NotConfigured)?;</code>
<code>Option<T></code>	Un valor o nada (Some/None)	<code>refresh_token: Option<String></code>
<code>async/.await</code>	Asincronía (con el <i>runtime</i> <code>tokio</code>)	<code>http.post(TOKEN_URL).form(&form).send().await</code>
<code>#[derive(Serialize, Deserialize)]</code>	<code>serde</code> genera la conversión a/desde JSON	<code>DriveFile</code> , <code>UploadRequest</code>
<code>#[serde(rename_all = "camelCase")]</code>	<code>file_id</code> en Rust ↔ <code>fileId</code> en JSON	Toda la API de Drive
<code>Mutex</code> (de <code>tokio</code>)	Exclusión mutua asíncrona	<code>token: Mutex<Option<AccessToken>></code>
<code>#[cfg(target_os = "ios")]</code>	Compilación condicional por plataforma	Dos versiones de <code>authorize</code>
Propiedad y préstamo (<code>&</code> , <code>&mut</code>)	Quién es dueño de cada dato; el compilador evita carreras de datos	<code>fn sign_in(app: &AppHandle, ...)</code>

15.3 Comandos propios

Un **comando** es una función Rust que JavaScript puede invocar. Se marca con `#[tauri::command]` y se registra en `invoke_handler`:

```
#[tauri::command]
pub async fn drive_list(state: State<'_, SyncState>, kind: String) -> R<Vec<DriveFile>> {
    logged("list", drive::list(&state, &kind).await)
}
```

Desde JavaScript (`src/features/sync/drive.ts`):

```
import { invoke } from "@tauri-apps/api/core";
export const tauriDrive: DriveApi = {
  list: (kind) => invoke<RemoteFile[]>("drive_list", { kind }),
  download: async (fileId) => new Uint8Array(await invoke<ArrayBuffer>("drive_download", {
    fileId })),
  upload: (request) => invoke<RemoteFile>("drive_upload", { request }),
};
```

Detalles que conviene saber:

- **Argumentos en camelCase:** `fileId` en JavaScript llega a Rust como `file_id`.
- **Estado compartido:** `.manage(sync::SyncState::default())` registra un objeto que los comandos reciben como `State<'_, SyncState>` (el cliente HTTP, el *access token* en memoria y la carpeta de Drive).

- **Errores:** un `Err(SyncError)` llega a JavaScript como el objeto que serializa. `SyncError` implementa `Serialize` a mano para producir `{ code, status, detail }` con un `code` **estable** que la interfaz traduce a un aviso.
- **Binarios sin JSON:** `drive_download` devuelve un `tauri::ipc::Response`, que llega como `ArrayBuffer` sin pasar por JSON.

15.4 Permisos: capabilities

Tauri 2 **deniega todo por defecto**. El `WebView` solo puede usar lo que declara `src-tauri/capabilities/default.json`:

```
{
  "identifier": "default",
  "windows": ["main"],
  "permissions": [
    "core:default",
    "sql:default", "sql:allow-execute",
    "core:window:allow-destroy",
    "notification:allow-is-permission-granted", "notification:allow-request-permission",
    "notification:allow-notify", "notification:allow-cancel", "notification:allow-get-pending",
    "haptics:allow-impact-feedback", "haptics:allow-notification-feedback", "haptics:allow-selection-feedback",
    { "identifier": "opener:allow-open-url",
      "allow": [{ "url": "tel:*" }, { "url": "mailto:*" }, { "url": "https://*" } ] }
  ]
}
```

Permisos	Para qué
<code>core:default</code> , <code>core:window:allow-destroy</code>	Lo básico; cerrar la ventana desde JavaScript tras sincronizar al salir
<code>sql:default</code> , <code>sql:allow-execute</code>	Abrir la base y leer; escribir
<code>notification:allow-*</code>	Pedir permiso, avisar y, en iOS, programar y cancelar
<code>haptics:allow-*</code>	Vibración del iPhone
<code>opener:allow-open-url</code> con tres esquemas	Abrir contactos; no <code>opener:default</code> , que además deja revelar archivos

Cuidado. Permisos **explícitos y mínimos**. Un plugin nuevo necesita su permiso aquí (lo justo) y su registro en `lib.rs`. `tauri-build` valida los permisos al compilar.

15.5 La CSP: qué puede cargar el WebView

La segunda barrera es la **Content Security Policy** (`app.security.csp` en `tauri.conf.json`): una lista de qué orígenes puede usar la página para scripts, imágenes, fuentes, conexiones... Si un ataque inyectara código en la página, la CSP le impediría cargar scripts de fuera o enviar datos a otro servidor.

Directiva	Valor	Por qué
<code>default-src</code> , <code>script-src</code>	<code>'self'</code>	Ningún script de fuera ni <code>eval</code>

Directiva	Valor	Por qué
<code>style-src</code>	<code>'self' 'unsafe-inline'</code>	Motion, GSAP y React escriben estilos en línea
<code>img-src</code>	<code>'self' data: blob:</code>	Miniaturas y arte SVG (<code>data:</code>), adjuntos y personajes (<code>blob:</code>)
<code>media-src</code>	<code>'self' blob:</code>	Música y el vídeo de Hu Tao, reproducidos desde memoria
<code>frame-src</code> , <code>object-src</code>	<code>blob:</code>	El visor de PDF de los adjuntos
<code>connect-src</code>	<code>'self' ipc: http://ipc.localhost</code>	Solo el puente con Rust
<code>base-uri</code> , <code>form-action</code>	<code>'none'</code>	No hay <code><base></code> ni formularios que se envíen

Lo notable es lo que **no** está: Google. Todas las llamadas a Drive las hace Rust, así que la CSP no se abre a internet (ADR-26, ADR-28). `devCsp` es la misma más lo que necesita Vite en desarrollo (scripts en línea y el WebSocket de la recarga en caliente).

Cuidado. Nunca `csp: null`. Si algo tuviera que cargar de otro origen, se añade a la directiva exacta, también en `devCsp`, y se documenta. Si algo «no carga» en la app nativa (vídeo, PDF, fondo), mira primero la CSP.

15.6 Secretos fuera de todo

- La credencial OAuth de Google (`google-client.json`) **no está en el repositorio**; `build.rs` la incrusta al compilar (en la CI, desde secretos).
- El **refresh token** va al **llavero del sistema** (crate `keyring`), cifrado con la cuenta del usuario.
- El **access token** solo vive en la memoria de Rust.
- El JavaScript nunca ve un token.

15.7 SQLite nativo

En la app nativa, `src/storage/eventStore.ts` abre la base con el plugin:

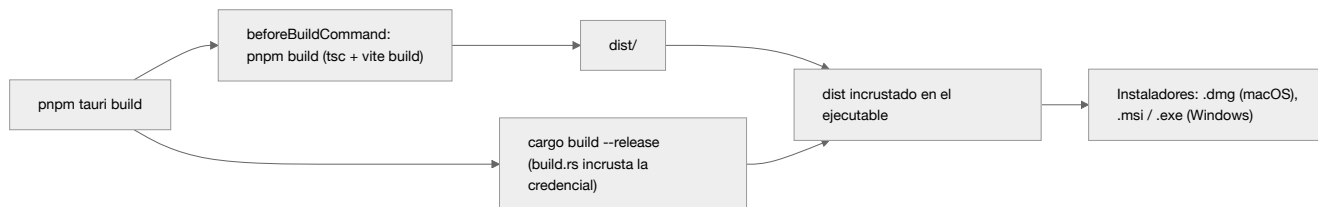
```
let sqlite: Promise<Database> | undefined;
export function sqliteDb(): Promise<Database> {
  sqlite ??= import("@tauri-apps/plugin-sql").then(({ default: Db }) =>
    Db.load("sqlite:quests.db"));
  return sqlite;
  // una conexión única, compartida por eventos y
  // binarios
}
```

- `sqlite:quests.db` se resuelve en la carpeta de datos de la app (`~/Library/Application Support/com.quests.app/` en macOS). El `import()` dinámico evita cargar el plugin en el navegador.
- No hay sistema de migraciones:** el esquema se crea con `CREATE TABLE IF NOT EXISTS` al abrir. Como el modelo es de eventos (JSON en `body`), el esquema SQL casi nunca cambia: los cambios de formato se re-

suelven con *upcasting*, no con `ALTER TABLE`.

- Con la app abierta verás `quests.db-wal` y `quests.db-shm`: SQLite en modo *write-ahead log*. **Copia o mueve los tres juntos** y solo con la app cerrada.

15.8 Compilar y empaquetar



El perfil de *release* de `Cargo.toml` reduce el tamaño (`lto = true`, `opt-level = 3`, `strip = true`, `panic = "abort"`, `codegen-units = 1`). La CI compila en macOS y Windows y deja los instaladores, **sin firmar**, como artefactos del run.

Ejercicios

Ejercicio 15.1. Añade (en una rama de pruebas) un comando `app_info` que devuelva `{ "version": "...", "os": "..." }` (pista: `env!("CARGO_PKG_VERSION")` y `std::env::consts::OS`). Regístralo y llámalo desde la consola de la app nativa con `window.__TAURI_INTERNALS__.invoke("app_info")`. ¿Hace falta un permiso nuevo? Compruébalo.

Ejercicio 15.2. Quita `"https://*"` del permiso de `opener` (en una rama de pruebas) e intenta abrir un contacto de tipo enlace en la app nativa. ¿Qué error ves y dónde?

Ejercicio 15.3. Explica qué pasaría con la seguridad si la sincronización usara `fetch` desde JavaScript con el token. Menciona la CSP, el WebView y el llavero.

Ejercicio 15.4. Lee `impl Serialize for SyncError` en `src-tauri/src/sync/mod.rs` y escribe el JSON que llega a JavaScript para `SyncError::Drive(403, "rate limit")`.

Capítulo 16. Interfaz móvil e iOS

16.1 El mismo proyecto, otro destino

Quests funciona en el **iPhone** como app nativa: es **el mismo proyecto de Tauri 2** compilado para iOS (`src-tauri/gen/apple`), con el mismo dominio, los mismos eventos, el mismo SQLite y la misma sincronización. Se descartaron una app móvil aparte (duplicaría las pantallas) y una PWA (no podría guardar el token en el llavero ni hablar con Drive sin abrir la CSP) (ADR-31).

Lo que cambia es la **interfaz**, adaptada a ~400 pt de ancho y táctil (`features/mobile`), y **una** pieza nativa: el inicio de sesión de Google (capítulo 17).

16.2 CSS primero, JavaScript solo donde hace falta

El punto de corte es **760 px** (`PHONE_MAX_WIDTH`). Cada funcionalidad pone su bloque de teléfono al final de su CSS. El JavaScript solo decide lo que el CSS no puede:

```
// src/features/mobile/phone.ts
export const PHONE_MAX_WIDTH = 760;
export const PHONE_QUERY = `(max-width: ${PHONE_MAX_WIDTH}px)`;
export const isPhone = (): boolean => window.matchMedia(PHONE_QUERY).matches;

/** Vuelve a pintar el componente al pasar de ventana ancha a teléfono. */
export function useIsPhone(): boolean {
  return useSyncExternalStore(subscribe, isPhone, () => false);
}
```

Lo decide JavaScript	Dónde
Tocar una tarjeta abre su detalle a pantalla completa	<code>App.tsx</code> (<code>isPhone()</code> → <code>openDetail</code>)
Aceptar cierra antes el detalle (el sello cae en la tarjeta)	<code>detailPrimaryAction</code>
La crónica enseña una página cada vez	<code>ChronicleModal</code>
«Toca para continuar» en vez de «Pulsa Enter»	<code>KeyHint</code>

Cuidado. El número 760 está repetido a mano en cada `@media` del CSS. Si se cambia en `phone.ts`, se cambia en todos.

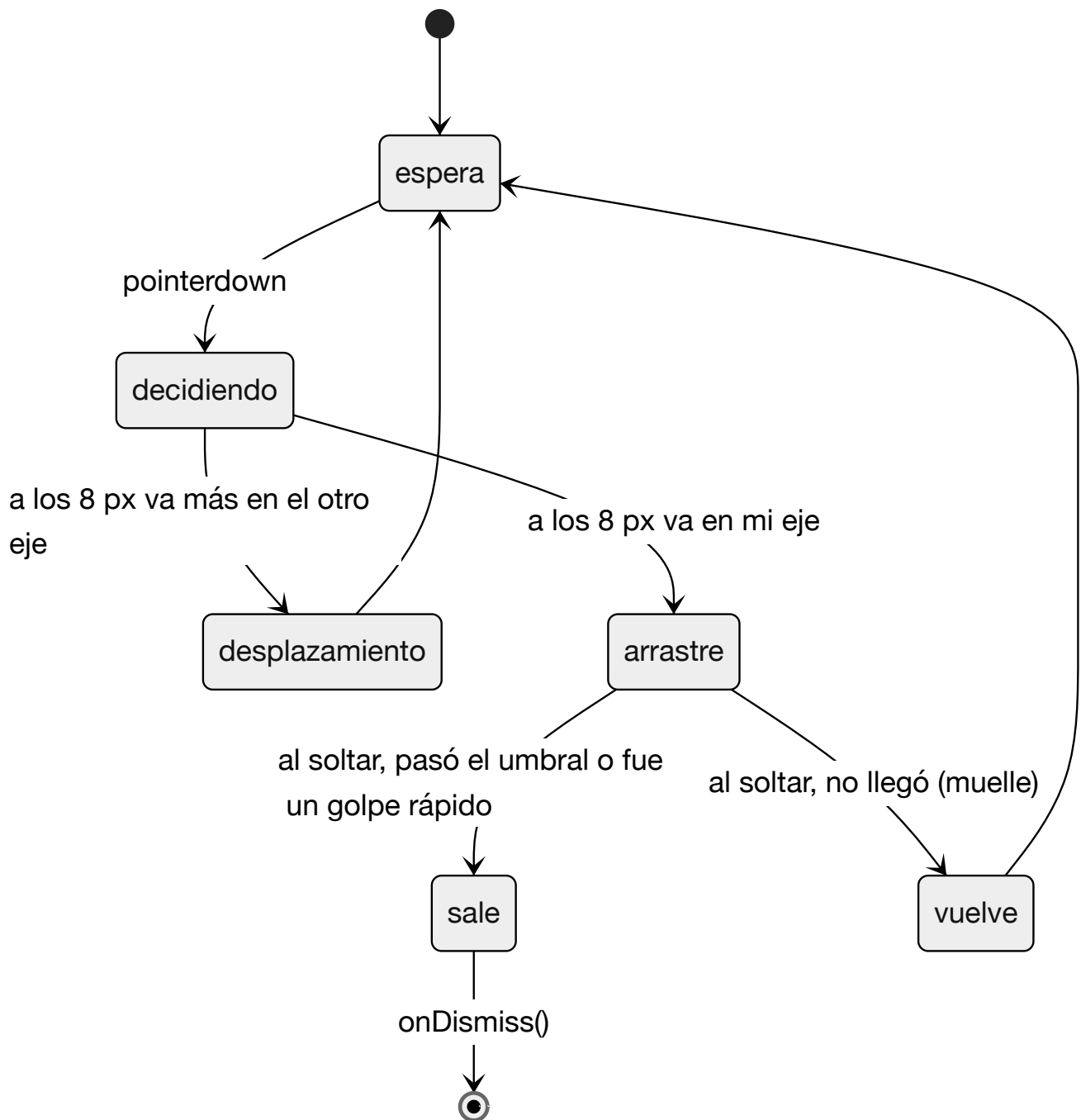
16.3 El armazón del teléfono

- **Cabecera en dos filas:** emblema, título y oro; debajo, rango, nivel y barra de XP.
- **Barra de abajo** (`MobileNav`): una cápsula de cristal (`backdrop-filter`) con Tablón · Encargos · Calendario · Menú. Su **selector** va con un muelle a la sección activa y, como en iOS, **se mantiene y se arrastra**: crece al poner el dedo, sigue al dedo estirándose con la velocidad y vibra en cada pestaña.
- **Rombo de crear** (`MobileCreate`): en el tablón abre el alta rápida; en encargos, un encargo; en el calendario, un bloque.

- **Detalle como hoja** que entra por la derecha (`.m-sheet`). En el escritorio, `.m-sheet` tiene `display: contents` y no existe a efectos de layout.
- **Ventanas a pantalla completa**: una regla de `mobile.css` sobre `.modal`.
- **Menú** con su propia distribución, como la pantalla de inicio de un gacha (`MenuPhone`).

16.4 Gestos de iOS

Un solo gesto reutilizable, `useDragDismiss` (`features/mobile/drag.ts`), con eventos de puntero y solo en el teléfono:



Gesto	Dónde	Detalle
Volver desde el borde	El detalle	Empieza a menos de 28 px del borde izquierdo; sale al 30 % del ancho
Bajar para cerrar	Alta rápida, ajustes, fichas	Solo desde el asa (SheetGrip), para no pelear con el desplazamiento interior
Deslizar una tarjeta	El tablón	A la derecha hace su acción principal (aceptar o reportar); se llena una franja dorada; umbral 45 %

El clic que llega justo al soltar (menos de 400 ms) se descarta, para no abrir lo que había debajo.

16.5 Detalles de iOS que muerden

- **Campos de 16 px como mínimo:** con menos, iOS amplía la página al tocarlos. **Botones de 40 px** o más.
- `touch-action: manipulation` (sin la espera del doble toque), sin resaltado gris al tocar ni rebote.
- Sin ampliación con dos dedos (`user-scalable=no` en `index.html`): es una app, no una página.
- `enterKeyHint` dice qué hace Intro en el teclado del teléfono («done», «go», «send»).
- Sin barras de desplazamiento y `overflow-x: hidden` donde solo se baja: un desbordamiento de 2 px bastaba para arrastrar la página de lado.
- **Vibración** con el plugin `haptics` (`src/lib/haptics.ts`), llamada desde `sfx.ts`; preferencia propia (`quests.haptics`).

16.6 Compilar para el iPhone

Preparación del Mac (una vez):

```
rustup target add aarch64-apple-ios aarch64-apple-ios-sim
xcodebuild -downloadPlatform iOS
rustup component add llvm-tools
brew install cocoapods
```

Compilar y probar:

```
pnpm tauri ios build --debug --target aarch64-sim # app para el simulador
pnpm tauri ios dev                               # simulador o iPhone, con recarga en
caliente
pnpm tauri ios dev --open                         # desde Xcode (para firmar)
pnpm iphone                                       # release firmada instalada en el iPhone
emparejado
```

- **Firma con un Apple ID gratuito:** Xcode → Settings → Accounts; el *Team ID* en `bundle.iOS.developmentTeam`; en el iPhone, confiar en el certificado. **Caduca a los 7 días** (hay que reinstalar con `pnpm iphone`; los datos se conservan).
- **Bundle ID de iOS:** `com.requenadonacarlos.quests` (`tauri.ios.conf.json`), distinto del de escritorio porque `com.quests.app` estaba registrado por otra cuenta.
- Hay arreglos para Xcode 27 ya en el repositorio (ADR-34): `swift_rs_globals()` en `build.rs`, el `PATH` del script de Xcode con `~/cargo/bin` delante y sin el `sandbox` de scripts, y las macros sin `strip` en `release`.

Cuidado. Xcode **no hereda** tus variables de entorno: lo que `build.rs` necesite leer al compilar para iOS (la credencial de Google) va en archivo.

16.7 Cómo se verifica

La interfaz de teléfono se diseña y se prueba mucho más rápido con `pnpm dev` en el navegador a **402 × 874** (el tamaño de un iPhone 17): es el mismo CSS. El simulador o el iPhone hacen falta para lo nativo: SQLite, llavero, inicio de sesión de Google, avisos y vibración. Lo que falta por probar en un iPhone real está en el «Estado actual» del README de `mobile`.

Ejercicios

Ejercicio 16.1. Abre `pnpm dev`, pon el navegador a 402 × 874 y recorre el tablón, el detalle, el menú y el mercader. Anota tres cosas que cambian solo con CSS y una que necesita JavaScript.

Ejercicio 16.2. Lee `useDragDismiss` y explica por qué se decide de quién es el gesto a los 8 px y no al primer movimiento.

Ejercicio 16.3. Añade a una ventana nueva (imaginaria) su bloque `@media (max-width: 760px)` siguiendo las normas: pantalla completa, botones de 40 px, campos de 16 px y asa si es una hoja de abajo.

Parte VIII · Sincronización

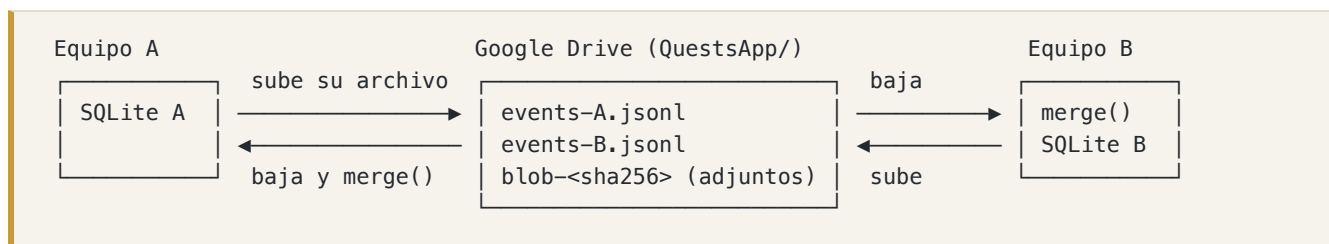
Varios equipos con el mismo progreso, sin servidor propio y sin conflictos: el Mac, Windows y el iPhone se ponen de acuerdo a través de Google Drive.

Capítulo 17. Sincronización con Google Drive

17.1 El problema y la idea

Quieres jugar en el Mac por la mañana, en el iPhone en el metro (sin conexión) y en Windows por la tarde, y que todo cuadre. Las soluciones clásicas (un servidor con una base de datos central, o subir «el estado» y que gane el último) no sirven: la primera exige infraestructura, cuentas y conexión; la segunda pierde datos.

Con event sourcing la idea es casi trivial: **cada equipo sube sus eventos y baja los de los demás**. Como los eventos son inmutables, tienen id único, se ordenan igual en todas partes y la proyección es determinista con guardas, **unir las listas basta** para que todos lleguen al mismo estado.



Para recién graduados. Es un **CRDT** de manual: un conjunto que solo crece (*grow-only set*) de eventos. La unión de conjuntos es conmutativa, asociativa e idempotente, así que da igual en qué orden y cuántas veces se fusione: todos convergen. La proyección determinista convierte ese conjunto en el estado.

17.2 Decisiones de diseño

Decisión	Alternativa descartada	Motivo
Un archivo JSONL por equipo, reescrito entero	Un archivo por evento; uno compartido	Drive no permite añadir al final de un archivo; miles de archivos serían miles de peticiones; con un archivo por equipo nunca hay dos escritores sobre el mismo (ADR-29)
Rust hace OAuth y todas las llamadas a Drive	fetch desde JavaScript	El token nunca toca el WebView y la CSP no se abre (ADR-28)
Refresh token en el llavero; access token en memoria	SQLite o un archivo	Es una credencial de Google
Permiso <code>drive.file</code> y una carpeta visible <code>QuestsApp/</code>	<code>drive.appdata</code> ; Drive entero	No requiere auditoría de Google y deja ver y copiar los datos
Archivos encontrados por <code>appProperties</code> , no por carpeta	Buscar dentro de la carpeta	Si dos equipos crean a la vez su carpeta, no se pierde nada
Binarios por SHA-256 , solo los que se usan	Toda la tabla <code>blobs</code>	No se suben huérfanos; nunca se duplican; se verifican al bajar
Ids fijos en los datos de ejemplo y lápida de quests retiradas	Ids aleatorios	Si no, dos equipos nuevos tendrían los ejemplos duplicados, y un equipo que llega tarde resucitaría lo retirado (ADR-30)

17.3 El formato: JSONL

JSONL (*JSON Lines*) es un objeto JSON por línea. Es fácil de escribir, de leer línea a línea y de tolerar fallos parciales:

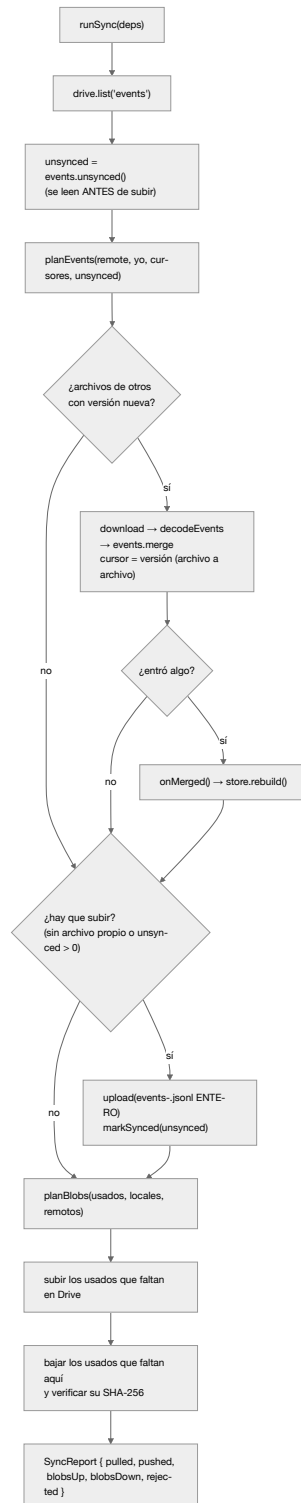
```
export function encodeEvents(events: readonly GameEvent[]): string {
  return [...events].sort(compareEvents).map((e) => JSON.stringify(e)).join("\n")
    .concat(events.length ? "\n" : "");
}

export function decodeEvents(text: string, deviceId: string) {
  const events: GameEvent[] = [];
  let rejected = 0;
  for (const line of text.split("\n")) {
    if (!line.trim()) continue;
    try {
      const e: unknown = JSON.parse(line);
      if (isEvent(e) && e.deviceId === deviceId) events.push(e); // ← solo los de ESE equipo
      else rejected++;
    } catch {
      rejected++; // una línea dañada no tumba
    }
  }
  return { events, rejected };
}
```

Idea clave. `decodeEvents` rechaza cualquier evento cuyo `deviceId` no sea el dueño del archivo. Un archivo dañado o manipulado no puede «hacerse pasar» por otro equipo.

17.4 Una sincronización completa: `runSync`

La lógica está en TypeScript (`src/features/sync/engine.ts`) y recibe **todo inyectado** (almacén de eventos, binarios, Drive, cursores, el recálculo). Así se prueba con dos o tres «equipos» en memoria y un Drive falso, y la app le pasa los de verdad.



Los detalles que hacen que sea correcto:

1. **Cursores por archivo.** Para cada archivo remoto se guarda la **version** ya leída (Drive la incrementa en cada cambio). Si no ha cambiado, no se descarga. El cursor se guarda **archivo a archivo**: si algo falla a medias, no se vuelve a leer lo ya fusionado.

```
export function planEvents(remote, deviceId, cursors, unsynced) {
  const own = remote.find((f) => keyOf(f) === deviceId);
  const download = remote.filter((f) => keyOf(f) && keyOf(f) !== deviceId &&
    cursors.files[f.id] !== f.version);
  return { download, own, upload: !own || unsynced > 0 };
}
```

2. **Lo hecho durante la subida no se pierde.** Los eventos sin subir se leen **antes** de subir y solo **esos** se marcan como subidos. Un evento creado mientras se subía queda sin marcar y va en la siguiente.
3. **Fusionar es idempotente** (INSERT OR IGNORE) y tras fusionar se recalcula todo (**rebuild**), porque los eventos de otros pueden caer en medio del historial.
4. **Binarios verificados.** El id de un binario es su SHA-256: al bajarlo se guarda, se recalcula el hash y, si no coincide, se descarta (archivo dañado en Drive).

17.5 Cuándo se sincroniza

features/sync/actions.ts decide **cuándo** (solo una a la vez):

- Al **abrir** la app (SyncWatcher → initSync).
- **Cada 5 minutos** (SYNC_EVERY_MS).
- Al **volver** a la ventana, si hace más de un minuto.
- Al **ocultarla** con algo sin subir (en el iPhone, al salir de la app: la última ocasión antes de que iOS la suspenda).
- Al **cerrarla**, esperando como mucho 8 s (CLOSE_TIMEOUT_MS); por eso existe el permiso `core:window:allow-destroy`.

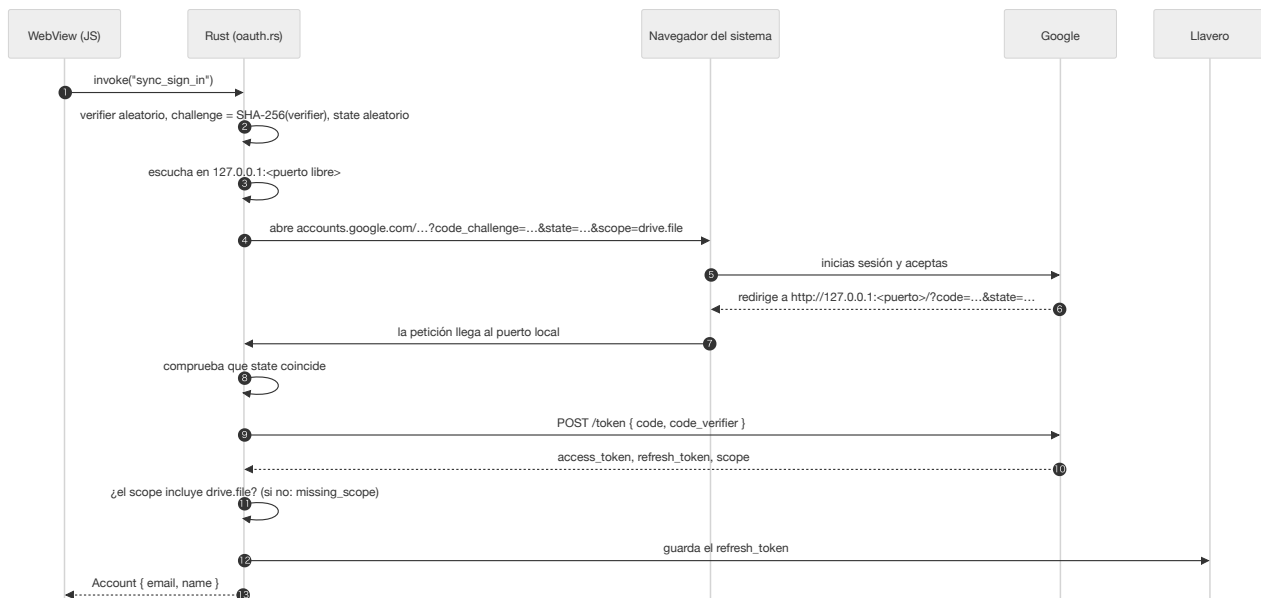
En el navegador (pnpm dev) la sincronización sale como «no disponible»: no hay Rust ni llavero.

17.6 Iniciar sesión: OAuth 2.0 con PKCE

OAuth 2.0 es el protocolo con el que una app obtiene permiso para usar tu cuenta de Google **sin conocer tu contraseña**. Google te enseña una página, aceptas y la app recibe un **código** que canjea por **tokens**:

- **Access token**: da acceso a Drive durante ~1 hora.
- **Refresh token**: permite pedir access tokens nuevos sin volver a preguntar.

Una app de escritorio no puede guardar un secreto de verdad (cualquiera puede extraerlo del ejecutable), así que se usa **PKCE** (*Proof Key for Code Exchange*): la app inventa un `verifier` aleatorio, envía a Google su hash (`challenge`) y, al canjear el código, demuestra que es la misma app enviando el `verifier`. Un tercero que interceptara el código no podría canjearlo.



En código (resumido de `src-tauri/src/sync/oauth.rs`):

```

let verifier = random_url_safe(64);
let challenge = URL_SAFE_NO_PAD.encode(Sha256::digest(verifier.as_bytes()));
let state = random_url_safe(24);
let (code, redirect) = authorize(app, &c, &challenge, &state).await?; // escritorio o iOS
// ... canjear el código con code_verifier ...
if !token.scope.as_deref().unwrap_or("").split(' ').any(|s| s == SCOPE) {
    return Err(SyncError::MissingScope); // se puede desmarcar Drive y aceptar igualmente
}
entry()?.set_password(&refresh).map_err(SyncError::keyring)?; // al llavero
  
```

El parámetro `state` protege contra respuestas que no pidió esta app (un ataque CSRF);
`access_type=offline` y `prompt=consent` aseguran que Google devuelva siempre un refresh token.

En el iPhone

Solo cambia **cómo** se obtiene el código (ADR-32). No se puede escuchar en un puerto local; se usa la **hoja de inicio de sesión del sistema** (`ASWebAuthenticationSession`) a través de un plugin propio en Swift (`src-tauri/plugins/web-auth`). Google redirige a un esquema propio (el id del cliente al revés: `com.googleusercontent.apps.<id>:/oauth2redirect`), la hoja se cierra y devuelve la URL **a Rust**, que comprueba el `state` y canjea el código igual que en el escritorio. El plugin no expone comandos al JavaScript: el código nunca pasa por el WebView. Cerrar la hoja sin terminar devuelve `cancelled` y se vuelve a «Conectar» sin aviso.

17.7 Renovar el token

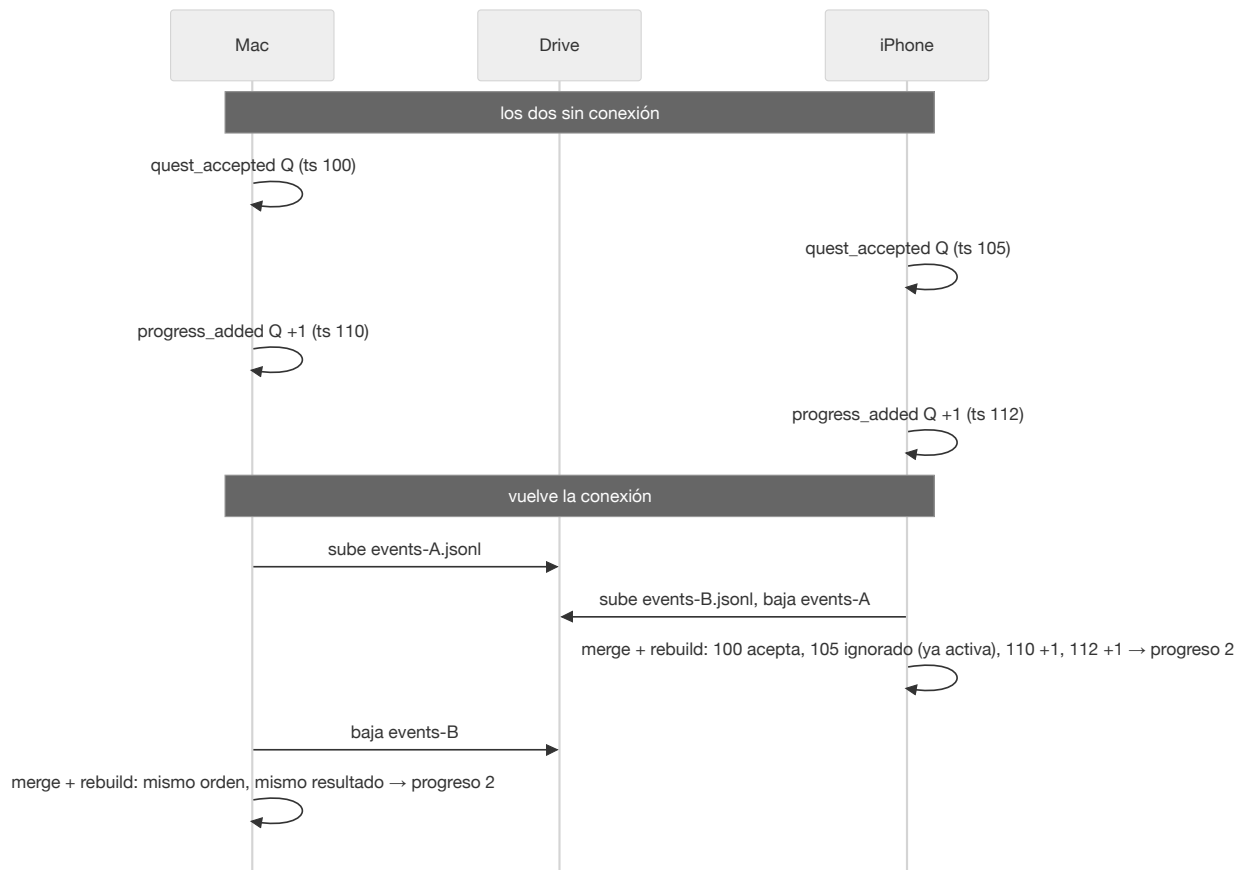
```
pub async fn access_token(http: &request::Client, cache: &Mutex<Option<AccessToken>>) ->
Result<String, SyncError> {
    let mut guard = cache.lock().await;
    if let Some(t) = guard.as_ref() {
        if t.expires_at > Instant::now() + Duration::from_secs(60) {
            return Ok(t.value.clone()); // el de memoria aún vale
        }
    }
    let refresh = stored_refresh_token()?.ok_or(SyncError::SignedOut)?;
    // POST /token { grant_type: refresh_token } ...
    // Si Google dice invalid_grant (caducado o retirado): se borra del llavero → SignedOut
}
```

El `Mutex` evita que dos peticiones simultáneas pidan dos tokens a la vez. Mientras la app de Google Cloud esté en modo «Testing», la sesión **caduca cada 7 días** (publicarla «In production» está en la hoja de ruta).

17.8 Errores con un código estable

code (Rust)	Qué pasa	Qué se ve
<code>not_configured</code>	Compilada sin credencial	Nube apagada: «sin configurar»
<code>signed_out</code>	No hay sesión o caducó	Se vuelve a «Conectar» y avisa
<code>consent_denied,</code> <code>missing_scope</code>	No se dio permiso o se desmarcó Drive	Aviso para volver a conectar
<code>cancelled</code>	iOS: hoja cerrada sin terminar	Nada
<code>timeout</code>	No se volvió del navegador en 5 minutos	Aviso
<code>network</code>	Sin conexión	Punto rojo; en las automáticas no avisa y reintenta
<code>drive,</code> <code>auth,</code> <code>keyring,</code> <code>other</code>	Fallo de Google o del sistema	Punto rojo y el motivo; detalle en la salida de Rust (<code>[sync]</code> ...)

17.9 Por qué converge: un caso completo



Las piezas que intervienen: **ids únicos** (no se duplican eventos), **orden total** (ts, id) (todos reproducen igual), **guardas** (la segunda aceptación se ignora), **deltas** (los dos +1 suman) e **idempotencia** (fusionar dos veces no cambia nada). Quita cualquiera y deja de converger.

17.10 Lo que falta y los límites

- El archivo de cada equipo se **reescribe entero** con cada novedad: cuando pase de unos MB habrá que partirlo por meses o compactarlo.
- Los binarios que dejan de usarse no se borran de Drive.
- Sin verificar con Drive de verdad: los adjuntos (solo en los tests), la sincronización al cerrar en iOS, la caducidad a los 7 días (lista en el README de sync).

Ejercicios

Ejercicio 17.1. Lee `src/features/sync/engine.test.ts`. ¿Cómo está construido el «Drive falso»? Escribe en pseudocódigo un test nuevo: tres equipos, cada uno completa una quest distinta sin conexión, sincronizan en el orden C, A, B, A, C y todos tienen la misma XP.

Ejercicio 17.2. ¿Qué pasaría si `runSync` marcara como subidos **todos** los eventos al terminar la subida en lugar de solo los leídos antes? Construye una secuencia temporal que pierda un evento.

Ejercicio 17.3. Explica con tus palabras qué ataque evita el parámetro `state` y cuál evita PKCE. ¿Por qué un cliente de escritorio no puede confiar en su *client secret*?

Ejercicio 17.4. Dos equipos nuevos arrancan vacíos, cada uno siembra los ejemplos y sincronizan. ¿Por qué no salen diez quests de ejemplo? ¿Y si el Mac retira el dragón antes de sincronizar y el iPhone no? Busca `deletedQuests` y la ADR-30.

Parte IX · Tests y rendimiento

Cómo se demuestra que todo lo anterior funciona, cómo se depura cuando no, y la arquitectura vista desde arriba.

Capítulo 18. Tests, debugging y arquitectura

18.1 Vitest

Los tests usan **Vitest**, un ejecutor de tests compatible con la API de Jest e integrado con Vite (entiende TypeScript sin configuración extra):

```
pnpm test           # todos (vitest run), en un par de segundos
pnpm exec vitest     # modo vigilante: repite al guardar
pnpm exec vitest projection  # solo los archivos que coinciden
```

La configuración (`vitest.config.ts`) tiene una decisión importante:

```
export default defineConfig({
  test: {
    include: ["src/**/*.test.ts"],
    env: { TZ: "Europe/Madrid" }, // zona horaria FIJA
  },
});
```

«Hoy», los plazos, la medianoche de una fecha límite y los cambios de hora dependen de la zona horaria. Fijarla hace que los tests den lo mismo en el Mac del propietario, en la CI de Windows y en cualquier otro equipo.

Los tests van **junto al módulo** que prueban (`model.ts` → `model.test.ts`) y usan **marcas de tiempo fijas** (`T0 = Date.UTC(2026, 0, 1)`), nunca `Date.now()`.

18.2 Anatomía de un test del dominio

```
import { describe, expect, it } from "vitest";
import { project } from "../projection";
import { questDef, withMeta } from "../test/streams";

const created = { type: "quest_created", quest: questDef("q", { reward: { xp: 150, gold: 30 } }) } as const;
const accept = { type: "quest_accepted", questId: "q" } as const;
const complete = { type: "quest_completed", questId: "q", reward: { xp: 150, gold: 30 } } as const;

describe("contabilidad de project()", () => {
  it("una quest completada suma su recompensa una sola vez", () => {
    const st = project(withMeta([created, accept, complete, complete])); // Arrange + Act
    expect(st.player).toMatchObject({ xp: 150, gold: 30, completedCount: 1 }); // Assert
    expect(st.quests.get("q")?.status).toBe("done");
  });
});
```

- `questDef(id, extra)` construye una definición válida con valores por defecto.
- `withMeta(bodies)` añade `id`, `deviceId` y un `ts` con **un minuto** entre eventos, empezando en `T0`.

- Como `project` es una función pura, el test no necesita base de datos, ni React, ni Tauri: datos dentro, estado fuera.

Idea clave. Que el dominio sea puro no es una manía estética: es lo que hace que cientos de tests corran en un par de segundos y que un fallo se pueda reproducir con una lista de eventos.

18.3 Los tipos de test del proyecto

Tipo	Qué comprueba	Ejemplo
Unitario de reglas	Una función pura con casos concretos y sus bordes	Fases del pomodoro, plazos con cambio de hora, precios, <code>parseQuick</code>
Guardas y fusión	Que un evento imposible o duplicado no rompa nada, en cualquier orden	«dos dispositivos que completan la misma quest cobran una vez, lleguen en el orden que lleguen»
Invariantes sobre historiales aleatorios	Propiedades que deben cumplirse siempre	«la XP es la suma exacta de las recompensas aceptadas», «el oro solo baja al comprar»
Equivalencia	Dos caminos dan exactamente lo mismo	Snapshot + cola = reproducir todo; incremental = completo
Integración	El store y las acciones de verdad, en un navegador simulado	<code>src/store/game.test.ts</code> (happy-dom), <code>merchant/actions.test.ts</code>
Convergencia	Varios equipos con un Drive falso llegan al mismo estado	<code>src/features/sync/engine.test.ts</code>

Tests basados en propiedades: `randomStream`

En vez de escribir a mano cada historia posible, `src/test/streams.ts` genera **historiales aleatorios pero reproducibles** (con semilla) que mezclan **todos los tipos de evento**, también imposibles (completar sin aceptar, duplicados), antiguos (sin `v`) y de una versión futura:

```
// Simplificado de src/domain/projection.test.ts
it("la XP es la suma exacta de las recompensas aceptadas", () => {
  for (const seed of ["a", "b", "c" /* ... */]) {
    const events = randomStream(seed, 400);
    const st = project(events);
    // recalcular a mano qué quest_completed y temporal_completed pasaron sus guardas...
    expect(st.player.xp).toBe(esperado);
  }
});
```

Si un invariante falla, la semilla reproduce exactamente el historial que lo rompe.

Cuidado. Si añades un tipo de evento, añádelo a `randomStream`, también con casos imposibles. Si no, los invariantes y los tests del snapshot no lo ejercitan nunca.

Integración con happy-dom

`game.test.ts` arranca el store **tal cual lo usa la app** sobre un navegador simulado (happy-dom): `localStorage` de verdad, snapshot de verdad, acciones de verdad. Solo el sonido es falso (en Node no hay `AudioContext`):

```
// @vitest-environment happy-dom
vi.mock("../lib/sfx", async () => (await import("../test/sfxMock")).sfxMock());

async function boot() { // «abrir la app otra vez»: módulos nuevos, mismo
  localStorage
  vi.resetModules();
  const { useGame } = await import("../game");
  await useGame.getState().init();
  return { useGame, g: () => useGame.getState() };
}

const truth = () => canonical(project(stored())); // reproducir todo lo
guardado
const consistent = (b) => expect(canonical(b.g().state)).toBe(truth()); // ¿el store dice lo
mismo?
```

`canonical` serializa el estado con las claves ordenadas y `Map/Set` como listas, para comparar dos estados sin que importe el orden de las claves.

18.4 Qué no tiene tests (y se dice)

No hay tests del almacén de binarios real (IndexedDB y SQLite), de los adjuntos de las acciones de encargos, de los componentes React ni de las animaciones. Está en la **deuda técnica** del informe. La norma: si tu tarea toca uno de esos puntos, resuélvelo o al menos no añadas más casos sin probar.

18.5 La puerta de calidad: CI y comprobaciones

Antes de dar algo por terminado (y la CI lo repite en macOS y Windows en cada push):

```
npx tsc --noEmit      # tipos
pnpm test             # tests
pnpm build            # build del frontend
pnpm docs:check       # documentación al día
```



`pnpm docs:check` (`scripts/docs.mjs`) comprueba que las tablas generadas de `docs/INDEX.md` están al día, que cada funcionalidad tiene su README con la plantilla, que los enlaces y los eventos citados existen, que no hay cifras que caducan fuera del historial y que ningún documento enseña comandos que borrarían datos del propietario.

Y además: **probarlo en ejecución** (no solo compilar) en el navegador a escritorio y a 402×874 , y en la app nativa si toca Rust o algo nativo. Y **decir con honestidad** qué no se probó (el sonido, Windows, la app empaquetada...).

18.6 Debugging: dónde mirar

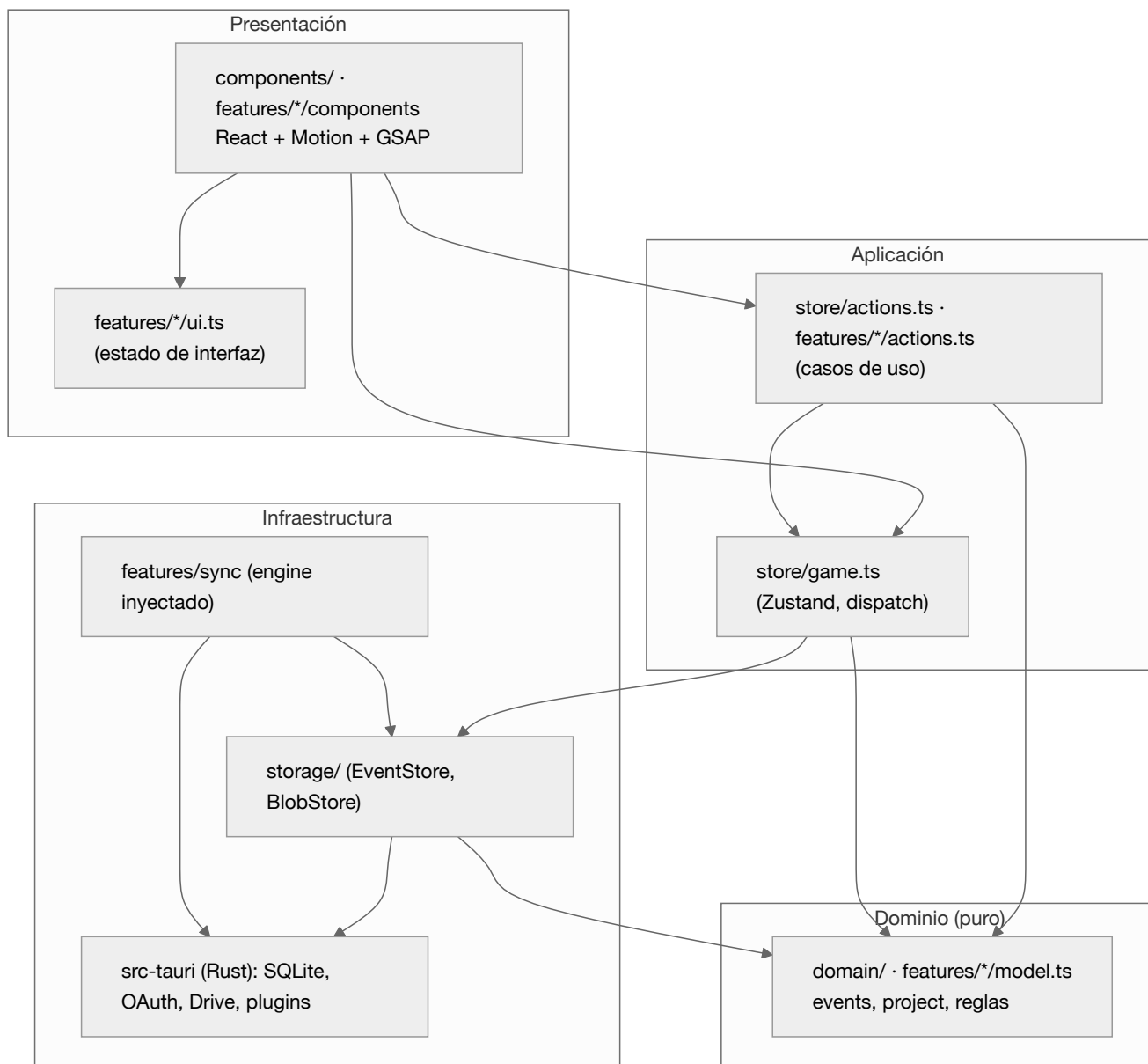
Síntoma	Dónde mirar	Herramienta
Un componente no se actualiza o se repinta demasiado	Los selectores y el <code>memo</code>	React DevTools («Highlight updates», «Profiler»)
El estado no es el esperado	El historial de eventos	Consola: <code>JSON.parse(localStorage["quests.events"])</code> (navegador) o <code>sqlite3</code> de solo lectura (nativo)
«Tras reiniciar, el estado cambia»	El snapshot y <code>PROJECTION_VERSION</code>	Aviso de <code>verifyAgainstFull</code> en la consola (desarrollo)
Una acción no hace nada	La validación de la acción y la guarda de la proyección	Un <code>console.log</code> en la acción; un test con <code>withMeta</code> que reproduzca el caso
Algo no carga en la app nativa (vídeo, PDF, fondo)	La CSP de <code>tauri.conf.json</code>	Inspector web de la ventana nativa (clic derecho → Inspeccionar en desarrollo)
Falla la sincronización	Los errores con <code>code</code> y la salida de Rust	Terminal de <code>pnpm tauri dev ([sync] ...)</code>
Un plugin no responde	Su permiso en <code>capabilities/default.json</code> y su registro en <code>lib.rs</code>	<code>cargo check</code> en <code>src-tauri/</code>
El teclado «no hace nada»	Algún <code>...Busy()</code> devuelve verdadero	Consola: llamar a cada <code>...Busy()</code>
Una animación deja un estado raro	El patrón GSAP (ref anterior, <code>gsap.set</code> , <code>tl.kill()</code>)	Comentar la línea de tiempo y ver si React pinta bien

Reproducir un fallo con eventos

La gran ventaja del event sourcing para depurar: **cualquier estado se reproduce con su lista de eventos**. Si el propietario ve algo raro, se puede exportar (de una **copia**) la lista de eventos, cargarla en el `localStorage` de pruebas del navegador o en un test, y reproducir el fallo de forma determinista.

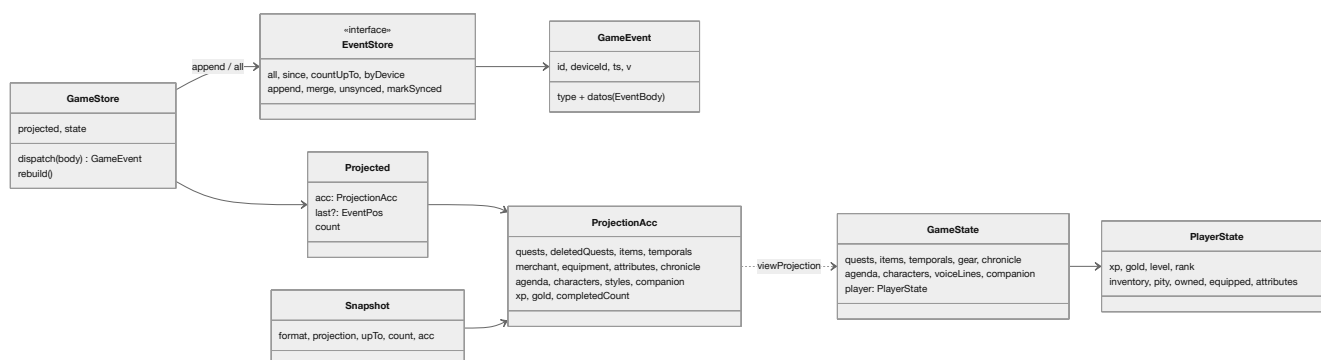
Cuidado. Lo que depende del tiempo no se prueba esperando: se **inyectan eventos con ts en el pasado** en el `localStorage` de pruebas (o en un test con `withMeta(..., start)`).

18.7 La arquitectura vista desde arriba



El **sentido de las dependencias** es norma: `components → store → domain ← storage`. El dominio no importa nada de React, Zustand, Tauri, `window` ni el DOM, y no llama a `Date.now()` ni a `Math.random()`.

Diagrama de clases simplificado



Cuidado. El diagrama de clases **oficial** del proyecto (`docs/img/diagrama-clases.png`) se redibuja cada vez que cambia un tipo del dominio, siguiendo `docs/runbooks/redibujar-diagramas.md`. El de arriba es una simplificación para estudiar.

Las decisiones que sostienen el proyecto

Cada decisión de arquitectura está en un **ADR** (*Architecture Decision Record*) en `docs/decisions/`, con lo descartado y por qué, para no reabrirla sin un motivo nuevo. Las fundamentales:

ADR	Decisión
ADR-01	Tauri frente a Electron (tamaño y backend nativo en Rust)
ADR-02	Event sourcing
ADR-03	Local-first: los datos viven en el equipo; la nube solo sincroniza
ADR-16	Snapshot de la proyección
ADR-24, ADR-25	Eventos versionados y reloj lógico híbrido
ADR-28, ADR-29	OAuth y Drive en Rust; un JSONL por equipo
ADR-54	Copia selectiva en <code>dispatch</code> (Immer)

Atributos de calidad

Atributo	Cómo lo consigue la arquitectura
Corrección	Dominio puro, guardas, tests de invariantes y de equivalencia
Funcionar sin conexión	Local-first: todo funciona sin red; la sincronización es oportunista
Sincronización sin conflictos	Eventos inmutables, ids únicos, HLC, deltas, idempotencia
Evolución	Versión de eventos y <i>upcasting</i> ; una carpeta por funcionalidad con su README
Seguridad	CSP estricta, permisos mínimos, tokens fuera del JavaScript
Rendimiento	Snapshot, copia selectiva, selectores finos, <code>memo</code> , partículas fuera de React
Portabilidad	Tauri (macOS, Windows, iOS) con el mismo código

Ejercicios

Ejercicio 18.1. Escribe un test que compruebe que **abandonar** una quest en curso con progreso deja el progreso a cero y que volver a aceptarla no lo recupera.

Ejercicio 18.2. Escribe un test de **invariante**: para 20 semillas de `randomStream(seed, 300)`, ninguna quest tiene `progress` mayor que el `target` de su objetivo.

Ejercicio 18.3. En `game.test.ts`, busca cómo se simula «cerrar y volver a abrir la app». ¿Por qué hace falta `vi.resetModules()`?

Ejercicio 18.4. Elige un atributo de calidad de la tabla y propón un cambio que lo empeoraría sin que fallara ningún test. ¿Qué test añadirías para protegerlo?

Parte X · Ejercicios y guía de extensión

Un recorrido práctico de principio a fin, la guía para añadir una funcionalidad y el glosario.

Capítulo 19. Ejercicios prácticos por capítulo

Cada capítulo termina con sus ejercicios. Aquí tienes un **plan de estudio** que los ordena y tres **proyectos guiados** que los combinan.

19.1 Plan de estudio

Semana	Capítulos	Ejercicios clave
1	1–4	1.2, 2.1, 2.2, 3.2, 4.1
2	5–9	5.1, 6.2, 7.2, 8.1, 9.2, 9.4
3	10–13	10.1, 11.1, 12.1, 13.1
4	14–18	14.1, 15.1, 16.1, 17.1, 18.2

19.2 Proyecto guiado A: un evento nuevo, `quest_noted`

Objetivo: permitir añadir una nota breve a una quest en curso, que se sincronice.

1. Rama `feature/notes` desde `main` actualizada.
2. `src/features/notes/` con su `README.md` (plantilla `docs/templates/feature-readme.md`).
3. `events.ts`: `{ type: "quest_noted"; questId: string; text: string }`; súmalo a `EventBody`.
4. `model.ts`: `cleanNote(text)` (recorta a 200 caracteres; vacío = no vale).
5. Proyección: un `case` con **guarda** (la quest existe y no está terminada) que guarde la nota. Como añade un campo a lo que da la proyección, **sube** `PROJECTION_VERSION`.
6. `actions.ts`: `noteQuest(id, text)` valida y llama a `dispatch`; ofrece «Deshacer» (añade el tipo a `UNDOABLE`).
7. Textos en `i18n.ts` (es + ja), montados en los diccionarios.
8. Añade el evento a `randomStream` y escribe tests: guarda, duplicados, orden.
9. `tsc`, `pnpm test`, `pnpm build`, `pnpm docs:index` y `pnpm docs:check`; pruébalo en el navegador también a `402 × 874`.

Solución orientativa. Compara tu trabajo con `features/checklist` (evento propio por elemento, guarda de quest activa) y con el runbook `docs/runbooks/migrar-evento.md`.

19.3 Proyecto guiado B: algo calculado, sin eventos

Objetivo: un contador de «días seguidos con alguna quest completada» en la crónica.

- **Sin eventos nuevos:** se calcula de `chronicle.entries` (como las rachas o los atributos).
- Función pura en `model.ts` con `now` como parámetro; tests con `withMeta` y fechas que crucen un cambio de hora.
- Pregúntate si va en el acumulador (si necesita lo que ya no existe, como quests retiradas) o al pintar.

19.4 Proyecto guiado C: un comando nativo

Objetivo: mostrar en los ajustes el tamaño de `quests.db`. Comando Rust que lea los metadatos del archivo (sin abrirlo para escribir), registro en `lib.rs`, llamada con `invoke` y texto traducido. Verifica en `pnpm tauri dev` y explica por qué no hace falta abrir la CSP.

19.5 Guía de extensión: lista de comprobación

- ☐ La lógica está en el dominio o en un `actions.ts`, no en componentes.
- ☐ Toda escritura pasa por `dispatch`; los eventos nuevos tienen guardas y están en `randomStream`.
- ☐ Si cambia el resultado de `project()` para eventos guardados, `PROJECTION_VERSION` subida; cambios de formato con *upcaster*.
- ☐ Funcionalidad nueva en `src/features/<nombre>/` con su README.
- ☐ Textos en `es` y `ja`; archivos estáticos en `public/`.
- ☐ Revisado a 402 × 874 y en japonés.
- ☐ `tsc`, `pnpm test`, `pnpm build` y `pnpm docs:check` en verde; probado en ejecución; dicho lo no probado.
- ☐ Rama propia; commit y merge solo con el visto bueno del propietario.

Capítulo 20. Glosario y diagramas Mermaid

20.1 Glosario

Término	Significado
Acción	Caso de uso: valida, emite eventos con <code>dispatch</code> y reacciona (<code>actions.ts</code>)
Acumulador	Estado a medio calcular (<code>ProjectionAcc</code>); lo que guarda el snapshot
ADR	Registro de una decisión de arquitectura (<code>docs/decisions/</code>)
Append-only	Almacén en el que solo se añade; nunca se modifica ni se borra
Blob / binario	Contenido de un archivo; vive en el <code>BlobStore</code> , no en los eventos
Capability	Permiso explícito de Tauri para que el <code>WebView</code> use algo nativo
Compartición estructural	Copiar solo lo que cambia y reutilizar el resto (<code>Immer</code>)
CSP	Content Security Policy: qué puede cargar el <code>WebView</code>
CRDT	Estructura replicada que converge sin coordinación (aquí, un conjunto de eventos que solo crece)
Delta	Evento que lleva un cambio (+1) y no un valor absoluto
Direccionado por contenido	La clave es el hash del contenido (SHA-256)
dispatch	Único punto de escritura del juego (<code>src/store/game.ts</code>)
Dominio puro	Código sin <code>React</code> , <code>Tauri</code> , <code>Date.now()</code> ni <code>Math.random()</code>
Easing	Curva de aceleración de una animación
Event sourcing	Guardar los hechos y calcular el estado a partir de ellos
Guarda	Condición de la proyección que ignora un evento imposible
HLC	Reloj lógico híbrido: ordena eventos de varios equipos (<code>nextTs</code>)
HMR	Recarga en caliente de <code>Vite</code>
i18n	Internacionalización
Idempotente	Hacerlo dos veces da lo mismo que una
IPC	Comunicación entre el <code>WebView</code> y el núcleo <code>Rust</code>
JSONL	Un JSON por línea; formato de los archivos de sincronización
Local-first	Los datos viven en el equipo; la nube solo sincroniza
Lockfile	<code>pnpm-lock.yaml</code> : versiones exactas de todas las dependencias
OAuth / PKCE	Protocolo de autorización / prueba de que quien canjea el código es quien lo pidió
Optimista	Actualizar la interfaz antes de confirmar la escritura en disco
Pity	Contador que garantiza una rareza alta tras muchas tiradas sin ella
PRNG con semilla	Generador «aleatorio» que da siempre la misma serie para la misma semilla
Proyección	Función que calcula el estado reproduciendo los eventos (<code>project</code>)

Término	Significado
Quest en reserva	Quest de un encargo sin aceptar: existe pero no se puede aceptar
Selector	Función que elige qué parte del store sigue un componente
Snapshot	Copia guardada del acumulador; una caché
Unión discriminada	Tipo con varias formas distinguidas por un campo (type)
Upcasting	Convertir datos antiguos al formato actual al leerlos
WebView	Motor web del sistema que pinta la interfaz

20.2 Atlas de diagramas

Diagrama	Capítulo
La app en una página (arquitectura general)	Cómo leer este tutorial
Herramientas del entorno	1
Arranque con <code>pnpm tauri dev</code> e <code>init()</code>	2
Árbol de componentes	3
Store: juego frente a interfaz	4
Ciclo del event sourcing y fusión sin duplicar XP	5
<code>nextTs</code> y <i>upcasting</i>	6
Implementaciones de <code>EventStore</code> ; binarios	7
Proyección completa frente a incremental	8
Compartición estructural y arranque con snapshot	9
Modelo de quest, pomodoro y ciclo de vida	10
Curva de XP y economía	11
Aceptar, reportar y flujo de un encargo	12
Carpeta de funcionalidad y mapa de funcionalidades	13
Web Audio, línea de tiempo del sello y patrón GSAP	14
Procesos de Tauri y empaquetado	15
Gesto <code>useDragDismiss</code>	16
<code>runSync</code> , OAuth con PKCE y convergencia	17
CI, capas y diagrama de clases	18

Un último diagrama, el resumen de todo el tutorial:

